

TPM Base - BRD & Code Prompts

CollMind TPM - Comprehensive Product and Module Definition Document

1. What is CollMind TPM? (General Information)

CollMind TPM is a next-generation SaaS (Software as a Service) platform designed to digitally transform the end-to-end Trade Promotion Management (TPM) processes for companies in the Fast-Moving Consumer Goods (FMCG) sector.

Tagline: Collaborative Intelligence for Promotion Excellence

What Problem Does It Solve?

Our platform targets the most significant "pain points" for companies in the sector:

Data Chaos: Prevents planning, budgeting, and results from being lost across dozens of different Excel files, emails, and presentations.

Profitability Loss: Calculates the true Return on Investment (ROI) of promotions in real-time, preventing investment in unprofitable campaigns.

Uncontrolled Spending: Provides instant, proactive budget tracking to prevent end-of-quarter surprises and budget overruns.

Operational Inefficiency: Eliminates time lost to manual data entry and departmental coordination issues.

2. Core Value Proposition (Advantages)

CollMind TPM unifies all stakeholders (Sales, Marketing, Finance) on a single, central, and intelligent platform. It provides companies with:

Strategic Clarity: Instant simulation of all KPIs (ROI, Profit, Volume) allows for data-driven decisions, not guesses.

Financial Control: Real-time budget tracking and proactive alerts prevent overspending and provide full financial predictability.

Operational Efficiency: Eliminates manual data entry, endless email chains, and version control issues by digitizing the entire workflow.

3. Roles & Core Business Workflow

The platform is built around four key user roles that collaborate in a seamless workflow.

3.1. Role Definitions

Admin (The Architect):

Purpose: To configure the entire system and set the rules.

Key Responsibilities: Manages all master data (SKUs, Customers, CPLs, FUs), defines users and roles, sets permissions (e.g., which Planner manages which CPL/Category), and configures the KPI calculation engine (formulas, RAG status).

Planner (The Executor):

Purpose: To create, manage, and optimize promotion plans.

Key Responsibilities: Creates new plans, adds products (FUs/SKUs), enters planned volumes and tactic spends, analyzes real-time KPI feedback (ROI, GP), and submits the final plan for approval.

Category Manager (CM) (The Strategist):

Purpose: To ensure all plans align with the company's brand, category, and profitability strategy.

Key Responsibilities: Reviews plans submitted by Planners, analyzes the KPI summary (especially ROI and RAG status), and either Approves the plan or Rejects it with comments for revision.

Finance Manager (FM) (The Controller):

Purpose: To own and control the promotion budget.

Key Responsibilities: Defines the budgets for different periods/categories/channels, monitors budget utilization in real-time via the reporting dashboard, and analyzes the financial performance of all approved promotions.

3.2. Role-Based Workflow

Step 1 (Admin - Setup): The Admin configures the system by setting up users, defining master data (SKUs, CPLs, FUs), and assigning granular permissions (e.g., "Planner Ali Veli can manage 'Dairy' and 'Beverage' categories only for the 'Carrefour CPL'").

Step 2 (Planner - Plan): The Planner creates a "New Plan." The system, using the permissions set by the Admin, automatically shows only the CPLs and Categories they are authorized to manage. When the Planner opens the plan, the system also displays only the Tactics (e.g., "CPP On-Invoice %") that the Admin has approved for that specific Category and Channel.

Step 3 (Planner - Submit): Once the plan is optimized and profitable, the Planner adds comments and clicks "Submit for Approval."

Step 4 (CM - Review): The Category Manager (who is authorized for that specific category) receives a notification. They review the plan's overall KPI summary. If the ROI is good (Green RAG status), they Approve it. If it's unprofitable (Red RAG status), they Reject it with a comment (e.g., "ROI is too low, please re-evaluate display fee").

Step 5 (Finance - Monitor): Once approved, the plan's total spend is automatically deducted from the budget set by the Finance Manager, who monitors this activity on their dashboard.

4. Main Application Screens (Detailed Explanation)

4.1. Planner: Plan List Screen (Home Screen)

Purpose: To provide the Planner with a single view to see, manage, and filter all their responsible promotion plans.

Functions:

Plan Table (Grid): Lists all plans with key headers: Plan Name, Status (Draft, Pending Approval, Approved, Rejected), CPL, Category, and Start Date.

Advanced Filtering: Dropdown menus above the list allow for instant filtering of plans by Status, CPL, or Category.

Create New Plan: A + New Plan button opens a modal to enter basic information: Plan Name, CPL, Category, and Date Range.

Access: A View Details button on each plan row navigates to the "Detailed Planning Screen."

4.2. Planner: Detailed Planning Screen (Grid)

Purpose: The heart of the promotion planning. This is the main workspace for creating, simulating, and optimizing a plan.

Functions:

Grand Totals Panel: A summary panel at the top of the screen showing real-time, plan-wide metrics like Total Planned Turnover (TO), Total Spend, and Overall GP ROI %.

Hierarchical Grid: An Excel-like, expandable (+) table.

FU (Forecasting Unit) Row: The main planning row (e.g., "Danone Activia Yogurt 4x100g"). Promotion tactics are entered at this level.

SKU (Product) Row: The detailed child rows (e.g., "SKU: Strawberry") nested under an FU. Planned volume is entered at this level.

Data Entry & Automatic Calculation:

The Planner enters Planned Volume at the SKU level. This value automatically rolls up, updating the total volume for the parent FU.

The Planner enters promotion spends (e.g., CPP On-Invoice %, Display Fee (\$)) at the FU level. This value is automatically distributed down to all child SKUs based on their Base Volume ratio.

Real-Time KPI Columns: The grid displays real-time calculated KPIs for every single FU and SKU row, such as iVol (Incremental Volume), Planned GP (Gross Profit), and GP ROI %.

RAG Status: Based on the GP ROI % of each row, a visual RAG (Red/Amber/Green) color indicator shows the financial health of that specific item.

Other Functions: + Add FU (with search), Add Comment (to leave notes for the CM), Delete Row, and Submit for Approval buttons.

4.3. Category Manager (CM): Approval Screen

Purpose: To review, and then approve or reject plans submitted by Planners based on strategic fit and profitability.

Functions:

Task List: Lists only plans with the status Pending Approval for the categories the CM is responsible for.

KPI Summary: Clicking a plan displays its "Grand Totals Panel" (ROI, Spend, Profit) for a quick financial overview.

Comments: Read comments left by the Planner and add new ones.

Action Buttons: Approve (sets plan status to 'Approved') and Reject (sets plan status to 'Rejected' and sends it back to the Planner).

4.4. Finance Manager (FM): Reporting Dashboard

Purpose: To monitor all promotion spending from a financial perspective, track budget compliance, and report to upper management.

Functions:

Comprehensive Filtering: Filters all Approved plan data by Channel, CPL, Month/Year.

Budget Utilization: Progress bars showing On-Invoice and Off-Invoice spending against the total budget.

Tactic Spend Distribution: A Pie Chart showing the distribution of total spending across different promotion types.

Monthly Spend Trend: A Line Chart illustrating spending trends throughout the year.

5. Admin Panel Screens (Detailed Explanation)

5.1. Dashboard

Purpose: To monitor the overall health and activity of the system.

Functions: Displays summary metrics (Total Users, Total FUs) and a "Recent Activity" feed from the Audit Log.

5.2. User Management

Purpose: To manage all users and their roles.

Functions: Standard user CRUD (Create, Read, Update, Delete). Its most important function is the Set Permissions button on each user row.

5.3. Integrated Permissions (Modal Window)

Purpose: To define exactly what data each user can see and manage, as per our detailed Canvas.

Functions: Opened via the "Set Permissions" button:

For Planners: A granular, rule-based assignment screen. The Admin selects a specific Planner, then adds authorization rules. Each rule links a specific CPL (e.g., 'Carrefour CPL') to a specific set of Categories (e.g., ['Dairy', 'Beverages']). This ensures a Planner can manage 'Dairy' for Carrefour but only 'Beverages' for Migros.

For Category Managers: A straightforward assignment screen where the Admin checks off the Categories (e.g., 'Dairy', 'Snacks') that the CM is responsible for approving.

5.4. Master Data Modules (SKU, Customer, UOM)

Purpose: To manage the foundational building blocks of the system.

SKU Management: A standard CRUD screen to manage all individual products. Form fields include SKU Name, Brand, Category, List Price, COGS, Status.

Customer Management: A standard CRUD screen to manage all individual customers/stores. Form fields include Customer Name, Channel, City.

UOM Management: A simple CRUD screen to define Units of Measure (e.g., Code: CS, Name: Case).

5.5. Relational Master Data (FU & CPL Management)

Purpose: To create the core planning groups (FUs and CPLs) by linking master data together, as defined in our Admin Canvas.

FU (Forecasting Unit) Management:

Functionality: An advanced, 2-step wizard.

Step 1: Define the FU Code and FU Name.

Step 2: Provides an interface to link SKUs. The user can search the SKU List by name and filter by Brand or Category, then multi-select the SKUs to associate with the FU.

CPL (Customer Planning Level) Management:

Functionality: An advanced, 2-step wizard.

Step 1: Define the CPL Name and Channel.

Step 2: Provides an interface to link Customers. The user can search the Customer List by name, then multi-select the Customers to associate with the CPL.

5.6. Tactic Management

Purpose: To define the types of promotions Planners can use and where they can use them.

Functions: A CRUD screen to manage:

Tactic Name (e.g., "Price Discount")

Calculation Type (Rate Based/Lumpsum)

Spending Type (On-Invoice/Off-Invoice)

Applicability/Scope (Crucial): An interface for the Admin to define which categories or channels this tactic is available for (e.g., "Display Fees" are only available for the "NKA" channel and "Dairy" category).

5.7. KPI Management

Purpose: To configure the brain of the calculation engine, as defined in our KPI Library.

Functions: A CRUD screen to manage KPI Name (e.g., "GP ROI %"), KPI Group ("ROI"), Unit ("%"), the Formula (e.g., "(PlannedGP - BaseGP) / TotalSpend"), and RAG Thresholds (e.g., Green > 0.2).

5.8. Audit Log

Purpose: To provide complete transparency and traceability.

Functions: Lists all critical changes in the system with Timestamp, User (Role), Action, Entity Affected, and Details (e.g., Field: RAG Green, Old Value: '> 0.2', New Value: '> 0.25' or Status: Pending -> Approved).

6. Appendix: Core KPI Library & Formulas

This section defines the core calculations that power the CollMind TPM platform.

KPI Group

KPI Name

Formula / Definition

Strategic Importance

Volume

Base Volume

[Historical Data]

The baseline sales volume expected without a promotion.

Planned Volume

[User Input]

The new total volume target with the promotion.

Incremental Volume (iVol)

Planned Volume - Base Volume

The net additional volume generated only by the promotion.

Volume Uplift %

iVol / Base Volume

Measures the percentage growth in volume, allowing comparison across different-sized products.

Turnover

Base Turnover (Base TO)

Base Volume * List Price

The baseline revenue expected without a promotion.

Planned Turnover (Planned TO)

Planned Volume * List Price

The total gross revenue before on-invoice discounts are applied.

Incremental Turnover (iTO)

Planned TO - Base TO

The additional gross revenue generated by the promotion.

Spend

Total Planned Spend

On-Invoice Spend + Off-Invoice Spend

The total cost of the promotion. Used as the 'Investment' in ROI.

On-Invoice Spend

Planned TO * % Discount

The cost of price reductions (e.g., CPP %).

Off-Invoice Spend

SUM(Lumpsum tactics)

The fixed costs of the promotion (e.g., Display Fees, Leaflets).

Profit

Planned Gross Profit (Planned GP)

(Planned TO - On-Invoice Spend) - COGS - Off-Invoice Spend

The final profit of the promotion after all costs are deducted.

Base Gross Profit (Base GP)

Base TO - (Base Volume * COGS)

The profit that would have been made without a promotion.

Incremental GP (iGP)

Planned GP - Base GP

The net additional profit generated only by the promotion. This is the most important profit metric.

ROI

GP ROI %

iGP / Total Planned Spend

The key success metric. Shows the profit return for every dollar invested in the promotion.

RAG Status

[Defined in KPI Mgt]

A visual (Red, Amber, Green) indicator of the GP ROI % based on admin-defined thresholds.

Business Requirements Document (BRD)

CollMind TPM - Trade Promotion Management System

Document Control

Version	Date	Author	Description
1.0	2025-11-04	Product Team	Initial BRD

Document Classification: Confidential - Internal Use Only

1. EXECUTIVE SUMMARY

1.1 Project Overview

CollMind TPM is a next-generation SaaS platform designed to digitally transform end-to-end Trade Promotion Management (TPM) processes for companies in the Fast-Moving

Consumer Goods (FMCG) sector.

Tagline:*Collaborative Intelligence for Promotion Excellence*

1.2 Business Problem

FMCG companies currently face significant challenges in managing trade promotions:

Problem Area	Current Pain Points	Business Impact
Data Chaos	Planning, budgeting, and results scattered across dozens of Excel files, emails, and presentations	• Lost promotional data • Version control issues • No single source of truth
Profitability Loss	Inability to calculate true ROI of promotions in real-time	• Investment in unprofitable campaigns • 15-30% of promotions deliver negative ROI • Hidden profit leakage
Uncontrolled Spending	Manual tracking leads to end-of-quarter budget surprises	• Budget overruns (average 12-18%) • Reactive budget management • No proactive controls
Operational Inefficiency	Time lost to manual data entry and departmental coordination	• 40-60 hours/month per planner • Delayed decision making • Cross-functional friction

1.3 Proposed Solution

A unified, intelligent platform that brings together Sales, Marketing, and Finance teams to:

- 1. **Plan promotions** with real-time KPI simulation
- 2. **Calculate ROI** instantly with dynamic formula engine

- 3. **Track budgets** proactively with alerts
- 4. **Collaborate** seamlessly across departments
- 5. **Optimize** campaigns based on data-driven insights

1.4 Expected Business Benefits

Benefit Category	Key Metrics	Expected Impact
Financial	• ROI improvement • Budget adherence • Profit recovery	• 10-15% increase in promotion ROI • 95%+ budget compliance • \$500K-2M annual profit recovery
Operational	• Time savings • Process efficiency • Data quality	• 70% reduction in planning time • 90% faster plan approvals • 100% data accuracy
Strategic	• Decision quality • Collaboration • Visibility	• Data-driven decisions (100% of plans) • Real-time cross-functional alignment • Complete promotional visibility

1.5 Success Criteria

Primary KPIs:

-  90% user adoption within 3 months of launch
-  50% reduction in time-to-plan (from 5 days to 2.5 days)
-  100% of plans calculated with ROI before approval
-  Zero budget overruns by Q2 post-implementation

2. BUSINESS OBJECTIVES

2.1 Strategic Objectives

Primary Objective: Digitize and optimize the entire trade promotion lifecycle to maximize profitability and efficiency.

Secondary Objectives:

1. Enhance Profitability

- Eliminate unprofitable promotions through real-time ROI visibility
- Improve average promotion ROI by 10-15%
- Recover hidden profit leakage

2. Improve Financial Control

- Achieve 100% budget visibility and tracking
- Reduce budget variance from 15% to <5%
- Enable proactive budget management with real-time alerts

3. Increase Operational Efficiency

- Reduce manual effort by 70%
- Accelerate approval cycles from 5 days to 24 hours
- Eliminate data entry errors and version control issues

4. Enable Data-Driven Decision Making

- Provide instant KPI simulation for all promotion scenarios
- Deliver actionable insights based on historical performance
- Support strategic planning with predictive analytics

5. Foster Cross-Functional Collaboration

- Unite Sales, Marketing, and Finance on single platform
- Eliminate email chains and spreadsheet chaos
- Create transparency across all stakeholders

2.2 Business Goals by Department

Sales/Trade Marketing:

- Create optimal promotion plans in <2 hours (vs. 2 days currently)
- Achieve 95%+ approval rate on first submission
- Access historical performance data for similar promotions

Category Management:

- Review and approve plans in <30 minutes (vs. 2-3 hours currently)
- Make approval decisions based on clear ROI thresholds
- Ensure all plans align with brand/category strategy

Finance:

- Monitor budget utilization in real-time
- Receive proactive alerts for budget thresholds
- Generate accurate financial reports on-demand

Senior Management:

- Gain complete visibility into promotional ROI across portfolio

- Track promotion effectiveness vs. targets
- Make strategic decisions based on real-time data

3. SCOPE

3.1 In Scope

Phase 1 (MVP) - Core TPM Functionality:

1. Promotion Planning

- Hierarchical planning (FU/SKU level)
- Dynamic KPI calculation engine
- Real-time ROI simulation
- Tactic configuration (on-invoice, off-invoice, lumpsum)
- Multi-level volume planning

2. Approval Workflow

- Role-based approval (Planner → Category Manager)
- Automated notifications
- Comment/feedback system
- Approval/rejection with reasons

3. Budget Management

- Budget allocation by channel/category/period
- Real-time utilization tracking
- Budget threshold alerts
- Spend categorization (on-invoice vs. off-invoice)

4. Reporting & Analytics

- Financial dashboard (Finance Manager view)
- Plan performance reports
- Budget utilization reports
- ROI analysis by channel/category

5. Admin & Configuration

- User management with role-based access
- Master data management (SKU, Customer, FU, CPL)
- KPI configuration with dynamic formulas
- Tactic library management
- Permission matrix configuration
- Audit logging

Phase 2 - Advanced Features:

1. AI-Powered Insights

- Promotion optimization recommendations

- ROI predictor
- Anomaly detection
- Budget optimization suggestions

2. Collaboration Tools

- Real-time co-editing
- Team chat/comments
- Activity feeds
- Version history with diff view

3. Advanced Reporting

- Custom report builder
- Executive dashboards
- Scenario planning ("What-If" analysis)
- Benchmarking against industry standards

4. Integrations

- ERP integration (SAP, Oracle, Dynamics)
- BI tool connectors (Power BI, Tableau)
- Email integration
- External data sources (market data, competitor intel)

3.2 Out of Scope

✗ Not Included in Initial Release:

1. Post-event execution tracking (actual vs. planned)
2. Retailer/customer portal access
3. Mobile native applications (mobile-responsive web only)
4. POS data integration
5. Demand forecasting
6. Assortment planning
7. Price optimization
8. Shelf space management
9. Promotional calendar visualization (future phase)
10. Multi-currency support (USD only in Phase 1)
11. Multi-language support (English only in Phase 1)

3.3 Assumptions

1. Technical:

- Users have modern web browsers (Chrome, Edge, Safari, Firefox - latest 2 versions)
- Stable internet connection (minimum 5 Mbps)
- Master data (SKU, Customer, Pricing) is available and accurate
- Historical volume data available for baseline calculations

2. Business:

- Dedicated project team with decision-making authority
- Executive sponsorship and budget approval secured
- Users available for training and onboarding
- Change management support from HR/Training
- Data migration support from IT

3. Organizational:

- Clear approval workflow defined
- Budget allocation process documented
- Promotion planning process standardized
- KPI definitions agreed upon by stakeholders

3.4 Constraints

1. Timeline:

- MVP delivery: 12-14 weeks from project kickoff
- User acceptance testing: 2 weeks
- Production launch: Week 16

2. Budget:

- Development budget: [To be confirmed]
- Infrastructure costs: Estimated \$2,000-5,000/month
- Training and change management: [To be confirmed]

3. Resources:

- Development team: 3-4 full-time developers
- Product owner: 1 dedicated resource
- Business analysts: 2 part-time resources
- User testers: 10-15 users across departments

4. Technical:

- Must use existing company authentication (SSO integration)
- Must comply with data privacy regulations (GDPR, local laws)
- Must achieve 99.5% uptime SLA
- Must support 100 concurrent users

4. STAKEHOLDERS

4.1 Primary Stakeholders

Stakeholder Group	Role	Key Interests	Success Metrics
Trade Marketing Planners	Create and optimize promotion plans	• Easy-to-use planning tools
• Fast ROI feedback
• Historical data access	• Plan creation time < 2 hours
• 95% approval rate
• User satisfaction > 4/5
Category Managers	Review and approve plans	• Quick review process
• Clear approval criteria
• Portfolio visibility	• Review time < 30 min
• Confidence in decisions
• Reduced back-and-forth
Finance Managers	Monitor budgets and ROI	• Real-time budget tracking
• Accurate financial reporting
• Budget control	• 100% budget visibility
• Zero surprises
• On-demand reports
Admins/IT	Configure and maintain system	• Easy configuration
• Minimal maintenance
• Data security	• <2 hours/week admin time
• Zero data breaches
• System uptime >99.5%

4.2 Secondary Stakeholders

Stakeholder	Role	Involvement
Chief Marketing Officer (CMO)	Executive sponsor	Strategic oversight, budget approval
Chief Financial Officer (CFO)	Financial oversight	Budget governance, ROI accountability
Sales Directors	Regional leaders	Input on regional strategies, user adoption
IT Security	Security compliance	Data security, access controls, audit
Data/BI Team	Reporting and analytics	Integration with BI tools, data warehouse

4.3 RACI Matrix

Activity	Planner	Category Manager	Finance Manager	Admin	IT
Create promotion plan	R/A	I	I	I	-
Approve/reject plan	I	R/A	I	I	-
Set budget	I	C	R/A	I	-
Configure KPIs	I	C	C	R/A	-
Manage users	I	I	I	R/A	C
System maintenance	-	-	-	R/A	R/A
Generate reports	C	C	R/A	C	-

R = Responsible, A = Accountable, C = Consulted, I = Informed

5. FUNCTIONAL REQUIREMENTS

5.1 User Roles & Permissions

5.1.1 Admin

Purpose: System architect - configures the entire platform and sets business rules.

Key Responsibilities:

- Manage all master data (SKU, Customer, CPL, FU, UOM)
- Define users and assign roles
- Configure granular permissions (which Planner manages which CPL/Category)
- Configure KPI calculation engine (formulas, RAG thresholds)
- Manage tactic library (applicability rules)
- Monitor system health via audit logs

Access Level: Full system access

Key Screens:

- Dashboard (system health metrics)
- User Management
- Permissions Configuration
- Master Data modules (SKU, Customer, CPL, FU, UOM)
- KPI Management
- Tactic Management
- Audit Log

5.1.2 Planner (Trade Marketing Planner)

Purpose: Executor - creates, manages, and optimizes promotion plans.

Key Responsibilities:

- Create new promotion plans
- Add products (FUs/SKUs) to plans
- Enter planned volumes at SKU level
- Enter tactic values (discounts, spends) at FU level
- Analyze real-time KPI feedback (ROI, profit)
- Optimize plans based on RAG status
- Submit plans for approval

Access Level: Limited to assigned CPLs and Categories

Key Screens:

- Plan List (with filters and search)
- Detailed Planning Screen (grid with real-time calculations)
- Plan Analytics (historical performance, benchmarks)

Business Rules:

- Can only create plans for authorized CPL/Category combinations
- Can only see tactics applicable to plan's channel/category
- Cannot submit plan if overall ROI is RED (configurable)
- Must add at least one FU before submission

5.1.3 Category Manager (CM)

Purpose: Strategist - ensures plans align with brand, category, and profitability strategy.

Key Responsibilities:

- Review plans submitted by Planners
- Analyze KPI summary (ROI, RAG status)
- Approve plans that meet criteria
- Reject plans with comments for revision
- Monitor category portfolio performance

Access Level: Limited to assigned Categories

Key Screens:

- Approval Queue (pending plans)
- Plan Review/Approval Screen
- Category Performance Dashboard

Business Rules:

- Can only approve/reject plans for authorized categories
- Must provide comment when rejecting
- Cannot edit plan details (read-only except approval action)
- Approval triggers budget deduction

5.1.4 Finance Manager (FM)

Purpose: Controller - owns and controls promotion budget.

Key Responsibilities:

- Define budgets by period/category/channel
- Monitor budget utilization in real-time
- Analyze financial performance of approved promotions
- Generate financial reports for management
- Investigate budget variances

Access Level: Full read access, budget management

Key Screens:

- Budget Management Screen
- Financial Reporting Dashboard
- Budget Utilization Reports
- Spend Analysis by tactic/channel/category

Business Rules:

- Can view all plans regardless of status
- Cannot edit or approve plans
- Receives alerts when budget thresholds exceeded

5.2 Core Functional Requirements

FR-1: Promotion Plan Creation

FR-2: Planning Grid - Data Entry

FR-3: Real-Time KPI Calculation

FR-4: RAG Status Evaluation

FR-5: Approval Workflow

FR-6: Budget Management

FR-7: Reporting & Analytics

FR-8: Admin Configuration

6. NON-FUNCTIONAL REQUIREMENTS

6.1 Performance Requirements

ID	Requirement	Target	Measurement Method
NFR-1.1	Page load time	< 2 seconds	Time to interactive on planning screen
NFR-1.2	KPI calculation time	< 500ms	Time from input change to UI update
NFR-1.3	Grid rendering	< 1 second	Initial render for 50 FUs (200+ SKUs)
NFR-1.4	API response time	< 300ms (p95)	Backend API latency
NFR-1.5	Database query time	< 200ms (p95)	Query execution time
NFR-1.6	Concurrent users	100 users	Load testing verification
NFR-1.7	Auto-save latency	< 1 second	Time from change to database write

6.2 Scalability Requirements

ID	Requirement	Target
NFR-2.1	Support 500+ SKUs per plan	Verified through testing
NFR-2.2	Support 50+ FUs per plan	Verified through testing
NFR-2.3	Support 1,000+ active plans	Database capacity planning
NFR-2.4	Support 500 concurrent users	Infrastructure sizing
NFR-2.5	Handle 10,000+ KPI calculations per second	Benchmarking

6.3 Availability & Reliability

ID	Requirement	Target
NFR-3.1	System uptime	99.5% (excluding planned maintenance)
NFR-3.2	Planned maintenance window	Max 4 hours/month, announced 48 hours in advance
NFR-3.3	Maximum downtime per incident	< 1 hour
NFR-3.4	Data backup frequency	Real-time replication + daily full backups
NFR-3.5	Recovery Time Objective (RTO)	< 4 hours
NFR-3.6	Recovery Point Objective (RPO)	< 5 minutes
NFR-3.7	Error rate	< 0.1% of requests

6.4 Security Requirements

ID	Requirement	Implementation
NFR-4.1	Authentication	SSO integration (SAML 2.0 / OAuth 2.0)
NFR-4.2	Authorization	Role-based access control (RBAC)
NFR-4.3	Data encryption	TLS 1.3 in transit, AES-256 at rest
NFR-4.4	Password policy	Minimum 12 characters, complexity requirements
NFR-4.5	Session timeout	30 minutes of inactivity
NFR-4.6	API security	API keys, rate limiting (100 req/min per user)
NFR-4.7	Audit logging	All user actions logged with timestamp and user ID
NFR-4.8	Data privacy	GDPR compliant, data retention policies
NFR-4.9	Penetration testing	Annual security audit by third party

6.5 Usability Requirements

ID	Requirement	Target
NFR-5.1	User onboarding time	New user productive within 30 minutes
NFR-5.2	User satisfaction score	> 4.0/5.0 (post-training survey)
NFR-5.3	Task completion rate	> 95% (create and submit plan)
NFR-5.4	Error recovery	Clear error messages with actionable guidance
NFR-5.5	Accessibility	WCAG 2.1 Level AA compliance
NFR-5.6	Browser support	Latest 2 versions of Chrome, Edge, Firefox, Safari
NFR-5.7	Mobile responsiveness	Functional on tablets (iPad), optimized desktop experience

6.6 Data Integrity Requirements

ID	Requirement	Implementation
NFR-6.1	Calculation accuracy	100% accurate to 2 decimal places
NFR-6.2	Data consistency	ACID compliance for all transactions
NFR-6.3	Validation	Client-side and server-side validation
NFR-6.4	Conflict resolution	Optimistic locking for concurrent edits
NFR-6.5	Version control	Track all plan versions with diff view

6.7 Compliance Requirements

ID	Requirement	Standard
NFR-7.1	Data privacy	GDPR, CCPA, local data protection laws
NFR-7.2	Financial reporting	SOX compliance for financial data
NFR-7.3	Audit trail	Immutable logs for all financial transactions
NFR-7.4	Data residency	Data stored in compliant geographic regions

7. USER STORIES & USE CASES

7.1 Epic 1: Promotion Planning

User Story 1.1: Create New Promotion Plan

As a Trade Marketing Planner **I want to** create a new promotion plan for my assigned CPL and Category **So that** I can start planning promotional activities

Acceptance Criteria:

- ☐ I can click "+ New Plan" button from Plan List screen
- ☐ Modal opens with form fields: Plan Name, CPL, Category, Start Date, End Date
- ☐ CPL dropdown shows only CPLs I have permission for
- ☐ Category dropdown filters based on selected CPL
- ☐ All fields are mandatory and validated
- ☐ Plan is created with status "Draft"
- ☐ I am redirected to Planning Grid screen
- ☐ Plan appears in my Plan List

Priority: Must Have **Story Points:** 5

User Story 1.2: Add Products to Plan

Asa** Trade Marketing Planner **I want to** add FUs (Forecasting Units) and their SKUs to my promotion plan **So that** I can plan volumes and tactics for specific products

Acceptance Criteria:

- I can click "Add FU" button in Planning Grid
- Modal opens with searchable list of FUs
- Search works by FU code and FU name
- I can filter by Brand and Category
- I can multi-select FUs
- System shows how many SKUs are in each FU
- When I add FU, all child SKUs automatically appear in grid
- Base volumes are auto-populated from historical data
- Each SKU row shows: SKU code, name, base volume, planned volume (empty)

Priority: Must Have **Story Points:** 8

User Story 1.3: Enter Planned Volumes

As a Trade Marketing Planner **I want to** enter planned volumes at SKU level **So that** the system can calculate promotional KPIs

Acceptance Criteria:

- Planned Volume column is editable only at SKU rows
- I can click to edit, type value, press Enter to save
- System validates input (positive numbers only)
- FU row shows sum of all child SKU volumes (read-only)
- All dependent KPIs recalculate within 500ms
- Changed values are highlighted briefly (green flash)

- Grand Totals Panel updates automatically
- Changes are auto-saved after 2 seconds

Priority: Must Have **Story Points:** 8

User Story 1.4: Configure Promotion Tactics

As a Trade Marketing Planner **I want to** enter discount percentages and lumpsum spends at FU level **So that** the system can calculate promotional costs and ROI

Acceptance Criteria:

- Only tactics applicable to my plan's channel/category are shown
- Tactic columns are editable only at FU rows
- I can enter percentages (e.g., CPP On-Invoice %)
- I can enter currency amounts (e.g., Visibility Lumpsum)
- System validates min/max values per tactic configuration
- SKU rows show inherited tactic values (read-only, grayed out)
- All SKUs under FU recalculate when tactic changes
- System shows warning if discount exceeds typical range

Priority: Must Have **Story Points:** 13

User Story 1.5: Analyze Real-Time ROI

As a Trade Marketing Planner **I want to** see GP ROI % and RAG status for each SKU/FU as I plan **So that** I can optimize my promotion before submitting

Acceptance Criteria:

- GP ROI % column shows calculated value for each SKU/FU
- RAG status column shows colored badge (Green/Amber/Red)
- Cell background color matches RAG status
- Hovering over ROI cell shows detailed tooltip with thresholds
- Grand Totals Panel shows overall plan ROI
- I can see which items are RED and need attention
- System suggests optimizations for RED items
- I can click to see detailed breakdown modal

Priority: Must Have **Story Points:** 13

7.2 Epic 2: Approval Workflow

User Story 2.1: Submit Plan for Approval

As a Trade Marketing Planner **I want to** submit my completed plan for Category Manager approval **So that** it can be reviewed and executed

Acceptance Criteria:

- "Submit for Approval" button is enabled when plan is valid
- Clicking button shows confirmation modal
- Modal displays plan summary (totals, ROI, RAG breakdown)
- I can add comments for the reviewer
- System validates: at least 1 FU added, no validation errors, budget available
- If validation fails, I see specific error messages
- If RED items exist, system shows warning (block if configured)
- On submit, plan status changes to "Pending Approval"
- Category Manager receives notification
- I can no longer edit plan while pending

Priority: Must Have **Story Points:** 8

User Story 2.2: Review and Approve Plan

As a Category Manager **I want to** review submitted plans and approve/reject them **So that** only profitable promotions are executed

Acceptance Criteria:

- I see list of plans with status "Pending Approval" for my categories
- List shows: Plan name, Planner, CPL, Category, Submission date, GP ROI %, Total Spend
- I can sort and filter the list
- Clicking plan opens Approval Detail modal
- Modal shows: Grand Totals Panel (read-only), Planning Grid (read-only), Comments
- I can see RAG breakdown (# of Green/Amber/Red items)
- RED items are highlighted for attention
- I can add my own comments
- I can click "Approve" or "Reject"
- Rejection requires comment
- Approved plans deduct from budget automatically
- Planner receives notification of decision

Priority: Must Have **Story Points:** 13

User Story 2.3: Revise Rejected Plan

As a Trade Marketing Planner **I want to** edit and re-submit a rejected plan **So that** I can address Category Manager's concerns

Acceptance Criteria:

- Rejected plan shows status "Rejected" with CM comments
- I receive notification with rejection reason
- I can open plan and edit it

- Plan status automatically changes back to "Draft"
- I can see rejection history/comments
- After changes, I can re-submit
- Rejection history is preserved in audit log

Priority: Must Have **Story Points:** 5

7.3 Epic 3: Budget Management

User Story 3.1: Define Promotion Budget

As a Finance Manager **I want to** set promotion budgets by period, channel, and category **So that** I can control promotional spending

Acceptance Criteria:

- I can access Budget Management screen
- I can create new budget with: Period (start/end date), Channel, Category (optional), On-Invoice Amount, Off-Invoice Amount
- System validates: Period not overlapping, amounts are positive
- I can see list of all budgets
- I can edit existing budgets (if not fully utilized)
- I can view budget utilization % for each budget
- Color coding shows: Green (<80%), Amber (80-95%), Red (>95%)

Priority: Must Have **Story Points:** 8

User Story 3.2: Monitor Budget Utilization

As a Finance Manager **I want to** see real-time budget utilization across all channels/categories **So that** I can prevent overspending

Acceptance Criteria:

- Dashboard shows budget utilization widgets
- Progress bars show: On-Invoice spend vs. budget, Off-Invoice spend vs. budget
- Color-coded status (Green/Amber/Red) based on utilization %
- I can filter by: Channel, Category, Month/Quarter/Year
- I can drill down to see which plans consumed budget
- Spend trend chart shows spending over time
- Export to Excel available

Priority: Must Have **Story Points:** 13

User Story 3.3: Receive Budget Alerts

As a Finance Manager **I want to** receive automatic alerts when budget thresholds are exceeded **So that** I can take corrective action proactively

Acceptance Criteria:

- I receive notification when budget reaches 80% (warning)
- I receive notification when budget reaches 95% (critical)
- I receive notification when budget is exceeded
- Notification shows: Budget name, Current spend, Budget amount, Utilization %
- I can click notification to go to Budget Dashboard
- Alert history is logged

Priority: Must Have **Story Points:** 5

7.4 Epic 4: Admin Configuration

User Story 4.1: Configure KPI with Dynamic Formula

As an Admin **I want to** create or edit KPIs with custom formulas **So that** the system calculates KPIs according to our business rules

Acceptance Criteria:

- I can access KPI Management screen
- I can create new KPI with: Code, Name, Group, Formula Type, Formula Text
- Formula editor supports: Expression, Conditional, JavaScript
- System validates formula syntax on save
- System extracts dependencies automatically from formula
- I can specify: Display format, Decimal places, Show in grid
- I can configure RAG thresholds (Green/Amber/Red values)
- I can test formula with sample data before saving
- System shows circular dependency errors
- I can see which other KPIs depend on this KPI

Priority: Must Have **Story Points:** 13

User Story 4.2: Configure Tactic Applicability

As an Admin **I want to** define which tactics are available for which channels and categories **So that** Planners only see relevant tactics

Acceptance Criteria:

- I can access Tactic Management screen
- I can create new tactic with: Code, Name, Input Type, Calculation Type
- I can specify applicability: Select channels (multi-select or "All")
- I can specify applicability: Select categories (multi-select or "All")
- I can set min/max value constraints
- I can enter calculation formula (how to calculate spend from input)
- I can test that tactic appears correctly in planning grid
- Example: "WS TPR" only shows for "Wholesale" channel plans

Priority: Must Have **Story Points:** 13

User Story 4.3: Set User Permissions

As an Admin **I want to** assign granular permissions to Planners and Category Managers **So that** users only see and manage their authorized data

Acceptance Criteria:

- I can access User Management screen
- For each user, I can click "Set Permissions" button
- For Planners: I can add rules linking CPL → Categories
- Example: "Planner A can manage Dairy and Beverages for Carrefour CPL"
- For Category Managers: I can select which categories they approve
- I can see permission matrix view (users vs. CPL/Categories)
- Changes are logged in audit trail
- Users immediately see updated access (no logout required)

Priority: Must Have **Story Points:** 13

7.5 Epic 5: Reporting & Analytics

User Story 5.1: View Financial Dashboard

As a Finance Manager **I want to** see comprehensive financial analytics of all promotions **So that** I can report to senior management

Acceptance Criteria:

- Dashboard shows: Total approved plans, Total spend, Average ROI, Active promotions
- Pie chart shows spend distribution by tactic type
- Line chart shows monthly spend trend
- Table shows top 10 performing plans (by ROI)
- I can filter by: Date range, Channel, Category, CPL
- All charts and tables update when filters change
- Export to PDF/Excel available
- Dashboard loads within 3 seconds

Priority: Must Have **Story Points:** 13

User Story 5.2: Analyze Plan Performance

As a Trade Marketing Planner **I want to** see how similar past promotions performed **So that** I can make data-driven planning decisions

Acceptance Criteria:

- In Planning Screen, I can click "View Analytics" button
- Modal shows historical performance of similar plans (same CPL, Category)
- Chart shows ROI distribution (how many plans achieved each ROI range)
- I can see average ROI, best ROI, worst ROI

- I can see which tactics were most effective
- I can see typical volume uplift % for this combination
- Benchmark data helps me set realistic targets

Priority: Should Have **Story Points:** 13

7.6 Epic 6: System Administration

User Story 6.1: Import Master Data

As an Admin **I want to** bulk import SKUs, Customers, and other master data from Excel **So that** I don't have to enter thousands of records manually

Acceptance Criteria:

- I can download Excel template for each entity type
- Template has all required columns with descriptions
- I can upload filled Excel file (drag & drop)
- System validates all data before import
- Validation shows: Row number, Field name, Error description
- I can choose to: Skip invalid rows or Fix and retry
- Valid rows are imported successfully
- Import summary shows: X records imported, Y skipped, Z errors
- All imports are logged in audit trail

Priority: Must Have **Story Points:** 13

User Story 6.2: View Audit Log

As an Admin **I want to** see complete history of all system changes **So that** I can track who did what and when

Acceptance Criteria:

- Audit Log screen shows all system events
- Columns: Timestamp, User, Role, Action, Entity Type, Entity ID, Details
- Details show before/after values for updates
- I can filter by: User, Date range, Action type, Entity type
- I can search by entity ID or keyword
- Export to CSV available
- Logs are immutable (cannot be edited or deleted)
- Retention: 2 years minimum

Priority: Must Have **Story Points:** 8

8. DATA REQUIREMENTS

8.1 Data Entities

8.1.1 Master Data Entities

SKU (Stock Keeping Unit)

Field	Type	Mandatory	Description	sku_id	UUID	Yes	Primary key	sku_code
VAR	CHAR	(50)	Yes	Unique	SKU identifier	sku_name	VAR	CHAR
(200)	Yes	Product						
name	brand	VAR	CHAR	(100)	Yes	Brand name	category	VAR
CHAR	(100)	Yes	Product					
category	list_price	DEC	IMAL	(10,2)	Yes	Standard	selling price per piece	
	cogs_per_piece	DEC	IMAL	(10,2)	Yes	Cost of goods sold per piece	uom	
	VAR	CHAR	(20)	Yes	Unit of measure	status	VAR	CHAR
	(20)	Yes	Active/Inactive					
created_at	TIME	STAMP	Yes	Record creation date				

Customer

Field	Type	Mandatory	Description	customer_id	UUID	Yes	Primary key	customer_code
VAR	CHAR	(50)	Yes	Unique	customer identifier	customer_name	VAR	CHAR
(200)	Yes	Customer/store name	channel	VAR	CHAR	(50)	Yes	NKA,
		Traditional Trade, E-Commerce, Export	city	VAR	CHAR	(100)	Yes	Location
status	VAR	CHAR	(20)	Yes	Active/Inactive			

FU (Forecasting Unit)

Field	Type	Mandatory	Description	fu_id	UUID	Yes	Primary key	fu_code
VAR	CHAR	(50)	Yes	Unique	FU identifier	fu_name	VAR	CHAR
(200)	Yes	FU name						
created_at	TIME	STAMP	Yes	Record creation date				

FU-SKU Mapping

Field	Type	Mandatory	Description	fu_sku_id	UUID	Yes	Primary key	fu_id
UUID	Yes	Foreign key to FU	sku_id	UUID	Yes	Foreign key to SKU		

CPL (Customer Planning Level)

Field	Type	Mandatory	Description	cpl_id	UUID	Yes	Primary key	cpl_name
VAR	CHAR	(100)	Yes	CPL name (e.g., "Carrefour CPL")	channel	VAR	CHAR	(50)
Yes	Associated channel	created_at	TIME	STAMP	Yes	Record creation date		

CPL-Customer Mapping

Field	Type	Mandatory	Description	cpl_customer_id	UUID	Yes	Primary key	cpl_id
UUID	Yes	Foreign key to CPL	customer_id	UUID	Yes	Foreign key to Customer		

8.1.2 Transactional Data Entities

Plan

Field	Type	Mandatory	Description
plan_id	UUID	Yes	Primary key
plan_name	VARCHAR(200)	Yes	User-defined plan name
planner_id	UUID	Yes	Foreign key to Users
cpl_id	UUID	Yes	Foreign key to CPL
category	VARCHAR(100)	Yes	Product category
start_date	DATE	Yes	Promotion start date
end_date	DATE	Yes	Promotion end date
status	VARCHAR(20)	Yes	Draft, Pending Approval, Approved, Rejected
submitted_at	TIMESTAMP	No	When submitted for approval
reviewed_by	UUID	No	Foreign key to Users (CM)
reviewed_at	TIMESTAMP	No	When reviewed
reviewer_comments	TEXT	No	CM comments
created_at	TIMESTAMP	Yes	Record creation date
updated_at	TIMESTAMP	Yes	Last update date

Plan FU

Field	Type	Mandatory	Description
plan_fu_id	UUID	Yes	Primary key
plan_id	UUID	Yes	Foreign key to Plan
fu_id	UUID	Yes	Foreign key to FU
display_order	INTEGER	Yes	Order in grid
created_at	TIMESTAMP	Yes	Record creation date

Plan SKU Volumes

Field	Type	Mandatory	Description
plan_sku_volume_id	UUID	Yes	Primary key
plan_fu_id	UUID	Yes	Foreign key to Plan FU
sku_id	UUID	Yes	Foreign key to SKU
base_volume	DECIMAL(15,2)	Yes	Historical baseline volume
planned_volume	DECIMAL(15,2)	Yes	User-entered planned volume
calculated_kpis	JSONB	Yes	All calculated KPI values
updated_at	TIMESTAMP	Yes	Last update date

Plan Mechanic Values

Field	Type	Mandatory	Description
plan_mechanic_value_id	UUID	Yes	Primary key
plan_fu_id	UUID	Yes	Foreign key to Plan FU
mechanic_id	UUID	Yes	Foreign key to Promo Mechanics
entered_value	DECIMAL(15,4)	Yes	User-entered value (%, \$)
calculated_spend	DECIMAL(15,2)	Yes	Result of formula calculation
updated_at	TIMESTAMP	Yes	Last update date

8.1.3 Configuration Data Entities

KPIs

Field	Type	Mandatory	Description
kpi_id	UUID	Yes	Primary key
kpi_code	VARCHAR(50)	Yes	Unique code (e.g., INCR_VOL)
kpi_name	VARCHAR(200)	Yes	Display name
kpi_group	VARCHAR(100)	Yes	Volume, Profit, ROI, etc.
formula_type	VARCHAR(50)	Yes	expression, conditional, user_input
formula_text	TEXT	Yes	Actual formula
depends_on_kpis	JSONB	Yes	Array of

dependency KPI codes calculation_order INTEGER Yes Execution order
 display_format VARCHAR(50) Yes number, currency, percentage
 decimal_places INTEGER Yes Formatting precision show_in_grid BOOLEAN Yes
 Display in planning grid rag_enabled BOOLEAN Yes Whether RAG applies
 rag_green_threshold DECIMAL(10,2) No Green threshold value
 rag_amber_threshold DECIMAL(10,2) No Amber threshold value
 rag_red_threshold DECIMAL(10,2) No Red threshold value is_active BOOLEAN
 Yes Active/Inactive created_at TIMESTAMP Yes Record creation date

Promo Mechanics

Field Type Mandatory Description mechanic_id UUID Yes Primary key
 mechanic_code VARCHAR(50) Yes Unique code mechanic_name VARCHAR(200) Yes
 Display name category VARCHAR(100) Yes on_invoice_discount,
 off_invoice_discount, lumpsum input_type VARCHAR(50) Yes percentage,
 currency min_value DECIMAL(15,4) No Minimum allowed value max_value
 DECIMAL(15,4) No Maximum allowed value calculation_formula TEXT Yes How
 to calculate spend applicable_channels JSONB Yes Array of channel names
 or "ALL" applicable_categories JSONB Yes Array of category names or
 "ALL" show_in_grid BOOLEAN Yes Display in planning grid is_active
 BOOLEAN Yes Active/Inactive created_at TIMESTAMP Yes Record creation
 date

Budgets

Field Type Mandatory Description budget_id UUID Yes Primary key
 period_start DATE Yes Budget period start period_end DATE Yes Budget
 period end channel VARCHAR(50) Yes Channel category VARCHAR(100) No
 Category (optional) on_invoice_budget DECIMAL(15,2) Yes On-invoice
 budget amount off_invoice_budget DECIMAL(15,2) Yes Off-invoice budget
 amount created_by UUID Yes Foreign key to Users created_at TIMESTAMP Yes
 Record creation date

User Permissions

Field Type Mandatory Description permission_id UUID Yes Primary key
 user_id UUID Yes Foreign key to Users permission_type VARCHAR(50) Yes
 planner_rule, cm_category cpl_id UUID No For planner rules category
 VARCHAR(100) No Category name created_at TIMESTAMP Yes Record creation
 date

8.1.4 Audit & Logging

Audit Log

Field	Type	Mandatory	Description
audit_log_id	UUID	Yes	Primary key
user_id	UUID	Yes	Who performed action
action	VARCHAR(50)	Yes	Created, Updated, Deleted, Approved, Rejected
entity_type	VARCHAR(100)	Yes	Plan, SKU, KPI, etc.
entity_id	UUID	Yes	ID of affected entity
details	JSONB	Yes	Before/after values
ip_address	VARCHAR(50)	No	User IP
timestamp	TIMESTAMP	Yes	When action occurred

8.2 Data Volume Estimates

Entity	Estimated Volume	Growth Rate
SKUs	5,000 - 10,000	5% annually
Customers	500 - 2,000	3% annually
FUs	500 - 1,500	5% annually
CPLs	10 - 30	Stable
Plans	500 - 1,000/year	10% annually
Users	50 - 200	20% annually
Audit Log Entries	100,000 - 500,000/year	Proportional to usage

8.3 Data Retention Policy

Data Type	Retention Period	Archive Strategy
Active Plans	Indefinite	Keep in primary database
Completed Plans	3 years	online
Audit Logs	7 years	Compliance requirement
Master Data	Indefinite	Soft delete (status = Inactive)
User Sessions	30 days	Purge automatically
System Logs	90 days	Archive to log management system

8.4 Data Integration Requirements

Inbound Data:

- SKU master data from ERP (daily sync)
- Customer master data from CRM (daily sync)
- Historical sales volumes from Data Warehouse (weekly sync)
- Pricing data from Pricing System (daily sync)
- Actual sales results (post-event, out of scope for Phase 1)

Outbound Data:

- Approved plans to Finance System (real-time on approval)
- Budget utilization to BI Dashboard (real-time)
- Plan summaries to Executive Dashboard (daily refresh)

9. USER INTERFACE REQUIREMENTS

9.1 General UI Principles

Design Philosophy:

- Modern, clean, enterprise SaaS aesthetic

- Data-dense but not cluttered
- Minimalist color palette with purpose (RAG status, brand colors)
- Responsive design (desktop-first, tablet-optimized)
- Accessibility compliant (WCAG 2.1 Level AA)

Color Scheme:

Element Color Usage Primary Brand #3B82F6 (Blue) Buttons, links, headers
 Success/Green #10B981 RAG Green, positive indicators Warning/Amber
 #F59E0B RAG Amber, warnings Danger/Red #EF4444 RAG Red, errors
 Gray/Neutral #9CA3AF RAG Gray, disabled states Background #F9FAFB Page
 background Text Primary #111827 Main text Text Secondary #6B7280
 Supporting text

9.2 Key Screen Layouts

9.2.1 Planning Grid Screen

Layout Structure:

HEADER: Plan Name Status Badge Actions (Save, Submit, etc.)									
GRAND TOTALS PANEL (Sticky)									
[Volume Card] [Turnover Card] [Spend Card] [Profit] [ROI] [Status]									
TOOLBAR: [+Add FU] [Expand All] [Filters] [Columns] [Export]									
PLANNING GRID (Scrollable)									
[Fixed Left: Item Name] [Scrollable: KPIs & Tactics] [Fixed]									
[-] FU: Activia 4x100g 8,000 10,500 ... [10%] ●									
↳ SKU: Strawberry 3,200 [4200] ... 10% ●									
↳ SKU: Vanilla 2,400 [3150] ... 10% ●									
[+] FU: Greek Yogurt... ...									

Key UI Elements:

- Sticky Grand Totals Panel (always visible on scroll)
- Sticky column headers

- Fixed left columns (item name) for context
- Fixed right columns (ROI, RAG) for quick status view
- Horizontal scroll for middle columns
- Expand/collapse icons for FU rows
- Editable cells highlighted with blue border on focus
- RAG color-coded cells with icons
- Animated value changes (green flash for updates)

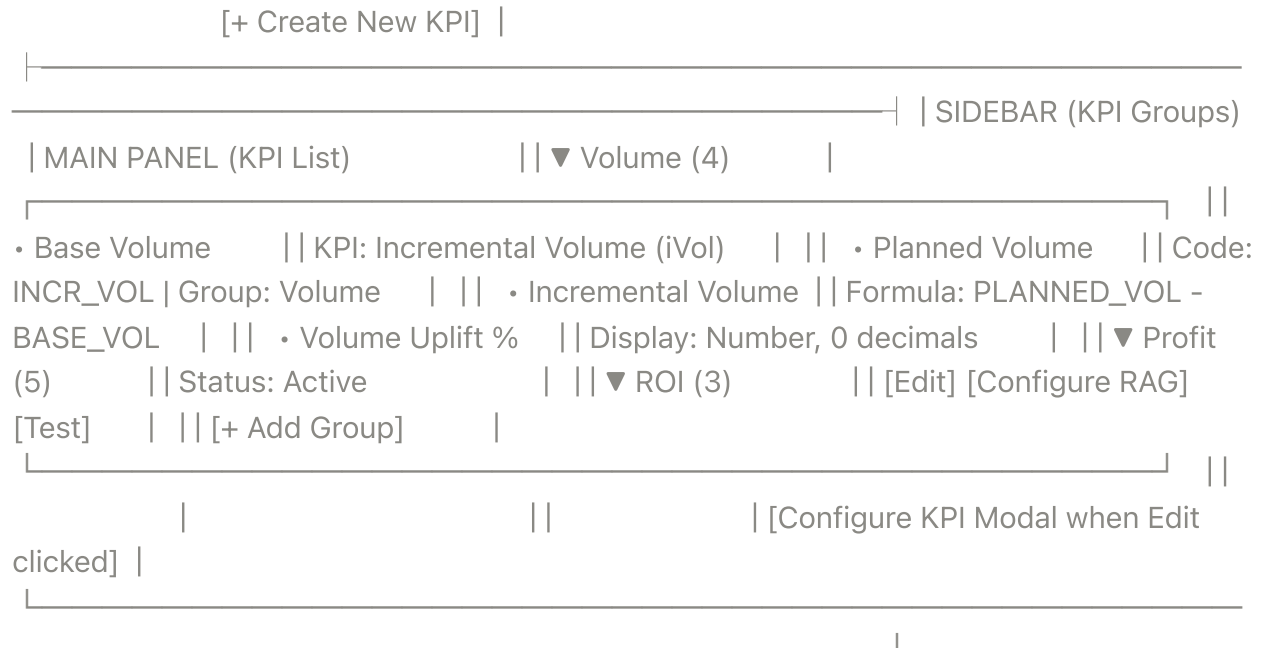
9.2.2 Approval Screen (Category Manager)

Layout Structure:

```
|  
| HEADER: Pending Approvals |  
|  
| FILTERS: [Status: Pending] [Category: Dairy] [Date Range] |  
|  
| APPROVAL QUEUE (List View) |  
|  
| Plan Name | Planner | ROI | Spend | Date | Action ||  
| Summer Dairy | John P. | 24.5% 🟢 | $95K | 3d ago | [View]||  
| Back to School | Sarah M. | 15.2% 🟡 | $120K | 1d ago | [View]||  
| Holiday Promo | Mike L. | 8.5% 🔴 | $180K | 5h ago | [View]||  
|
```

Approval Detail Modal:

```
| Plan: Summer Dairy Promotion [X] Close |
|
| PERFORMANCE SUMMARY |
| Overall Status: 🟢 GREEN | GP ROI: 24.9% | Total Spend: $95,250 |
| Items: 16 Green | 3 Amber | 1 Red |
|
| [Tab: Grid View] [Tab: Comments] [Tab: History] |
| |
| PLANNING GRID (Read-only preview) |
```

9.3 Responsive Design Requirements

****Desktop (>1440px):****

- Full grid with all columns visible
- Side-by-side panels (e.g., sidebar + main content)
- Multi-column layouts for dashboards
- Optimal experience

****Tablet (768px - 1440px):****

- Reduced column count (hide optional columns)
- Horizontal scroll for grid
- Condensed Grand Totals Panel (3 columns instead of 6)
- Hamburger menu for navigation

****Mobile (<768px):****

- Card-based layout instead of grid
- Vertical stacking of all elements
- Collapsible sections
- Bottom navigation bar
- Touch-optimized controls (larger tap targets)
- Simplified views (view-only for planning grid)

9.4 Interaction Patterns

****Data Entry:****

- Click to edit (inline editing)
- Auto-focus and select all text on edit
- Save on blur or Enter key
- Cancel on Escape key
- Tab to move to next editable field
- Real-time validation with inline errors

****Feedback:****

- Toast notifications for success/error (top-right corner, 3s duration)
- Loading spinners for async operations
- Skeleton screens for initial data load
- Animated value changes (green highlight, 300ms)
- Progress bars for long operations

****Navigation:****

- Breadcrumbs for hierarchy (Plan List > Plan Detail)
- Back button where applicable
- Confirm dialog for destructive actions
- Auto-save eliminates "unsaved changes" warnings

****Help & Guidance:****

- Tooltip on hover (info icon or help text)
- Empty states with clear call-to-action
- Inline help text for complex fields
- Contextual help links

****10. INTEGRATION REQUIREMENTS****

10.1 Authentication Integration

****Requirement:****

Integrate with company's existing Single Sign-On (SSO) system.

****Technical Details:****

- Identity Provider: [To be specified by client]
- User attributes required: Email, Full Name, Employee ID
- Session timeout: 30 minutes of inactivity
- Multi-factor authentication (MFA): Support if enabled in IdP

10.2 ERP Integration (Phase 2)

****Purpose:****

Sync master data and push approved plans to ERP system.

****Data Flows:****

****Inbound (ERP → CollMind):****

- SKU master data (daily sync)
- Customer master data (daily sync)
- Pricing data (daily sync)
- Historical sales volumes (weekly sync)

****Outbound (CollMind → ERP):****

- Approved promotion plans (real-time on approval)
- Budget reservations (real-time)
- Spend actuals (post-event, future phase)

****Integration Method:****

- REST API or SOAP (based on ERP capabilities)
- Scheduled batch jobs for master data
- Event-driven webhooks for real-time updates

10.3 BI Tool Integration (Phase 2)

****Purpose:****

Enable reporting and analytics in existing BI tools.

****Supported BI Tools:****

- Microsoft Power BI
- Tableau
- Looker

****Integration Method:****

- Direct database connection (read-only views)

- Pre-built data models and relationships
- Scheduled data refresh (hourly or daily)

10.4 Email Integration

****Purpose:****

Send notifications and reports via email.

****Email Types:****

- Plan submitted for approval (to Category Manager)
- Plan approved/rejected (to Planner)
- Budget threshold alerts (to Finance Manager)
- Weekly/monthly reports (to stakeholders)

****Integration Method:****

- SMTP server configuration
- Email templates with dynamic content
- Unsubscribe management
- Inline HTML formatting

****11. TRAINING & CHANGE MANAGEMENT****

11.1 Training Requirements

****Training Audience:****

Role	Training Duration	Training Method
Planners	4 hours	Hands-on workshop
Category Managers	2 hours	Hands-on workshop
Finance Managers	2 hours	Demo + Q&A
Admins	8 hours	Technical workshop + documentation

****Training Content:****

****For Planners:****

1. System overview and navigation
2. Creating and configuring plans
3. Understanding the planning grid

5. Interpreting KPIs and RAG status
6. Optimizing plans for better ROI
7. Submission workflow
8. Handling rejections

****For Category Managers:****

1. System overview
2. Approval queue and filtering
3. Reviewing plan performance
4. Approval criteria and best practices
5. Providing constructive feedback

****For Finance Managers:****

1. System overview
2. Budget management
3. Reading financial dashboards
4. Interpreting reports
5. Budget alerts and actions

****For Admins:****

1. Master data management
2. User and permission configuration
3. KPI formula configuration
4. Tactic setup and applicability
5. System monitoring and audit logs
6. Troubleshooting common issues

****Training Materials:****

- User manuals (role-specific)
- Video tutorials (5-10 min each)
- Quick reference guides (1-2 pages)
- Interactive sandbox environment
- FAQ document

11.2 Change Management Plan

****Communication Strategy:****

****Phase 1: Awareness (Weeks 1-2)****

- Executive announcement email

- Benefits and timeline communication
- FAQs published

****Phase 2: Education (Weeks 3-6)****

- Role-specific training sessions
- User guide distribution
- Office hours for Q&A
- Success stories from pilot users

****Phase 3: Adoption (Weeks 7-12)****

- Go-live announcement
- Daily support availability
- User feedback collection
- Quick wins highlighted

****Phase 4: Reinforcement (Ongoing)****

- Monthly tips and tricks emails
- Advanced training sessions
- User community forum
- Recognition of power users

****Success Metrics:****

- 90% user adoption within 3 months
- <10% support tickets per user per month
- User satisfaction score >4/5
- 50% reduction in planning time achieved

****Resistance Management:****

- Identify champions in each department
- Address concerns proactively
- Provide 1-on-1 coaching for hesitant users
- Gather and act on feedback quickly

**12. RISKS & MITIGATION**

12.1 Project Risks

Risk	Probability	Impact	Mitigation Strategy
------	-------------	--------	---------------------

****Data quality issues in master data****	High	High	• Conduct data audit before migration • Implement strict validation rules • Provide data cleansing tools
****User resistance to change****	Medium	High	• Strong change management program • Executive sponsorship • Early involvement of key users
****Formula complexity and errors****	Medium	Medium	• Thorough testing of all formulas • Validation with business users • Formula audit by finance team
****Integration delays with ERP/SSO****	Medium	Medium	• Start integration planning early • Have fallback options (manual import) • Dedicated integration specialist
****Performance issues with large datasets****	Low	High	• Load testing before go-live • Database optimization • Implement caching strategies
****Budget overruns****	Medium	Medium	• Detailed project plan with buffers • Regular budget reviews • Prioritize MVP features
****Scope creep****	High	Medium	• Clear requirements sign-off • Change control process • Park Phase 2 features firmly
****Key personnel turnover****	Low	High	• Document all processes • Cross-train team members • Knowledge transfer sessions

12.2 Business Risks

Risk	Probability	Impact	Mitigation Strategy
Inaccurate ROI calculations	Low	Critical	• Extensive UAT with finance team • Parallel run with Excel for 1 month • Independent audit of formulas
Budget overruns undetected	Low	High	• Real-time budget tracking • Multiple alert thresholds • Daily budget reports
Unauthorized access to financial data	Low	Critical	• Strict RBAC implementation • Regular access reviews • Audit logging of all actions
System downtime during critical planning period	Low	High	• 99.5% SLA with penalties • Redundant infrastructure • Maintenance windows scheduled off-peak

13. SUCCESS CRITERIA & KPIs

13.1 Project Success Criteria

****Must Achieve (Go/No-Go):****

- [] All core functionality delivered and tested
- [] 90% of UAT test cases passed
- [] Performance targets met (<2s page load, <500ms calculations)
- [] Security audit passed
- [] 80% of users trained before go-live
- [] Executive sign-off obtained

13.2 Business KPIs (Post-Launch)

****Efficiency Metrics:****

KPI	Baseline	Target (6 months)	Measurement Method
-----	-----	-----	-----
Time to create plan	2 days	<4 hours	User survey + system logs
Time to approve plan	1-2 days	<4 hours	System timestamps
Planning errors	15-20%	<5%	Rejection rate
Manual data entry time	40 hrs/month/planner	<10 hrs/month	User survey

****Financial Metrics:****

KPI	Baseline	Target (6 months)	Measurement Method
-----	-----	-----	-----
Average promotion ROI	18%	>22%	System calculation
% of profitable promotions	70%	>85%	Plans with ROI >0%
Budget variance	±15%	<±5%	Actual vs. budget
Profit recovered	N/A	\$500K-2M annually	Finance analysis

****Adoption Metrics:****

KPI	Target	Measurement Method
-----	-----	-----
Active users	90% of licensed users	Login analytics
Plans created in system	100%	vs. Excel plans
User satisfaction	>4.0/5.0	Quarterly survey
Support tickets	<10/user/month	Help desk metrics

****Process Metrics:****

Plans with calculated ROI before approval	0%	100%	System data
Real-time budget visibility	No	Yes	Feature usage
Data quality (errors/plan)	High (unmeasured)	<3 errors/plan	
Validation errors			

13.3 System Health KPIs (Ongoing)

KPI	Target	Alert Threshold
System uptime	99.5%	<99.0%
API response time (p95)	<300ms	>500ms
Page load time (p95)	<2s	>3s
Calculation time	<500ms	>1s
Error rate	<0.1%	>1%
Database query time	<200ms	>500ms

14. ASSUMPTIONS & DEPENDENCIES

14.1 Assumptions

Business Assumptions:

1. Current promotion planning process is primarily Excel-based
2. ROI calculation formulas are agreed upon by Finance and Marketing
3. Budget allocation process is clearly defined
4. Approval hierarchy is single-level (Planner → CM)
5. Users have basic computer literacy and Excel experience
6. Company has reliable internet connectivity
7. Promotion planning happens primarily during office hours (9-5)

Technical Assumptions:

1. Master data (SKU, Customer, Pricing) is available and reasonably accurate
2. Historical sales data is available for at least 12 months
3. Company has SSO system for authentication
4. Modern web browsers are standard (Chrome, Edge, latest 2 versions)
5. Cloud infrastructure is approved for sensitive financial data
6. Database backups are managed by cloud provider

****Resource Assumptions:****

1. Dedicated product owner available full-time
2. Business users available for UAT (10-15 users, 2 weeks)
3. IT support for SSO/ERP integration
4. Training facilities and schedule available
5. Budget approved for full project scope

14.2 Dependencies

****External Dependencies:****

Dependency	Owner	Impact if Delayed	Mitigation
SSO integration credentials	IT Security	Cannot launch to production	Start early, have temporary local auth fallback
Master data export from ERP	IT/Data team	Cannot populate initial data	Manual import as backup
Budget approval	CF0	Project cannot start	Secure early commitment
User availability for training	Department heads	Low adoption	Record sessions, provide self-service

****Internal Dependencies:****

Dependency	Owner	Required By	Status
Requirements sign-off	Business stakeholders	Week 2	Pending
Design approval	Product owner	Week 4	Pending
UAT environment	DevOps	Week 10	Pending
Production infrastructure	DevOps	Week 14	Pending

**15. APPROVAL & SIGN-OFF**

15.1 Document Approvers

Name	Role	Signature	Date
[Name]	Chief Marketing Officer (Executive Sponsor)	_____	_____
[Name]	Chief Financial Officer	_____	_____

[Name]	Finance Director	_____	_____
[Name]	IT Director	_____	_____
[Name]	Product Owner	_____	_____

15.2 Sign-Off Criteria

This BRD is considered approved when:

- [] All Must Have requirements are agreed upon
- [] Scope boundaries are clear and accepted
- [] Budget and timeline are approved
- [] All executive sponsors have signed off
- [] No major unresolved issues remain

15.3 Change Control Process

****After sign-off, any changes to this BRD must:****

1. Be submitted via formal change request
2. Include impact analysis (scope, time, cost)
3. Be reviewed by Product Owner
4. Be approved by Executive Sponsor
5. Be documented and communicated to all stakeholders

****Change Request Template:****

- CR ID: [Unique identifier]
- Requested by: [Name, Role]
- Date: [Date]
- Description: [What needs to change]
- Justification: [Why this change is needed]
- Impact: [On scope, timeline, budget]
- Priority: [Critical, High, Medium, Low]
- Recommendation: [Approve/Defer/Reject]

16. APPENDICES

Appendix A: Glossary

Term	Definition
-----	-----

****COGS****	Cost of Goods Sold - direct cost per unit
****CPL****	Customer Planning Level - grouping of customers for planning purposes
****FU****	Forecasting Unit - grouping of SKUs for promotion planning
****GP****	Gross Profit = Turnover - COGS
****GSV****	Gross Sales Value - sales at list price before discounts
****iGP****	Incremental Gross Profit - additional profit from promotion
****iVol****	Incremental Volume - additional volume from promotion
****KPI****	Key Performance Indicator - measurable value
****LTA****	Long-Term Agreement - ongoing trade agreements
****NIV****	Net Invoice Value - sales after on-invoice discounts
****RAG****	Red/Amber/Green status indicator
****ROI****	Return on Investment - profit per dollar spent
****SKU****	Stock Keeping Unit - individual product
****TPM****	Trade Promotion Management
****TPR****	Temporary Price Reduction

Appendix B: Formula Library Reference

See separate KPI Library document (attached) for complete formulas.

Appendix C: Sample Workflows

****Workflow 1: Create and Submit Plan****

1. Planner logs in → Plan List screen
2. Clicks "+ New Plan"
3. Enters: Plan name, selects CPL and Category, sets dates
4. System creates plan with status "Draft"
5. Planner clicks "Add FU"
6. Searches and selects FUs
7. System loads FUs with child SKUs
8. Planner enters planned volumes at SKU level
9. Planner enters tactics at FU level
10. System calculates all KPIs in real-time
11. Planner reviews RAG status, optimizes RED items
12. Planner clicks "Submit for Approval"
13. System validates, changes status to "Pending Approval"
14. CM receives notification

1. CM logs in → sees notification
2. Opens Approval Queue
3. Clicks on pending plan
4. Reviews Grand Totals, RAG breakdown
5. Reviews planning grid (read-only)
6. Reads planner comments
7. If satisfied: Adds comments, clicks "Approve"
 - System changes status to "Approved"
 - Budget is deducted
 - Planner notified
8. If not satisfied: Adds comments explaining why, clicks "Reject"
 - System changes status to "Rejected"
 - Planner notified with comments

****Workflow 3: Monitor Budget****

1. FM logs in → Financial Dashboard
2. Selects filters (channel, category, period)
3. Reviews budget utilization progress bars
4. Sees alert: Off-Invoice at 90%
5. Drills down to see which plans consumed budget
6. Exports report to share with management
7. Adjusts budget allocation if needed

Appendix D: Screen Mockups

[Link to Figma/Design Files]

Appendix E: Technical Architecture Diagram

[Link to Architecture Document]

Appendix F: Project Timeline (High-Level)

Phase	Duration	Key Milestones
Requirements & Design	Weeks 1-4	Requirements finalized, Design approved, BRD signed off
Development - Sprint 1	Weeks 5-6	Master data management, User management, Authentication

| ****Development - Sprint 3**** | Weeks 9-10 | Real-time calculations,
Approval workflow, Budget management |
| ****Development - Sprint 4**** | Weeks 11-12 | Reporting dashboard, RAG
visualization, Grand totals panel |
| ****Testing & UAT**** | Weeks 13-14 | System testing, UAT, Bug fixes |
| ****Training & Go-Live Prep**** | Week 15 | User training, Data migration,
Final prep |
| ****Production Launch**** | Week 16 | Go-live, Hypercare support |
| ****Post-Launch Support**** | Weeks 17-20 | Bug fixes, Optimization,
Adoption monitoring |

Appendix G: Contact Information

Role	Name	Email	Phone
Executive Sponsor (CMO)	[Name]	[Email]	[Phone]
Product Owner	[Name]	[Email]	[Phone]
Business Analyst	[Name]	[Email]	[Phone]
Development Lead	[Name]	[Email]	[Phone]
Finance SME	[Name]	[Email]	[Phone]
Trade Marketing SME	[Name]	[Email]	[Phone]

DOCUMENT REVISION HISTORY

Version	Date	Author	Changes
0.1	[Date]	[Name]	Initial draft
0.2	[Date]	[Name]	Added user stories and workflows
0.3	[Date]	[Name]	Incorporated stakeholder feedback
1.0	2025-11-04	Product Team	Final version for sign-off

****END OF BUSINESS REQUIREMENTS DOCUMENT****

****Next Steps:****

2. Address any questions or concerns
3. Obtain formal sign-offs
4. Kick off design phase
5. Begin development planning

RAG System Architecture & Detailed Prompts

Prompt A: RAG Threshold Management - Admin Configuration

Create a comprehensive RAG Threshold Configuration system in the Admin Panel:

1. RAG Configuration Screen (Admin → KPI Management → Configure RAG)

Screen Layout:

Header Section:

- Title: "RAG Threshold Configuration"
- Subtitle: "Define color-coded performance indicators for real-time plan evaluation"
- Info icon with tooltip: "RAG thresholds determine visual feedback in the planning grid. Green = Excellent, Amber = Acceptable, Red = Poor"

Main Configuration Interface:

A. KPI Selection Panel (Left Side):

- List all KPIs where RAG is applicable (primarily ROI-related metrics)
- Each KPI card shows:
 - * KPI Name (e.g., "GP ROI %")
 - * Current RAG status: Active/Inactive toggle
 - * Last modified date and by whom
 - * "Configure" button

B. Threshold Configuration Panel (Right Side):

Opens when "Configure" is clicked:

```
| Configure RAG for: GP ROI % |
|-----|
| |
| Enable RAG Status: [✓] Active |
| |
| Threshold Type: |
|   ○ Percentage Based |
|   ○ Absolute Value |
|   ○ Custom Formula |
| |
| └─ GREEN (Excellent Performance) ──── |
| | Condition: Greater than or equal to | |
| | Value: [____20____] % | |
| | Color Preview: [████] #10B981 | |
| | Label Text: "Excellent ROI" | |
| |-----|
| |
| └─ AMBER (Acceptable Performance) ──── |
| | Condition: Between | |
| | Lower: [____10____] % | |
| | Upper: [____20____] % (exclusive) | |
| | Color Preview: [████] #F59E0B | |
| | Label Text: "Acceptable ROI" | |
| |-----|
| |
| └─ RED (Poor Performance) ──── |
| | Condition: Less than | |
| | Value: [____10____] % | |
| | Color Preview: [████] #EF4444 | |
| | Label Text: "Poor ROI - Review Plan" | |
| |-----|
| |
| └─ GRAY (No Data / Not Calculated) ──── |
| | Applies when: Value is NULL or 0 | |
| | Color Preview: [████] #9CA3AF | |
| | Label Text: "Insufficient Data" | |
| |-----|
| |
| [Preview Scenarios] button |
| |
```

2. Advanced Features:

A. Multiple Threshold Sets:

Allow creating different threshold configurations for different contexts:

Example use cases:

- Different thresholds for different Channels (NKA vs Traditional Trade)
- Different thresholds for different Categories (Dairy vs Beverages)
- Seasonal adjustments (Holiday promotions vs Regular promotions)

Interface:

Threshold Set Management
Active Threshold Set: [Default ▼]
Available Sets:
• Default (System-wide)
• NKA Channel (Channel-specific)
• Holiday Promotions (Seasonal)
• High Value Categories (Category)
[+ Create New Threshold Set]

When creating new set:

- Set Name*
- Apply to: [Dropdown: All Plans / Specific Channel / Specific Category / Date Range]
- Clone from: [Dropdown: Start from scratch / Copy from existing set]
- Priority: [1-10] (higher priority overrides lower when multiple sets match)

B. Preview Scenarios Tool:

Modal that opens to test thresholds:

```

| Test RAG Thresholds |
|-----|
| |
| Test GP ROI % values: |
| |
| Input Value: [____25____] % |
| Result: [████] GREEN - Excellent ROI |
| |
| Input Value: [____15____] % |
| Result: [████] AMBER - Acceptable ROI |
| |
| Input Value: [____5____] % |
| Result: [████] RED - Poor ROI |
| |
| └─ Quick Test Range ──────────── |
| | Test values from [-10] to [50] by 5 | |
| | | |
| | -10%: [████] RED | |
| | -5%: [████] RED | |
| | 0%: [████] RED | |
| | 5%: [████] RED | |
| | 10%: [████] AMBER | |
| | 15%: [████] AMBER | |
| | 20%: [████] GREEN | |
| | 25%: [████] GREEN | |
| | 30%: [████] GREEN | |
| ──────────────────── |
| |
| [Close] |
|-----|

```

3. Database Schema:

```

CREATE TABLE rag_configurations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  config_name VARCHAR(100) NOT NULL,
  kpi_id UUID REFERENCES kpis(id) ON DELETE CASCADE,
  is_active BOOLEAN DEFAULT true,
  threshold_type VARCHAR(50), -- 'percentage', 'absolute', 'formula'
  -- Green thresholds

```

```

green_value_lower DECIMAL(10,2),
green_value_upper DECIMAL(10,2),
green_color VARCHAR(7) DEFAULT '#10B981',
green_label TEXT DEFAULT 'Excellent Performance',
-- Amber thresholds
amber_condition VARCHAR(20),
amber_value_lower DECIMAL(10,2),
amber_value_upper DECIMAL(10,2),
amber_color VARCHAR(7) DEFAULT '#F59E0B',
amber_label TEXT DEFAULT 'Acceptable Performance',
-- Red thresholds
red_condition VARCHAR(20),
red_value_lower DECIMAL(10,2),
red_value_upper DECIMAL(10,2),
red_color VARCHAR(7) DEFAULT '#EF4444',
red_label TEXT DEFAULT 'Poor Performance',
-- Gray (no data)
gray_color VARCHAR(7) DEFAULT '#9CA3AF',
gray_label TEXT DEFAULT 'Insufficient Data',
-- Applicability rules
apply_to_channel VARCHAR(50), -- NULL = all channels
apply_to_category VARCHAR(100), -- NULL = all categories
apply_to_date_start DATE, -- NULL = no date restriction
apply_to_date_end DATE,
priority INTEGER DEFAULT 0, -- Higher number = higher priority
created_by UUID REFERENCES users(id),
created_at TIMESTAMP DEFAULT NOW(),
updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_rag_config_kpi ON rag_configurations(kpi_id);
CREATE INDEX idx_rag_config_active ON rag_configurations(is_active);
CREATE INDEX idx_rag_config_priority ON rag_configurations(priority
DESC);

```

4. Validation Rules:

Implement these validations:

1. Threshold Overlap Prevention:

- System should auto-validate: "Green starts where Amber ends"
- Show warning if gaps exist between ranges

2. Logical Consistency:

- For percentage-based: values must be between -100 and 100
- Upper value must be greater than lower value
- At least one threshold must be defined

3. Priority Conflicts:

- Warn if two active configurations with same priority could apply to same plan
- Show conflict resolution preview

5. User Interface Components:

Build reusable components:

1. ColorPicker Component:

- Preset color swatches
- Custom hex input
- Live preview

2. ThresholdSlider Component:

- Visual range slider showing Green/Amber/Red zones
- Draggable boundaries
- Numeric inputs synced with slider

3. RAGStatusBadge Component (for display):

- Props: value, ragConfig, showLabel, size
- Renders colored badge with optional tooltip
- Variations: dot, badge, cell background

Example usage in planning grid:

```
<RAGStatusBadge
value={15.5}
ragConfig={gpRoiConfig}
showLabel={true}
size="md"
/>
// Renders: [■■■■ AMBER] "Acceptable ROI"
```

6. Admin Experience Features:

1. Bulk Configuration:

- "Apply to all KPIs" checkbox
- Copy configuration to multiple KPIs at once

2. Configuration Templates:

- Save as template: "Aggressive Targets", "Conservative Targets"
- Load from template
- Share templates across organizations (future feature)

3. History & Audit:

- View threshold change history
- Compare old vs new configurations
- Rollback to previous configuration
- Impact analysis: "X plans will change from Green to Amber"

4. Import/Export:

- Export configurations as JSON
- Import from file (with validation)

Log all RAG configuration changes to audit_log with detailed before/after values.

Prompt B: RAG Display in Planning Grid

Implement RAG visual indicators in the Planning Grid:

1. Grid Cell RAG Display

For each KPI column that has RAG enabled:

Cell Rendering Options (Admin configurable per KPI):

Option A - Background Color:

┌
| GP ROI %: 25.5% | <- Green background (#10B981 at 20% opacity)

Option C - Badge Style:

Option D - Full Color Fill (Bold):

2. Interactive RAG Features:

A. Hover Tooltip:

When user hovers over RAG-colored cell:

B. RAG Drill-Down Model:

Click on RAG indicator opens detail:

```

| Threshold Breakdown: |
| └─ Green Zone: ≥ 20% [You are here: 25.5%]|
| └─ Amber Zone: 10-20% |
| └─ Red Zone: < 10% |
| |
| └─ Visual Range ────────────────────────────| | | | |
| | | |
| | Red | Amber | Green | |
| | ────────────|───────────|───────────| |
| | 0% 10% 20% ▲ 25.5% | |
| | | |
| └───────────────────────────────────────────| |
| |
| To reach next level (N/A - already optimal) |
| |
| [Close] |
└───────────────────────────────────────────┘

```

3. Row-Level RAG Aggregation:

For FU rows (parent rows that aggregate SKU data):

Display logic:

- If ALL child SKUs are Green → FU row is Green
- If ANY child SKU is Red → FU row is Red
- If mixed Green/Amber → FU row is Amber
- Show breakdown on hover: "3 Green, 2 Amber, 0 Red"

Visual indicator on FU row:

```

┌───────────────────────────────────────────┐
| [+] FU: Danone Yogurt 4x100g |
| GP ROI: 22.3% [■■■] (3●2●0●) |
└───────────────────────────────────────────┘

```

└─ Green/Amber/Red count

4. Grand Totals Panel RAG:

Display plan-level RAG status prominently:

```

┌───────────────────────────────────────────┐

```

Overall Plan Status			
[████████]			
GREEN			
Excellent Performance			
GP ROI: 24.8%			
Breakdown by Item:			
• Green: 12 items (75%)			
• Amber: 3 items (19%)			
• Red: 1 item (6%)			
[View Items by Status ▼]			

5. Dynamic RAG Calculation Function:

TypeScript implementation:

```
```typescript
interface RAGConfig {
 green_condition: 'gte' | 'lte' | 'between' | 'equals';
 green_value_lower: number;
 green_value_upper?: number;
 amber_condition: 'between';
 amber_value_lower: number;
 amber_value_upper: number;
 red_condition: 'lt' | 'gt';
 red_value_lower: number;
 green_color: string;
 amber_color: string;
 red_color: string;
 gray_color: string;
 green_label: string;
 amber_label: string;
 red_label: string;
}
```

```
}
```

```
interface RAGResult {
 status: 'green' | 'amber' | 'red' | 'gray';
 color: string;
 label: string;
 value: number;
 thresholdInfo: string;
}
```

```
function calculateRAGStatus(
 value: number | null | undefined,
 config: RAGConfig
): RAGResult {
 // Handle null/undefined values
 if (value === null || value === undefined || isNaN(value)) {
 return {
 status: 'gray',
 color: config.gray_color,
 label: config.gray_label,
 value: 0,
 thresholdInfo: 'No data available'
 };
 }
```

```
 // Check Green condition
 if (config.green_condition === 'gte' && value >=
 config.green_value_lower) {
 return {
 status: 'green',
 color: config.green_color,
 label: config.green_label,
 value: value,
 thresholdInfo: `≥ ${config.green_value_lower}% = Green`
 };
 }
```

```
 if (config.green_condition === 'lte' && value <=
 config.green_value_lower) {
 return {
```

```
color: config.green_color,
label: config.green_label,
value: value,
thresholdInfo: `≤ ${config.green_value_lower}% = Green`
};
}
```

```
if (
config.green_condition === 'between' &&
value >= config.green_value_lower &&
value <= config.green_value_upper!
) {
return {
status: 'green',
color: config.green_color,
label: config.green_label,
value: value,
thresholdInfo: `${config.green_value_lower}% -
${config.green_value_upper}% = Green`
};
}
```

```
// Check Amber condition
if (
value >= config.amber_value_lower &&
value < config.amber_value_upper
) {
return {
status: 'amber',
color: config.amber_color,
label: config.amber_label,
value: value,
thresholdInfo: `${config.amber_value_lower}% -
${config.amber_value_upper}% = Amber`
};
}
```

```
// Default to Red
return {
status: 'red',
```

```

label: config.red_label,
value: value,
thresholdInfo: `< ${config.red_value_lower}% = Red`
};
}

// Usage in grid
const ragResult = calculateRAGStatus(gpRoiValue, ragConfig);
```

```

6. Filtering and Sorting by RAG:

Add filter options above grid:

| |
|---|
| Show only:
 ☒ Green ☒ Amber ☒ Red ☐ Gray |
|---|

Table header for RAG-enabled columns:

| |
|---|
| GP ROI % <input type="button" value="▼"/> <input type="button" value="●"/> <- Sort by value <input type="button" value="▼"/> , Filter by RAG <input type="button" value="●"/> |
|---|

Click opens:

| |
|--|
| Filter by Status
 ☒ Green
 ☒ Amber
 ☐ Red
 ☐ No Data

 [Apply][Clear] |
|--|

7. Export with RAG:

When exporting to Excel:

- Apply conditional formatting with same colors
- Add legend sheet explaining RAG thresholds

8. Mobile RAG Display:

On mobile devices, use compact RAG indicators:

```
| FU: Danone Yogurt |
| • GP ROI: 25.5% | <- Green dot
| Tap for details |
```

Store RAG calculation results in plan cache for performance.

Prompt C: RAG in Approval & Reporting

Implement RAG indicators in Category Manager and Finance Manager interfaces:

1. CM Approval Screen RAG Summary:

When CM reviews a plan, show RAG distribution prominently:

```
| Plan: Summer Promo - Carrefour Dairy |
Submitted by: John Planner on May 15, 2025
```

| | | | |
|---|---------|-------|--------|
| 16 Green | 3 Amber | 1 Red | 0 Gray |
| ITEMS REQUIRING ATTENTION | | | |
| <p>▲ Red Status Items (1):</p> <ul style="list-style-type: none"> SKU: Activia Strawberry 500g GP ROI: 7.2% (Below 10% threshold) Reason: High display fee [View Details] | | | |
| <p>< Amber Status Items (3):</p> <ul style="list-style-type: none"> FU: Danone Natural Yogurt Line GP ROI: 14.5% FU: Activia Probiotic Range GP ROI: 16.8% FU: Greek Yogurt 200g GP ROI: 12.3% | | | |
| <p>🚦 Recommendation:</p> <p>Plan is generally strong, but consider reviewing the Activia Strawberry 500g promotion terms before approval.</p> <p>[Approve] [Request Changes] [Reject]</p> | | | |

2. Auto-Rejection Rules (Optional Admin Feature):

Allow admins to set automatic validation rules:

| |
|--|
| Automatic Validation Rules |
| <p>☒ Block submission if ANY item is Red</p> <p>☒ Warn if >20% items are Amber or Red</p> |

```
| [✓] Block if Overall Plan ROI is Red |
| |
| Grace Period: Allow [2] Red items max |
| |
|_____|
```

When planner tries to submit with violations:

```
|_____|
| ⚠ Cannot Submit Plan |
|_____|
| |
| This plan contains 3 items with RED |
| status, which exceeds the allowed |
| maximum of 2. |
| |
| Red Status Items: |
| • Activia Strawberry 500g (ROI: 7.2%) |
| • Greek Yogurt Plain (ROI: 5.8%) |
| • Natural Set Yogurt (ROI: 8.9%) |
| |
| Please improve these items or remove |
| them from the plan before submitting. |
| |
| [Optimize Plan] [Cancel] |
|_____|
```

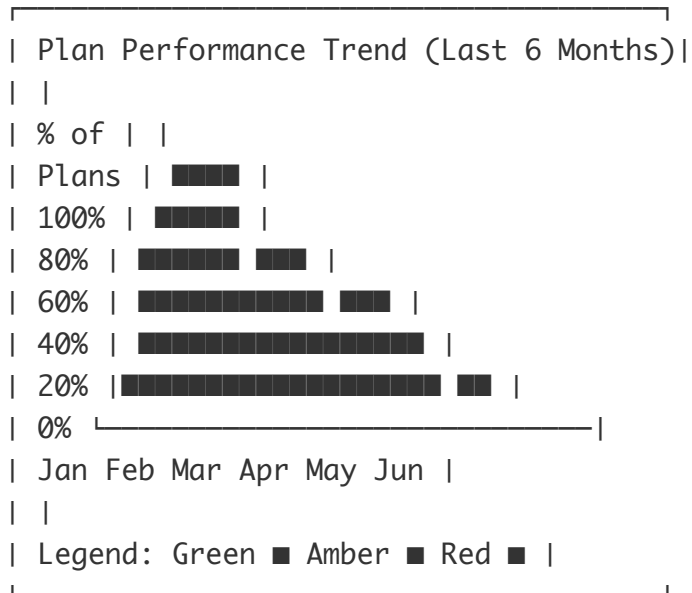
3. Finance Manager Dashboard RAG Widgets:

Add RAG-based widgets to FM dashboard:

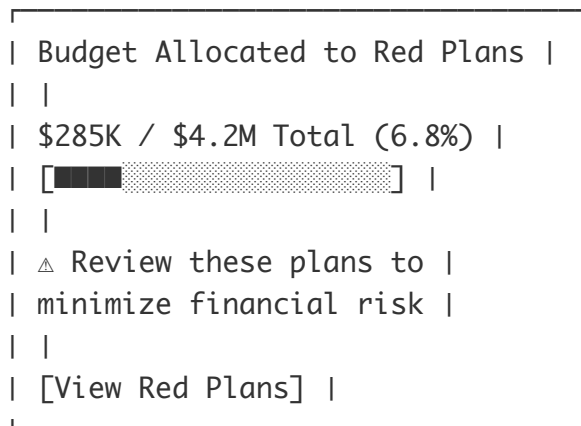
Widget 1: Plan Health Distribution

```
|_____|
| Approved Plans This Month |
| |
| [Donut Chart] |
| ●●●● 65% Green (13) |
| ●● 25% Amber (5) |
| • 10% Red (2) |
| |
```

Widget 2: RAG Trend Over Time



Widget 3: Budget at Risk



4. RAG-Based Alerts & Notifications:

Implement intelligent notification system:

Alert Type 1: Threshold Change Impact

To: All Planners

Subject: RAG Threshold Updated

- Green: $\geq 25\%$ (was 20%)
- Amber: 15-25% (was 10-20%)
- Red: $< 15\%$ (was $< 10\%$)

This affects 12 of your plans:

- 8 plans changed from Green to Amber
- 4 plans changed from Amber to Red

[Review Affected Plans]"

Alert Type 2: Plan Degradation Warning

To: Planner & Category Manager

Subject: Plan Performance Degraded

"Plan 'Summer Promo' overall status changed:
GREEN → AMBER

Reason: Incremental volume target decreased,
reducing GP ROI from 22.5% to 16.8%

[View Plan] [View Changes]"

Alert Type 3: Budget Risk Alert

To: Finance Manager

Subject: High-Risk Budget Allocation

"Warning: 15% of approved budget (\$630K) is
allocated to RED-status plans.

These plans are underperforming ROI targets
and may require intervention.

[View Report] [Review Plans]"

5. RAG in Reports & Analytics:

A. Custom Report Filter:

When building reports, allow filtering by RAG:

| | |
|---|--|
| | |
| Include plans with status: | |
| ☒ Green | |
| ☒ Amber | |
| ☐ Red | |
| | |
| Date range: [Last Quarter ▼] | |
| Channel: [All ▼] | |
| | |
| _____ | |

B. RAG Heatmap Report:
 Matrix showing RAG distribution across dimensions:

Carrefour Migros BIM
 Dairy Green Green Amber
 Beverages Green Amber Red
 Snacks Amber Green Green
 Frozen Red Amber Amber

Click any cell to drill down to those specific plans.

C. Executive Summary with RAG:

| | |
|--|--|
| Q2 2025 Promotion Performance Summary | |
| _____ | |
| | |
| Overall Portfolio Health: GREEN ✓ | |
| | |
| Key Metrics: | |
| • 78% of plans achieved Green status | |
| • Average GP ROI: 23.4% | |
| • Zero Red-status plans in NKA channel | |
| | |
| Areas for Improvement: | |
| • Traditional Trade: 30% Amber/Red | |
| • Frozen category underperforming | |
| | |
| _____ | |

Create materialized view for RAG performance queries:

```
CREATE MATERIALIZED VIEW plan_rag_summary AS
SELECT
p.id as plan_id,
p.plan_name,
p.category,
p.cpl_id,
p.status as plan_status,
-- Calculate overall plan ROI
(SUM(psk.incremental_gp) / NULLIF(SUM(psk.total_spend), 0)) * 100 as
overall_roi,
-- Determine overall RAG status
CASE
WHEN (SUM(psk.incremental_gp) / NULLIF(SUM(psk.total_spend), 0)) * 100
>= rc.green_value_lower
THEN 'green'
WHEN (SUM(psk.incremental_gp) / NULLIF(SUM(psk.total_spend), 0)) * 100
>= rc.amber_value_lower
THEN 'amber'
ELSE 'red'
END as rag_status,
-- Count items by RAG
COUNT(CASE WHEN sku_rag.status = 'green' THEN 1 END) as green_count,
COUNT(CASE WHEN sku_rag.status = 'amber' THEN 1 END) as amber_count,
COUNT(CASE WHEN sku_rag.status = 'red' THEN 1 END) as red_count
```

Enhanced KPI System - Comprehensive Prompts

Based on the detailed KPI library document, I'll create expanded prompts for implementing all KPIs in the admin panel and planning screen.

PHASE A: Enhanced KPI Management - Admin Configuration

Prompt A1: Complete KPI Library Setup with All Metrics

Expand the KPI Management module to include ALL KPIs from the comprehensive library:

1. KPI Management Screen - Enhanced Structure

A. KPI Group Navigation (Left Sidebar):

Create collapsible tree menu:

```
| KPI LIBRARY |
|
| ▼ Master Data (2) |
| • List Price per Piece |
| • COGS per Piece |
| |
| ▼ Volume (4) |
| • Base Volume |
| • Planned Volume |
| • Incremental Volume (iVol) |
| • Volume Uplift % |
| |
| ▼ Gross Sales Value - GSV (3) |
| • Base GSV |
| • Planned GSV |
| • Incremental GSV (iGSV) |
| |
| ▼ Net Invoice Value - NIV (3) |
| • Base NIV |
| • Planned NIV |
| • Incremental NIV (iNIV) |
| |
| ▼ Turnover (4) |
| • Base Turnover |
| • Planned Turnover |
| • Incremental TO (iT0) |
| • TO Uplift % |
| |
| ▼ LTA Spend (8) |
| • LTA On-Invoice % |
| • LTA Off-Invoice % |
| • Base LTA Spend On-Invoice |
```


- | • Planned LTA Spend On-Invoice |
- | • Planned LTA Spend Off-Invoice |
- | • Total Base Spend |
- | |
- | ▼ Promo Spend by Mechanic (11) |
- | • CPP On-Invoice % Spend |
- | • CPP Off-Invoice % Spend |
- | • Price Support per Unit Spend |
- | • Visibility Lumpsum MT/PH |
- | • Visibility Lumpsum GT |
- | • Drive/TPR On-Invoice % Spend |
- | • TPR/Drives Lumpsum Spend |
- | • WS TPR On-Invoice % Spend |
- | • WS TPR Off-Invoice % Spend |
- | |
- | ▼ Total Promo Spend (6) |
- | • Planned Promo Spend On-Invoice |
- | • Planned Promo Spend Off-Invoice |
- | • Total Planned Spend On-Invoice |
- | • Total Planned Spend Off-Invoice |
- | • Total Planned Spend |
- | • Incremental Planned Spend |
- | |
- | ▼ Gross Profit (5) |
- | • Base COGS |
- | • Planned COGS |
- | • Base Gross Profit |
- | • Planned Gross Profit |
- | • Incremental GP (iGP) |
- | |
- | ▼ Gross Margin (3) |
- | • Base Gross Margin % |
- | • Planned Gross Margin % |
- | • Incremental Gross Margin % |
- | |
- | ▼ ROI & RAG (3) |
- | • Planned TO ROI % |
- | • Planned GP ROI % |
- | • RAG Status |
- | |

Show count of KPIs per group in parentheses.

2. KPI Configuration Form - Complete Fields

When Admin clicks on a KPI, open detailed configuration panel:

| | | |
|--|--|--|
| Configure KPI: Incremental Volume (iVol) | | |
| | | |
| | | |
| BASIC INFORMATION | | |
| | | |
| KPI Name*: [Incremental Volume (iVol)] | | |
| Display Name: [iVol] | | |
| KPI Code: [INCR_VOL] (used in formulas) | | |
| KPI Group*: [Volume ▼] | | |
| Sub-Category: [Performance Metrics ▼] | | |
| | | |
| Description: | | |
| | | |
| The additional units expected to be sold because | | |
| of the promotion. It's the lift in volume above | | |
| the baseline (normal sales without promotion). | | |
| | | |
| | | |
| CALCULATION CONFIGURATION | | |
| | | |
| Calculation Method*: | | |
| ○ Formula-Based (Automatic) | | |
| ● User Input (Manual Entry) | | |
| ○ External Data Source | | |
| | | |
| Formula*: | | |
| | | |
| PLANNED_VOL - BASE_VOL | | |
| | | |
| [Validate Formula] [Insert KPI Code ▼] | | |

| | ✓ Formula is valid | |
| | Available variables: PLANNED_VOL, BASE_VOL, BPTT... | |
| | | |
| | Alternative Formula (Optional): | |
| | [For different calculation contexts] | |
| |
| |
| DISPLAY SETTINGS |
| |
| | Unit of Measure*: [Pieces ▼] | |
| | Unit Symbol: [pcs] Display: [After Value ▼] | |
| | | |
| | Number Format: | |
| | • Decimal Places: [0▼] | |
| | • Thousand Separator: [✓] Comma | |
| | • Negative Display: [Red Color ▼] | |
| | • Preview: 1,234 pcs | |
| | | |
| | Display Icon: [] [Choose Icon] | |
| | Display Color: [] #3B82F6 | |
| |
| |
| GRID VISIBILITY |
| |
| | Show in Planning Grid: | |
| | [✓] SKU Level [✓] FU Level [✓] Plan Total | |
| | | |
| | Column Position: [15] (Order in grid) | |
| | Column Width: [Auto ▼] or [120] px | |
| | | |
| | Conditional Display Rules: | |
| | Show only when: [Always visible ▼] | |
| | ○ Always visible | |
| | ○ Only for specific channels | |
| | ○ Only for specific categories | |
| | ○ Based on user role | |
| | | |
| | Cell Highlighting: | |
| | [✓] Highlight when negative (Red background) | |
| | [✓] Highlight when zero (Gray background) | |

```
| |_____| |
| |
| RAG CONFIGURATION (For ROI & Key Metrics) |
	_____			
	Enable RAG Status: [✓] Active			
	Threshold Configuration:			
	└ GREEN (Excellent) _____			
		Condition: [Greater than or equal to ▼]		
		Value: [____0____] pcs		
		Color: [████] #10B981		
		Label: "Positive Volume Uplift"		
		Show Badge: [✓] Show Icon: [✓]		
		_____		
	└ AMBER (Caution) _____			
		Not applicable for this KPI		
		[Skip Amber threshold]		
		_____		
	└ RED (Poor) _____			
		Condition: [Less than or equal to ▼]		
		Value: [____0____] pcs		
		Color: [████] #EF4444		
		Label: "No Volume Uplift - Review Plan"		
		Alert Planner: [✓] Block Submission: [ ]		
		_____		
	[Preview RAG Scenarios]			
	_____			
BUSINESS RULES & VALIDATIONS				
	_____			
	Data Validation:			
	[ ] Must be positive			
	[✓] Cannot be null			
	[ ] Must be greater than: [_____]			
	[ ] Must be less than: [_____]			
```

| | ☐ Warn if exceeds: [_____] pcs | |
| | ☒ Warn if negative: "Planned volume is below base" | |
| | | |
| | Dependencies: | |
| | Required KPIs: PLANNED_VOL, BASE_VOL | |
| | Impacts: VOLUME_UPLIFT_PCT, iGSV, all profit metrics | |
| | [\[View Dependency Tree\]](#) | |
| |
| |
| REPORTING & ANALYTICS |
| |
| | Include in: | |
| | ☒ Executive Dashboard | |
| | ☒ Plan Summary Reports | |
| | ☒ Category Manager View | |
| | ☒ Finance Dashboard | |
| | ☐ External API | |
| | | |
| | Aggregation Method: | |
| | • At FU Level: [SUM ▼] | |
| | • At Plan Level: [SUM ▼] | |
| | • At CPL Level: [SUM ▼] | |
| | | |
| | Benchmarking: | |
| | ☒ Track historical averages | |
| | ☒ Compare against category benchmarks | |
| | ☐ Show industry standards | |
| |
| |
| HELP & DOCUMENTATION |
| |
| | Tooltip Text (Hover Help): | |
| |

Additional volume gained above baseline due to
promotion. Positive = Success. Zero/Negative =
Promotion failed to lift volume.

 | |
| | | |
| | User Guide Link: [\[Add documentation URL\]](#) | |
| | Tutorial Video: [\[Upload or link video\]](#) | |

```

| |
| AUDIT TRAIL |
| _____ |
	Created by: John Admin on Jan 15, 2025	
	Last Modified: Sarah Admin on Mar 22, 2025	
	Version: 2.1	
	[View Change History]	
_____		
[Cancel] [Save as Draft] [Save & Activate]		
_____		

```

3. Enhanced Database Schema for Complete KPI System

```

CREATE TABLE kpis (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
-- Basic Info
kpi_code VARCHAR(50) UNIQUE NOT NULL, -- e.g., 'INCR_VOL'
kpi_name VARCHAR(200) NOT NULL,
display_name VARCHAR(100),
kpi_group VARCHAR(100) NOT NULL, -- Master Data, Volume, GSV, etc.
sub_category VARCHAR(100),
description TEXT,
-- Calculation
calculation_method VARCHAR(50) NOT NULL, -- 'formula', 'user_input',
'external'
formula TEXT, -- The actual formula: "PLANNED_VOL - BASE_VOL"
alternative_formula TEXT, -- For special contexts
formula_validated BOOLEAN DEFAULT false,
depends_on JSONB, -- Array of KPI codes this depends on
impacts JSONB, -- Array of KPI codes impacted by this
-- Display Settings
unit_of_measure VARCHAR(50), -- 'pieces', 'currency', 'percentage'
unit_symbol VARCHAR(20), -- 'pcs', '$', '%'
unit_position VARCHAR(20) DEFAULT 'after', -- 'before' or 'after'
decimal_places INTEGER DEFAULT 2,
use_thousand_separator BOOLEAN DEFAULT true,
negative_display VARCHAR(50) DEFAULT 'red_color', -- 'red_color',
'parentheses', 'minus'
display_icon VARCHAR(50),
display_color VARCHAR(7),

```

```
-- Grid Visibility
show_in_grid_sku BOOLEAN DEFAULT true,
show_in_grid_fu BOOLEAN DEFAULT true,
show_in_grid_plan BOOLEAN DEFAULT true,
column_position INTEGER,
column_width VARCHAR(20), -- 'auto' or pixel value
conditional_display_rule VARCHAR(50), -- 'always', 'channel',
'category', 'role'
conditional_display_config JSONB,
-- Cell Highlighting
highlight_negative BOOLEAN DEFAULT false,
highlight_zero BOOLEAN DEFAULT false,
highlight_top_performers BOOLEAN DEFAULT false,
-- RAG Configuration
rag_enabled BOOLEAN DEFAULT false,
rag_green_condition VARCHAR(20),
rag_green_value_lower DECIMAL(15,4),
rag_green_value_upper DECIMAL(15,4),
rag_green_color VARCHAR(7) DEFAULT '#10B981',
rag_green_label TEXT,
rag_green_show_badge BOOLEAN DEFAULT true,
rag_green_show_icon BOOLEAN DEFAULT true,
rag_amber_enabled BOOLEAN DEFAULT true,
rag_amber_condition VARCHAR(20),
rag_amber_value_lower DECIMAL(15,4),
rag_amber_value_upper DECIMAL(15,4),
rag_amber_color VARCHAR(7) DEFAULT '#F59E0B',
rag_amber_label TEXT,
rag_amber_show_badge BOOLEAN DEFAULT true,
rag_red_condition VARCHAR(20),
rag_red_value_lower DECIMAL(15,4),
rag_red_value_upper DECIMAL(15,4),
rag_red_color VARCHAR(7) DEFAULT '#EF4444',
rag_red_label TEXT,
rag_red_alert_planner BOOLEAN DEFAULT false,
rag_red_block_submission BOOLEAN DEFAULT false,
rag_gray_color VARCHAR(7) DEFAULT '#9CA3AF',
rag_gray_label TEXT DEFAULT 'No Data',
-- Business Rules
validation_required BOOLEAN DEFAULT false,
```

```

validation_cannot_be_null BOOLEAN DEFAULT false,
validation_min_value DECIMAL(15,4),
validation_max_value DECIMAL(15,4),
warning_threshold_upper DECIMAL(15,4),
warning_threshold_lower DECIMAL(15,4),
warning_message TEXT,
-- Reporting
include_in_exec_dashboard BOOLEAN DEFAULT false,
include_in_plan_summary BOOLEAN DEFAULT true,
include_in_cm_view BOOLEAN DEFAULT true,
include_in_fm_view BOOLEAN DEFAULT true,
include_in_api BOOLEAN DEFAULT false,
aggregation_method_fu VARCHAR(20) DEFAULT 'sum', -- sum, avg, min, max
aggregation_method_plan VARCHAR(20) DEFAULT 'sum',
aggregation_method_cpl VARCHAR(20) DEFAULT 'sum',
track_historical_avg BOOLEAN DEFAULT false,
compare_against_benchmarks BOOLEAN DEFAULT false,
-- Help
tooltip_text TEXT,
user_guide_url VARCHAR(500),
tutorial_video_url VARCHAR(500),
-- Status
status VARCHAR(20) DEFAULT 'active', -- active, draft, archived
version VARCHAR(20) DEFAULT '1.0',
-- Audit
created_by UUID REFERENCES users(id),
created_at TIMESTAMP DEFAULT NOW(),
updated_by UUID REFERENCES users(id),
updated_at TIMESTAMP DEFAULT NOW()
);

-- Index for performance
CREATE INDEX idx_kpis_group ON kpis(kpi_group);
CREATE INDEX idx_kpis_code ON kpis(kpi_code);
CREATE INDEX idx_kpis_status ON kpis(status);
CREATE INDEX idx_kpis_grid_visibility ON kpis(show_in_grid_sku,
show_in_grid_fu, show_in_grid_plan);

-- KPI Change History
CREATE TABLE kpi_change_history (

```



```

kpi_id UUID REFERENCES kpis(id),
changed_by UUID REFERENCES users(id),
change_type VARCHAR(50), -- 'created', 'formula_changed', 'rag_updated',
'archived'
field_changed VARCHAR(100),
old_value TEXT,
new_value TEXT,
change_reason TEXT,
created_at TIMESTAMP DEFAULT NOW()
);

```

```

-- KPI Context-Specific Overrides
CREATE TABLE kpi_context_overrides (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
kpi_id UUID REFERENCES kpis(id),
context_type VARCHAR(50), -- 'channel', 'category', 'cpl', 'date_range'
context_value VARCHAR(200),
override_field VARCHAR(100), -- 'formula', 'rag_thresholds',
'display_format'
override_config JSONB,
priority INTEGER DEFAULT 0,
is_active BOOLEAN DEFAULT true,
created_at TIMESTAMP DEFAULT NOW()
);

```

4. Bulk KPI Setup - Seed Data Script

Create a seed data function to populate all KPIs from the library:

```

```sql
-- Insert all Master Data KPIs
INSERT INTO kpis (kpi_code, kpi_name, kpi_group, formula,
unit_of_measure, unit_symbol, description, calculation_method) VALUES
('BPTT', 'List Price per Piece', 'Master Data', 'FROM_MASTER_DATA',
'currency', '$', 'Standard list price for a single unit before
discounts', 'external'),
('COGS', 'COGS per Piece', 'Master Data', 'FROM_MASTER_DATA',
'currency', '$', 'Direct cost to produce or acquire one piece',
'external');

```

```

-- Insert all Volume KPIs

```

```

INSERT INTO kpis (kpi_code, kpi_name, display_name, kpi_group, formula,
unit_of_measure, unit_symbol, description, calculation_method,
show_in_grid_sku, show_in_grid_fu, show_in_grid_plan, column_position)
VALUES
('BASE_VOL', 'Base Volume', 'Base Vol', 'Volume',
'FROM_HISTORICAL_DATA', 'pieces', 'pcs', 'Expected sales volume without
promotion', 'external', true, true, true, 10),
('PLANNED_VOL', 'Planned Volume', 'Planned Vol', 'Volume', 'USER_INPUT',
'pieces', 'pcs', 'Projected units to be sold during promotion',
'user_input', true, true, true, 11),
('INCR_VOL', 'Incremental Volume (iVol)', 'iVol', 'Volume', 'PLANNED_VOL
- BASE_VOL', 'pieces', 'pcs', 'Additional units sold due to promotion',
'formula', true, true, true, 12),
('VOL_UPLIFT_PCT', 'Volume Uplift %', 'Vol Uplift %', 'Volume',
'(INCR_VOL / BASE_VOL) * 100', 'percentage', '%', 'Percentage increase
in volume', 'formula', true, true, true, 13);

```

```
-- Insert GSV KPIs
```

```

INSERT INTO kpis (kpi_code, kpi_name, display_name, kpi_group, formula,
unit_of_measure, unit_symbol, calculation_method, column_position)
VALUES
('BASE_GSV', 'Base Gross Sales Value', 'Base GSV', 'Gross Sales Value',
'BPTT * BASE_VOL', 'currency', '$', 'formula', 14),
('PLANNED_GSV', 'Planned Gross Sales Value', 'Planned GSV', 'Gross Sales
Value', 'BPTT * PLANNED_VOL', 'currency', '$', 'formula', 15),
('INCR_GSV', 'Incremental GSV (iGSV)', 'iGSV', 'Gross Sales Value',
'PLANNED_GSV - BASE_GSV', 'currency', '$', 'formula', 16);

```

```
-- Continue for all other KPI groups...
```

```
-- (NIV, Turnover, LTA, Promo Spend, Gross Profit, Gross Margin, ROI)
...
```

## ## 5. KPI Formula Validator

Build intelligent formula validation:

```

```typescript
interface FormulaValidationResult {
  isValid: boolean;
  errors: string[];
  warnings: string[];

```

```
circularReferences: string[];  
}
```

```
function validateKPIFormula(  
  formula: string,  
  kpiCode: string,  
  allKPIs: KPI[]  
): FormulaValidationResult {  
  const result: FormulaValidationResult = {  
    isValid: true,  
    errors: [],  
    warnings: [],  
    dependencies: [],  
    circularReferences: []  
  };  
};
```

```
// Extract KPI codes referenced in formula  
const referencedKPIs = extractKPICodes(formula);  
result.dependencies = referencedKPIs;
```

```
// Check if all referenced KPIs exist  
referencedKPIs.forEach(ref => {  
  const kpiExists = allKPIs.find(k => k.kpi_code === ref);  
  if (!kpiExists) {  
    result.errors.push(`Referenced KPI '${ref}' does not exist`);  
    result.isValid = false;  
  }  
});
```

```
// Check for circular dependencies  
const circular = detectCircularDependency(kpiCode, formula, allKPIs);  
if (circular.length > 0) {  
  result.circularReferences = circular;  
  result.errors.push(`Circular dependency detected: ${circular.join(' → ')}`);  
  result.isValid = false;  
}
```

```
// Validate mathematical operations  
try {
```

```

const parsed = parseFormula(formula);
if (!parsed.valid) {
  result.errors.push(`Invalid formula syntax: ${parsed.error}`);
  result.isValid = false;
}
} catch (e) {
  result.errors.push(`Formula parsing error: ${e.message}`);
  result.isValid = false;
}

// Check for division by zero risks
if (formula.includes('/')) {
  const divisors = extractDivisors(formula);
  divisors.forEach(divisor => {
    result.warnings.push(`Ensure ${divisor} is never zero to avoid division
errors`);
  });
}

return result;
}
...

```

6. KPI Import/Export Feature

Allow admins to export/import KPI configurations:

Modal: "Import/Export KPI Library"

Export Options:

Export KPI Configuration
Select KPI Groups:
☒ All KPIs (57 total)
☐ Volume KPIs only
☐ ROI & Profit KPIs only
☐ Custom selection
Export Format:

```
| ○ Excel (Editable template) |
| ○ CSV (Basic info only) |
| |
| Include: |
| [✓] RAG configurations |
| [✓] Display settings |
| [✓] Formulas |
| [✓] Validation rules |
| |
| [Cancel] [Export] |
└──────────────────────────────────┘
```

Import Wizard:

Step 1: Upload file (JSON/Excel/CSV)
Step 2: Map fields (if different structure)
Step 3: Validate formulas and dependencies
Step 4: Preview changes
Step 5: Confirm import (with option to skip conflicts)

7. KPI Performance Monitoring Dashboard

Create admin dashboard showing KPI health:

```
┌────────────────────────────────────────────────────────────────────────────────┐
| KPI SYSTEM HEALTH MONITOR |
└────────────────────────────────────────────────────────────────────────────────┘
| |
| < Calculation Performance |
| • Average calc time: 247ms |
| • Slowest KPI: Planned GP ROI % (1.2s) |
| • Failed calculations (24h): 3 |
| |
| 📊 Usage Statistics |
| • Most viewed KPI: GP ROI % |
| • Most filtered KPI: RAG Status |
| • Unused KPIs: 5 |
| |
| ⚠ Issues & Warnings |
| • 2 KPIs have circular dependencies (Review) |
| • 1 KPI formula failed validation |
```

	 Recent Changes
	• GP ROI thresholds updated (2h ago)
	• New KPI added: Net Profit Margin (1d ago)

Track formula execution times and optimize slow calculations.

Prompt A2: Promo Mechanics Input Configuration

Create dedicated Promo Mechanics Management for user input parameters:

1. Promo Mechanics Library (Admin Setup)

These are the INPUT tactics users enter in planning screen.

Create separate section in Admin Panel:

	PROMO MECHANICS CONFIGURATION

S
Continue

```

| |
| These are user-input tactics that drive KPI |
| calculations. Configure which mechanics are |
| available and their validation rules. |
| |
|-----|
| |
| ON-INVOICE DISCOUNTS |
|-----|
| | |
| | ✓ CPP On-Invoice % | |
| | Input Type: Percentage (0-100%) | |
| | Calculation: (PlannedGSV - LTA0n) * Value/100 | |
| | Applies to: [All Channels ▼] | |
| | [Edit] [Configure] | |
| | |
| | ✓ Drive/TPR On-Invoice % | |
| | Input Type: Percentage (0-50%) | |
| | Calculation: (PlannedGSV - LTA0n) * Value/100 | |
| | Applies to: [NKA, Traditional Trade ▼] | |
| | [Edit] [Configure] | |
| | |
| | ✓ WS TPR On-Invoice % | |
| | Input Type: Percentage (0-30%) | |
| | Applies to: [Wholesale Channel Only ▼] | |
| | [Edit] [Configure] | |
| | |
|-----|
| [+ Add On-Invoice Mechanic] |
| |
| OFF-INVOICE DISCOUNTS & REBATES |
|-----|
| | |
| | ✓ CPP Off-Invoice % | |
| | Input Type: Percentage (0-20%) | |
| | Calculation: PlannedNIV * Value/100 | |
| | Payment Terms: [Net 30 Days ▼] | |
| | [Edit] [Configure] | |
| | |
| | ✓ WS TPR Off-Invoice % | |

```

Input Type: Percentage (0-15%)	
Calculation: $\text{PlannedNIV} * \text{Value}/100$	
[Edit] [Configure]	
[+ Add Off-Invoice Mechanic]	
PER-UNIT SUPPORTS	
✓ Price Support per Unit	
Input Type: Currency (\$ per piece)	
Calculation: $\text{Value} * \text{PlannedVolume}$	
Max Allowed: [\$ 5.00] per unit	
[Edit] [Configure]	
[+ Add Per-Unit Support]	
LUMPSUM SPENDS	
✓ Visibility Lumpsum MT/PH	
Input Type: Currency (Fixed \$)	
Description: Modern Trade/Pharmacy displays	
Min: \$500 Max: \$50,000	
[Edit] [Configure]	
✓ Visibility Lumpsum GT	
Input Type: Currency (Fixed \$)	
Description: General Trade displays	
Min: \$100 Max: \$20,000	
[Edit] [Configure]	
✓ TPR/Drives Lumpsum	
Input Type: Currency (Fixed \$)	
Description: Drive/TPR fixed costs	
Min: \$1,000 Max: \$100,000	
[Edit] [Configure]	


```

| [+ Add Lumpsum Spend] |
| |
| LONG-TERM AGREEMENTS (LTA) |
| |
| | | |
| | ✓ LTA On-Invoice % | |
| | Source: [Master Data / CPL Agreements ▼] | |
| | Typical Range: 2-10% | |
| | [Edit] [Configure] | |
| | | |
| | ✓ LTA Off-Invoice % | |
| | Source: [Master Data / CPL Agreements ▼] | |
| | Typical Range: 1-5% | |
| | [Edit] [Configure] | |
| | | |
| |
| |
| |

```

2. Detailed Mechanic Configuration Modal

Click "Configure" on any mechanic:

```

| Configure Promo Mechanic: CPP On-Invoice % |
| |
| BASIC SETTINGS |
| |
| | Mechanic Name*: [CPP On-Invoice % ] | | | |
| | Display Name: [CPP On-Inv % ] | |
| | Mechanic Code: [CPP_ON_INV_PCT] (system use) | |
| | | |
| | Category: [On-Invoice Discount ▼] | |
| | Spending Type: [On-Invoice ▼] | |
| | | |
| | Description: | |
| | |
| | | Consumer Price Promotion - immediate discount | | |
| | | applied on invoice to reduce consumer price | | |

```

INPUT CONFIGURATION		
Input Type*	☒ Percentage ☐ Currency ☐ Units	
Input Constraints:		
• Minimum Value:	[____0____] %	
• Maximum Value:	[___100___] %	
• Default Value:	[____0____] %	
• Step Increment:	[___0.5__] % (for slider)	
Validation Rules:		
☒ Cannot exceed 100%		
☒ Must be non-negative		
☐ Requires approval if > [50]%		
☒ Show warning if > [25]% (high discount)		
Input Method:		
• Text input	☐ Slider	☐ Dropdown presets

CALCULATION FORMULA		
Spend Calculation*:		
	$(PLANNED_GSV - PLANNED_LTA_ON_INV) * VALUE/100$	
[Validate Formula]	[Test Calculation]	
Formula Explanation:		
• Take Planned Gross Sales Value		
• Subtract LTA On-Invoice amount		
• Apply the discount percentage		
• Result = Amount deducted from invoice		
Example Calculation:		
If Planned GSV = \$100,000		
LTA On-Invoice = \$5,000		
User enters: 10%		

| | = \$9,500 | |
| _____ |
| |
| APPLICABILITY RULES |
| _____ |
	Available for:	
	Channels:	
	☒ NKA (Modern Trade)	
	☒ Traditional Trade	
	☒ E-Commerce	
	☒ Export	
	Categories: [All Categories ▼]	
	◦ All Categories	
	• Specific Categories	
	☒ Dairy	
	☒ Beverages	
	☐ Snacks	
	☒ Frozen	
	Seasonal Restrictions:	
	☐ Only available during specific periods	
	Date Range: [____] to [____]	

BUDGET & CONTROLS		

	Budget Tracking:	
	☒ Track against On-Invoice budget	
	☒ Require budget availability check	
	☐ Reserve budget on plan creation	
	Approval Requirements:	
	☐ No approval needed	
	☒ CM approval required if total > \$[10,000]	
	☐ Finance approval required if % > [30]%	
	Combination Rules:	
	☒ Can combine with other on-invoice discounts	

| | [] Maximum combined discount: [____] % | |
| _____ |
| |
| DISPLAY IN PLANNING GRID |
| _____ |
	Show in Grid: [✓] Enabled	
	Column Header: [CPP On-Inv %]	
	Column Width: [120] px	
	Column Position: [After LTA columns]	
	Cell Format:	
	• Show unit symbol: [✓] % symbol	
	• Decimal places: [1]	
	• Placeholder text: "Enter discount %"	
	Help Text (tooltip):	
	"Enter promotional discount % to be applied on	
	customer invoice. Max 100%."	

HISTORICAL DATA & BENCHMARKS		

	[✓] Track historical usage	
	[✓] Show average value used (last 6 months)	
	[✓] Show best performing range	
	Benchmarks:	
	• Typical range: 5-15%	
	• Industry average: 10%	
	• Your company average: 12%	
	[View Usage Analytics]	

STATUS & AUDIT		

	Status: • Active ○ Draft ○ Archived	
	Created by: Admin User on Jan 10, 2025	
	Last Modified: Sarah Manager on Mar 15, 2025	

		[View Change History]	
	[Cancel]	[Save as Draft]	[Save & Activate]

3. Database Schema for Promo Mechanics

```

CREATE TABLE promo_mechanics (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
-- Basic Info
mechanic_code VARCHAR(50) UNIQUE NOT NULL,
mechanic_name VARCHAR(200) NOT NULL,
display_name VARCHAR(100),
category VARCHAR(100) NOT NULL, -- 'on_invoice_discount',
'off_invoice_discount', 'per_unit_support', 'lumpsum'
spending_type VARCHAR(50) NOT NULL, -- 'on_invoice', 'off_invoice'
description TEXT,
-- Input Configuration
input_type VARCHAR(50) NOT NULL, -- 'percentage', 'currency', 'units'
min_value DECIMAL(15,4),
max_value DECIMAL(15,4),
default_value DECIMAL(15,4),
step_increment DECIMAL(15,4),
-- Validation
cannot_exceed_max BOOLEAN DEFAULT true,
must_be_non_negative BOOLEAN DEFAULT true,
requires_approval_threshold DECIMAL(15,4),
warning_threshold DECIMAL(15,4),
warning_message TEXT,
-- Input Method
input_method VARCHAR(50) DEFAULT 'text_input', -- 'text_input',
'slider', 'dropdown'
-- Calculation
calculation_formula TEXT NOT NULL,
formula_explanation TEXT,
-- Applicability
applicable_channels JSONB, -- Array of channel names
applicable_categories JSONB, -- Array of category names or "ALL"

```

```

season_start_date DATE,
season_end_date DATE,
-- Budget & Controls
track_against_budget BOOLEAN DEFAULT true,
budget_type VARCHAR(50), -- 'on_invoice', 'off_invoice'
require_budget_check BOOLEAN DEFAULT true,
reserve_budget_on_creation BOOLEAN DEFAULT false,
-- Approval
cm_approval_threshold DECIMAL(15,4),
finance_approval_threshold DECIMAL(15,4),
-- Combination Rules
can_combine_with_others BOOLEAN DEFAULT true,
mutually_exclusive_with JSONB, -- Array of mechanic codes
max_combined_discount DECIMAL(15,4),
-- Display
show_in_grid BOOLEAN DEFAULT true,
column_header VARCHAR(100),
column_width INTEGER DEFAULT 120,
column_position INTEGER,
show_unit_symbol BOOLEAN DEFAULT true,
decimal_places INTEGER DEFAULT 1,
placeholder_text VARCHAR(200),
help_text TEXT,
-- Analytics
track_historical_usage BOOLEAN DEFAULT true,
show_benchmarks BOOLEAN DEFAULT true,
typical_range_min DECIMAL(15,4),
typical_range_max DECIMAL(15,4),
industry_average DECIMAL(15,4),
company_average DECIMAL(15,4),
-- Status
status VARCHAR(20) DEFAULT 'active',
-- Audit
created_by UUID REFERENCES users(id),
created_at TIMESTAMP DEFAULT NOW(),
updated_by UUID REFERENCES users(id),
updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_promo_mechanics_category ON promo_mechanics(category);

```

```
CREATE INDEX idx_promo_mechanics_spending_type ON
promo_mechanics(spending_type);
```

```
-- Link mechanics to actual plan values
CREATE TABLE plan_mechanic_values (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
plan_fu_id UUID REFERENCES plan_fus(id) ON DELETE CASCADE,
mechanic_id UUID REFERENCES promo_mechanics(id),
entered_value DECIMAL(15,4), -- User input
calculated_spend DECIMAL(15,4), -- Result of formula
currency_code VARCHAR(3) DEFAULT 'USD',
notes TEXT,
entered_by UUID REFERENCES users(id),
entered_at TIMESTAMP DEFAULT NOW(),
updated_at TIMESTAMP DEFAULT NOW()
);
```

```
CREATE INDEX idx_plan_mechanic_values_plan_fu ON
plan_mechanic_values(plan_fu_id);
CREATE INDEX idx_plan_mechanic_values_mechanic ON
plan_mechanic_values(mechanic_id);
```

```
-- Historical usage tracking
CREATE TABLE mechanic_usage_history (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
mechanic_id UUID REFERENCES promo_mechanics(id),
plan_id UUID REFERENCES plans(id),
value_entered DECIMAL(15,4),
spend_calculated DECIMAL(15,4),
plan_roi DECIMAL(10,2),
plan_rag_status VARCHAR(20),
used_at TIMESTAMP DEFAULT NOW()
);
```

4. Smart Input Assistance

When planner enters values, show intelligent helpers:

```
| CPP On-Invoice %: [____15____] % |
```


| [Remove CPP] [Remove WS TPR] [Cancel] |

6. Mechanic Template Library

Pre-configured mechanic templates for common scenarios:

| Quick Apply Mechanic Templates |

| |

| 🎯 Standard Promotion |

| • CPP On-Invoice: 10% |

| • Visibility MT/PH: \$2,000 |

| [Apply Template] |

| |

| 🔥 Aggressive Promotion |

| • CPP On-Invoice: 20% |

| • CPP Off-Invoice: 5% |

| • Price Support: \$0.50/unit |

| • Visibility MT/PH: \$5,000 |

| [Apply Template] |

| |

| 💰 Conservative Promotion |

| • CPP On-Invoice: 5% |

| • Visibility GT: \$500 |

| [Apply Template] |

| |

| 🏪 Wholesale Focus |

| • WS TPR On-Invoice: 12% |

| • WS TPR Off-Invoice: 3% |

| [Apply Template] |

| |

| [+ Create Custom Template] |

Save planner's frequently used combinations as personal templates.

Implement all validation rules and budget checks in real-time as planners enter mechanic values.

PLAN: Summer Dairy Promo - Carrefour Status: Draft ROI: 24.5% [GREEN]						
GRAND TOTALS PANEL (Sticky top section)						
Planned Volume	Planned Turnover	Total Spend	Incremental GP	GP ROI %	RAG Status	
125,000 pcs	\$487,500	\$95,300	\$23,750	24.9%	[GREEN] GREEN	
↑ 25K vs base	↑ \$97.5K vs base	↑ \$15.3K vs base	↑ \$8.5K vs base	Target: >20%	Excellent	
FILTERS & CONTROLS						
[+ Add FU] [Import from Template] [Copy from Previous Plan] View: [All Columns ▼] [Customize Columns] [Export] [Save] [Refresh]						

MAIN GRID (Hierarchical & Scrollable):

[illegible]

[+] FU: Activia Yogurt 4x100g [TACTIC INPUT CELLS] [CALCULATED VALUES]

		8,000	10,500	2,500	31.3%	32,000	42,000	10,000	5%	2%
[10%]		[5%]		[0.25]		[2000]	6,825	8,500	10,250	1,750
25.6%		[GREEN]								

└ SKU: Strawberry 400g

3,200	4,200	1,000	31.3%	12,800	16,800	4,000						
2,730	3,400	4,100	700	24.4%		[GREEN]						

└ SKU: Vanilla 400g

			2,400	3,150	750	31.3%		9,600		12,600		3,000					
2,048		2,550		3,075		525		24.4%		[GREEN]							

└ SKU: Blueberry 400g

			1,600	2,100	500	31.3%		6,400		8,400		2,000					
1,365		1,700		2,050		350		24.4%		[GREEN]							

└ SKU: Peach 400g

[illegible]

[+] FU: Greek Yogurt Plain 200g

	12,000	14,400	2,400	20.0%	60,000	72,000	12,000	5%	2%	
[8%]	[3%]	[0.15]	[1500]	8,950	15,500	17,820	12,320	38.8%	25.9%	[GREEN]

└ (SKUs collapsed)

[+] FU: Natural Set Yogurt Line

PROMPT: Comprehensive Planning Screen Implementation
Create the most critical screen of CollMind TPM - the Detailed Planning Screen with complete specifications:

SECTION 1: SCREEN LAYOUT & ARCHITECTURE

1. Overall Screen Structure

| TOP NAVIGATION BAR (Global - not part of planning screen) |

| PLAN HEADER SECTION (Fixed - Always visible) |
| • Plan Name, Status Badge, Dates, Action Buttons |
| • Height: 80px |

| GRAND TOTALS PANEL (Sticky - Stays visible on scroll) |
| • 6 Key Metric Cards with real-time values |
| • Height: 140px |

| TOOLBAR & FILTERS (Sticky - Stays visible on scroll) |
| • Add FU, Column Selector, View Options, Export, Save |
| • Height: 60px |

| |
| MAIN PLANNING GRID (Scrollable - Both horizontal & vertical) |
| • Hierarchical FU/SKU rows |
| • Dynamic columns based on context |
| • Editable & Calculated cells |
| • Height: Remaining viewport (flexible) |
| |

| COMMENTS & ACTIVITY PANEL (Collapsible right sidebar) |
| • Width: 320px (collapsed: 0px) |

SECTION 2: PLAN HEADER SECTION - DETAILED SPECIFICATION

| ← Back to Plans List PLAN DETAIL [💬][🔔][⚙️] |

| |
| 📅 Summer Dairy Promotion - Carrefour [✎ Rename] |
| CPL: Carrefour CPL | Category: Dairy | Jun 1 - Jun 30, 2025 |
| |
| Status: [●●● DRAFT] Last saved: 2 minutes ago | Owner: John Planner |
| |
| Actions: |
| [💾 Save Draft] [📤 Submit for Approval] [📄 Duplicate] [🗑 Delete] |
| [📥 Import FUs] [📊 View Analytics] [📝 Add Comment] [📅 History] |
| |

A. Back Navigation:

- Icon: Left arrow
- Text: "Back to Plans List"
- Action: Navigate to Plan List screen
- Confirm if unsaved changes exist

B. Plan Name (Editable):

- Display: Large bold text (24px font)
- Icon: Document icon
- Click behavior: Inline edit mode
- Max length: 100 characters
- Auto-save on blur

C. Plan Meta Information:

- CPL: Display CPL name (read-only, set at creation)
- Category: Display category name (read-only)
- Date Range: Display start and end dates
- All three displayed in single line, pipe-separated

D. Status Badge:

Visual design by status:

- DRAFT: Gray badge with pulsing dot
- PENDING APPROVAL: Yellow/Orange badge with clock icon
- APPROVED: Green badge with checkmark icon
- REJECTED: Red badge with X icon

Badge shows:

- Current status
- Last action timestamp
- User who performed action (on hover)

E. Auto-Save Indicator:

- "Last saved: X minutes ago" (updates every 30 seconds)
- Changes to "Saving..." when changes detected
- Shows "✓ Saved" briefly after successful save
- Shows "⚠ Save failed" if error (with retry button)

F. Action Buttons:

PRIMARY ACTIONS (Large buttons):

1. Save Draft:

- Keyboard: Ctrl/Cmd + S
 - Action: Save all changes without status change
 - Disabled if: No changes to save
2. Submit for Approval:
- Icon: 
 - Color: Blue (Primary)
 - Action: Opens confirmation modal → Changes status to "Pending Approval"
 - Disabled if:
 - * Plan has validation errors
 - * No FUs added
 - * Status is not "Draft"
 - Pre-submission validation checks:
 - * All required tactics entered
 - * No RED items (if configured)
 - * Budget availability
- SECONDARY ACTIONS (Icon buttons):
3. Duplicate Plan:
- Icon: 
 - Action: Creates copy with "[Copy]" appended to name
 - Copies: All FUs, SKUs, tactic values
 - Does not copy: Status, approvals, comments
4. Delete Plan:
- Icon: 
 - Color: Red (Danger)
 - Action: Opens confirmation modal → Soft delete
 - Disabled if: Status is "Approved"
 - Warning: "This action cannot be undone"
5. Import FUs:
- Icon: 
 - Action: Opens modal with options:
 - * Import from Excel template
 - * Copy from another plan
 - * Bulk add from FU library
6. View Analytics:
- Icon: 
 - Action: Opens analytics panel showing:
 - * Historical performance of similar plans
 - * Benchmark comparisons
 - * Optimization suggestions

- Icon: 📝
- Action: Opens comment input in right sidebar
- Shows unread comment count badge

8. View History:

- Icon: 📅
- Action: Opens modal showing:
 - * All changes to this plan
 - * Who made changes and when
 - * Ability to compare versions

G. Utility Icons (Top Right):

- 💬 Comments: Toggle comments sidebar (shows unread count)
- 🔔 Notifications: Plan-specific notifications
- ⚙️ Settings: Quick settings (column visibility, autosave frequency)

SECTION 3: GRAND TOTALS PANEL - DETAILED SPECIFICATION

| PLAN PERFORMANCE OVERVIEW |

VOLUME		TURNOVER		SPEND		PROFIT	
Planned Total	Planned Total	Total Planned	Incremental				
125,450 pcs	\$487,320	\$95,250	Gross Profit				
	\$23,680						
Incremental	Incremental	Incremental					
+25,450 pcs	+\$97,464	+\$15,250	Base GP				
↑ 25.4% vs base	↑ 20.0% vs base	vs base	\$31,450				
Base: 100,000	Base: \$389,856	Base: \$80,000	Planned GP				


```
|
|
┌───┐
| |
| |
|
|
┌───┐
| |
| | 📊 EFFICIENCY | 🎯 OVERALL STATUS | | | |
| | | |
| | GP ROI | ██████████ | |
| | 24.9% | GREEN | |
| | Target: >20% | EXCELLENT PERFORMANCE | |
| | ✓ On Target | | |
| | | Plan Status: 16 Green | 3 Amber | 1 Red | |
| | GP Margin | Budget Impact: 18.5% of monthly allocation | |
| | 11.3% | | |
| | (Planned) | [View Breakdown ▼] [Optimize Plan] [What-If] | |
| | | |
|
|
┌───┐
| |
| |
|
```

- Planned Total = $\text{SUM}(\text{plan_sku_volumes.planned_volume})$ for all SKUs in plan
- Base Total = $\text{SUM}(\text{plan_sku_volumes.base_volume})$ for all SKUs in plan
- Incremental = Planned Total - Base Total
- Uplift % = $(\text{Incremental} / \text{Base Total}) * 100$

Display Rules:

- Show ↑ green arrow if Incremental > 0
- Show ↓ red arrow if Incremental < 0
- Show → gray arrow if Incremental = 0
- Animate numbers on change (count-up effect)
- Update in real-time as user edits planned volumes

Hover Tooltip:

"Total units planned to be sold during promotion period.
Base volume is historical average without promotion."

CARD 2: TURNOVER METRICS

💰 TURNOVER	
Planned Total: \$487,320	<- Net sales after all discounts
Incremental: +\$97,464	<- Planned TO - Base TO
↑ 20.0% vs base	<- $iTO / \text{Base TO} * 100$
Base: \$389,856	<- Base sales without promo

Calculation Logic:

Base Turnover = Base GSV - Base Trade Spend

where Base GSV = $\text{SUM}(\text{SKU: base_volume} * \text{list_price})$

Base Trade Spend = $\text{SUM}(\text{BaseLTASpendOn} + \text{BaseLTASpendOff})$

Planned Turnover = Planned GSV - Planned Total Spend

where Planned GSV = $\text{SUM}(\text{SKU: planned_volume} * \text{list_price})$

Planned Total Spend = $\text{SUM}(\text{all on-invoice} + \text{off-invoice spends})$

Incremental TO = Planned Turnover - Base Turnover

Display Rules:

- Format as currency with thousand separators
- Color code uplift: Green if positive, Red if negative
- Show warning icon if Planned TO < Base TO

CARD 3: SPEND METRICS

🧩 SPEND	
Total Planned: \$95,250	<- All promotional costs
Incremental: +\$15,250	<- Additional spend vs base
vs base	
Base: \$80,000	<- Normal trade spend

On-Invoice: \$58,200 (61%)	<- Breakdown
Off-Invoice: \$37,050 (39%)	

Calculation Logic:

Total Planned Spend = PlannedPromoTotalSpendOn +
PlannedPromoTotalSpendOff

PlannedPromoTotalSpendOn =
PlannedPromoLTAOnInvoice +
SUM(all on-invoice mechanic spends: CPP_On, TPR_On, WSTPR_On)

PlannedPromoTotalSpendOff =
PlannedPromoLTAOffInvoice +
SUM(all off-invoice spends: CPP_Off, WSTPR_Off, PriceSupport,
VisibilityMTPH, VisibilityGT, TPRDriveLumpsum)

Base Trade Spend = BaseLTASpendOn + BaseLTASpendOff

Incremental Spend = Total Planned Spend - Base Trade Spend

Off-Invoice % = (PlannedPromoTotalSpendOff / Total Planned Spend) * 100

Display Rules:

- Show spend breakdown with percentages
- Color bars representing On vs Off proportion
- Click to expand detailed mechanic breakdown
- Show budget remaining indicator (if budget assigned)

Expanded View (on click):

SPEND BREAKDOWN	
ON-INVOICE: \$58,200	
• LTA On-Invoice: \$18,500	
• CPP On-Invoice: \$28,500	
• TPR On-Invoice: \$11,200	
OFF-INVOICE: \$37,050	
• LTA Off-Invoice: \$9,750	
• CPP Off-Invoice: \$12,183	
• Price Support: \$6,273	
• Visibility MT/PH: \$5,000	
• Visibility GT: \$1,500	
• TPR Lumpsum: \$2,344	
Budget Status: 18.5% of \$515K	
Remaining	

CARD 4: PROFIT METRICS

💎 PROFIT	
Incremental GP: \$23,680	<- Key metric (iGP)
Base GP: \$31,450	<- Without promo
Planned GP: \$55,130	<- With promo
iGP Margin: 24.3%	<- iGP/iTO*100

Calculation Logic:

Base GP = Base Turnover - Base COGS

where Base COGS = $\text{SUM}(\text{SKU: base_volume} * \text{cogs_per_piece})$

Planned GP = Planned Turnover - Planned COGS

where Planned COGS = $\text{SUM}(\text{SKU: planned_volume} * \text{cogs_per_piece})$

Incremental GP (iGP) = Planned GP - Base GP

iGP Margin % = $(\text{iGP} / \text{Incremental Turnover}) * 100$

Display Rules:

- Highlight iGP prominently (larger font)
- Show green if iGP > 0, red if iGP < 0
- Show warning if iGP < 0: "Promotion decreases profit"

CARD 5: EFFICIENCY METRICS

		EFFICIENCY	
	GP ROI: 24.9% <- Primary success metric		
	Target: >20%		
	✓ On Target		
	GP Margin: 11.3% <- Profitability		
	(Planned)		
	TO ROI: 63.9% <- Alternative ROI		

Calculation Logic:

GP ROI % = $(\text{Incremental GP} / \text{Incremental Spend}) * 100$

Planned GP Margin % = $(\text{Planned GP} / \text{Planned Turnover}) * 100$

TO ROI % = $(\text{Incremental TO} / \text{Incremental Spend}) * 100$

- GP ROI is THE most important metric
- Show comparison to target (from RAG configuration)
- Display status: ✓ On Target / △ Below Target / ★ Exceeds Target
- Animate value changes
- Show sparkline of how ROI changed during planning session

RAG Status Indicator:

- GREEN (>20%): Show green background, checkmark
- AMBER (10-20%): Show amber background, warning icon
- RED (<10%): Show red background, alert icon, block submission

CARD 6: OVERALL STATUS (Large Card)

	🎯 OVERALL STATUS	
	GREEN	
	EXCELLENT PERFORMANCE	
	Plan Composition:	
	16 Green items (80%)	
	3 Amber items (15%)	
	1 Red item (5%)	
	Budget Impact: \$95,250 = 18.5% of \$515K monthly	
	△ Items Needing Attention: 1	
	● SKU: Activia Strawberry 500g (ROI: 7.2% - RED)	
	[View Details]	
	[View Full Breakdown ▼] [Optimize] [What-If]	

Calculation Logic:

Overall RAG Status = Determined by Plan-level GP ROI %

- If GP ROI >= Green threshold (e.g., 20%): GREEN
- If GP ROI >= Amber threshold (e.g., 10%): AMBER
- If GP ROI < Red threshold (e.g., 10%): RED

Item Breakdown:

- Calculate percentages
- Display as horizontal bar chart with segments

Budget Impact:

- Calculate: $(\text{Total Planned Spend} / \text{Monthly Budget}) * 100$
- Show as percentage and progress bar
- Color code: Green (<80%), Amber (80-95%), Red (>95%)

Items Needing Attention:

- List all RED status items
- Show count prominently
- Click to filter grid to show only those items
- Suggest actions: "Consider reducing display fee" or "Increase volume target"

Action Buttons:

1. View Full Breakdown:

Opens modal with detailed RAG analysis by CPL, Category, Channel

2. Optimize Plan:

AI-powered optimization suggestions:

- "Increase planned volume for high-margin SKUs"
- "Reduce CPP % from 15% to 12% to improve ROI"
- "Remove Activia Strawberry 500g to achieve all-green status"

3. What-If Analysis:

Opens scenario simulator:

- Adjust tactics/volumes
- See instant KPI impact
- Compare to current plan

Real-Time Calculation Trigger Logic:

The Grand Totals Panel recalculates IMMEDIATELY when:

1. User enters/changes Planned Volume in any SKU row

→ Triggers: Volume totals, GSV, Turnover, COGS, GP, all dependent KPIs

2. User enters/changes any Tactic value (CPP%, Visibility \$, etc.)

→ Triggers: Spend calculations, Turnover, GP, ROI

3. User adds or removes FU/SKU from plan
→ Triggers: Full recalculation of all aggregates

4. Admin changes RAG thresholds (in background)
→ Triggers: RAG status re-evaluation for all items

Calculation Performance Optimization:

- Debounce rapid changes (300ms delay)
- Calculate incrementally (only affected rows)
- Use Web Workers for complex calculations
- Show loading spinner only if calculation > 500ms
- Cache intermediate results

Visual Feedback During Calculation:

```
|  TURNOVER |  
| |  
| Planned Total: [ Updating...] | <- Spinner shows briefly  
| |
```

After calculation:

- Briefly pulse/flash the updated values (green highlight)
- Show "✓ Updated" indicator for 2 seconds
- If error occurs, show "⚠ Calculation error" with retry button

SECTION 4: TOOLBAR & FILTERS - DETAILED SPECIFICATION

```
| [+ Add FU] [ Import] [ Templates] | View: [ Customize Columns ▼]  
|  
| |  
| Show: [✓All ▼] [ Search FUs...] RAG: [✓G][✓A][✓R] [] [] []  
[v] |
```


Component Specifications:

A. PRIMARY ACTIONS (Left Side):

1. "+ Add FU" Button:

Click opens modal:

Add Forecasting Units to Plan	
Search: Search by FU code or name...	
Filters:	
Category: [Dairy (Plan Category) ▼]	
Brand: [All Brands ▼]	
Status: [Active Only ▼]	
Available FUs: (234 results)	
☐	FU001 - Activia Yogurt 4x100g
Brand: Danone 4 SKUs	
Last used: 3 days ago (ROI: 25.2%)	
☐	FU002 - Greek Yogurt Natural 200g
Brand: Fage 2 SKUs	
Last used: 1 week ago (ROI: 18.5%)	
☐	FU003 - Activia Probiotic Range
Brand: Danone 6 SKUs	
Never used in promotions	
(Scrollable list...)	
✓ Also add Base Volume data	
✓ Auto-populate from last promotion	
Selected: 0 FUs	
[Select All] [Clear] [Cancel] [Add Selected]	

Features:

- Real-time search (fuzzy matching)
- Show FU metadata (# SKUs, brand, last performance)
- Multi-select with checkboxes
- Keyboard navigation (arrows + space/enter)
- "Last used" shows historical ROI for quick reference
- Option to pre-fill base volumes from historical data

2. "Import" Button:

Dropdown menu with options:

- Import from Excel
 - Opens file upload modal
 - Downloads template first-time
 - Validates data before import
- Copy from Another Plan
 - Opens plan selector
 - Select which FUs/tactics to copy
 - Preserves/replaces values
- Load from Template
 - Select saved promotion template
 - "Summer Promo Standard"
 - "Aggressive Discount Template"
 - Applies preset tactics/volumes

3. "Templates" Button:

Quick apply promotion templates:

Apply Promotion Template
Your Templates:
• Standard Dairy Promo (Used 12x)
• Aggressive Summer Drive (Used 3x)
• Conservative Launch (Used 7x)
Company Templates:
• NKA Standard Promotion
• Traditional Trade Bundle
Apply to:

Selected FUs only
Override existing values: []
Cancel Preview Apply

B. VIEW CONTROLS (Center):

4. "⚙ Customize Columns" Dropdown:

Customize Grid Columns																																				
View Presets: Compact View Standard View Detailed View My Custom View + Save Current																																				
VISIBLE COLUMNS (Drag to reorder) <table> <tr><td>☒</td><td>Base Volume</td><td>120px</td></tr> <tr><td>☒</td><td>Planned Volume</td><td>130px</td></tr> <tr><td>☒</td><td>Incremental Volume</td><td>140px</td></tr> <tr><td>☒</td><td>Volume Uplift %</td><td>120px</td></tr> <tr><td>☒</td><td>Base GSV</td><td>130px</td></tr> <tr><td>☒</td><td>Planned GSV</td><td>130px</td></tr> <tr><td>☒</td><td>CPP On-Invoice %</td><td>140px</td></tr> <tr><td>☒</td><td>CPP Off-Invoice %</td><td>140px</td></tr> <tr><td>☒</td><td>Total Planned Spend</td><td>150px</td></tr> <tr><td>☒</td><td>Incremental GP</td><td>140px</td></tr> <tr><td>☒</td><td>GP ROI %</td><td>120px</td></tr> <tr><td>☒</td><td>RAG Status</td><td>100px</td></tr> </table>	☒	Base Volume	120px	☒	Planned Volume	130px	☒	Incremental Volume	140px	☒	Volume Uplift %	120px	☒	Base GSV	130px	☒	Planned GSV	130px	☒	CPP On-Invoice %	140px	☒	CPP Off-Invoice %	140px	☒	Total Planned Spend	150px	☒	Incremental GP	140px	☒	GP ROI %	120px	☒	RAG Status	100px
☒	Base Volume	120px																																		
☒	Planned Volume	130px																																		
☒	Incremental Volume	140px																																		
☒	Volume Uplift %	120px																																		
☒	Base GSV	130px																																		
☒	Planned GSV	130px																																		
☒	CPP On-Invoice %	140px																																		
☒	CPP Off-Invoice %	140px																																		
☒	Total Planned Spend	150px																																		
☒	Incremental GP	140px																																		
☒	GP ROI %	120px																																		
☒	RAG Status	100px																																		
HIDDEN COLUMNS <table> <tr><td>☐</td><td>Incremental GSV</td><td></td></tr> </table>	☐	Incremental GSV																																		
☐	Incremental GSV																																			

☐	Planned NIV	
☐	Base Turnover	
☐	Planned Turnover	
☐	LTA On-Invoice %	
☐	LTA Off-Invoice %	
☐	Price Support per Unit	
☐	Visibility Lumpsum MT/PH	
☐	Visibility Lumpsum GT	
☐	TPR/Drive On-Invoice %	
☐	TPR/Drive Lumpsum	
☐	WS TPR On-Invoice %	
☐	WS TPR Off-Invoice %	
☐	Base COGS	
☐	Planned COGS	
☐	Base Gross Profit	
☐	Planned Gross Profit	
☐	Base Gross Margin %	
☐	Planned Gross Margin %	
☐	Incremental Gross Margin %	
☐	Planned TO ROI %	
☐		
☐	[Show All] [Hide All]	
Column Grouping:		
☒	Group by KPI category	
☒	Show group headers	
Reset to Default	Cancel	Apply

Features:

- Drag & drop to reorder columns
- Checkbox to show/hide each column
- Adjust column width (drag or input px value)
- Save custom views with names
- Apply preset views (Compact/Standard/Detailed)
- Group columns by category with collapsible headers

Filter which FUs/SKUs are visible:

• All Items (23 FUs) |

◦ Expanded Only (5 FUs) |

◦ Collapsed Only |

◦ Has Changes (8 FUs) |

◦ Errors/Warnings (1 FU) |

 Search FUs..." Input:

Live search as you type

Searches: FU code, FU name, SKU names

Highlights matching text in grid

Shows "X results found"

Clear button (X) appears when text entered

RAG Filter Buttons:

Three toggle buttons:

[✓ G] = Show Green items (checked by default)

[✓ A] = Show Amber items (checked by default)

[✓ R] = Show Red items (checked by default)

Click to toggle on/off

Instantly filters grid rows

Shows count: [✓ G 16] [✓ A 3] [✓ R 1]

C. UTILITY ACTIONS (Right Side):

 Analytics Icon:

Opens analytics side panel showing:

Performance trends

Comparison to benchmarks

Recommendations

 Export Icon:

Dropdown menu:

Export Options |

| • Export to Excel (Full Plan) |
| • Export Visible Rows Only |
| • Export Summary Report |
| • Export for Finance Review |
| • Export for Presentation |
| |
| Include: |
| [✓] Comments |
| [✓] Change History |
| [✓] RAG Status Formatting |
| |
| [Cancel] [Export] |
|_____|

[] Quick Save Icon:

One-click save (same as Ctrl+S)
Shows brief "✓ Saved" confirmation
Pulsing icon if unsaved changes exist

[] Refresh Icon:

Reload data from server
Useful if multiple users editing
Warns if local changes will be lost
Shows "Last updated: X seconds ago" on hover

SECTION 5: MAIN PLANNING GRID - COMPLETE SPECIFICATION

5.1 Grid Architecture & Column Structure

CRITICAL: Dynamic Column Logic

The columns displayed in the grid are determined by:

Plan Context (Highest Priority):

Plan's Category (e.g., "Dairy")

These are set when plan is created and cannot be changed

Tactic Applicability Rules:

Only show tactic columns where:

(tactic.applicable_channels CONTAINS plan.channel OR

tactic.applicable_channels = "ALL")

AND

(tactic.applicable_categories CONTAINS plan.category OR

tactic.applicable_categories = "ALL")

Example:

Plan: Channel = "NKA", Category = "Dairy"

Tactic: "CPP On-Invoice %"

applicable_channels = ["NKA", "Traditional Trade", "E-Commerce"]

applicable_categories = ["ALL"]

RESULT: ✓ SHOW (NKA is in applicable channels)

Tactic: "WS TPR On-Invoice %"

applicable_channels = ["Wholesale"]

applicable_categories = ["ALL"]

RESULT: ✗ HIDE (Wholesale not in applicable channels)

Tactic: "Display Fee MT/PH"

applicable_channels = ["NKA"]

applicable_categories = ["Dairy", "Beverages"]

RESULT: ✓ SHOW (Both NKA and Dairy match)

User Preferences:

User can hide/show any column via "Customize Columns"

Preferences saved per user

But cannot show tactics that don't apply to plan context

KPI Configuration:

KPI must have show_in_grid_fu = true (for FU rows)

KPI must have show_in_grid_sku = true (for SKU rows)

Status must be "active"

Column Order & Grouping

Columns are organized in logical groups:

KPI Group	
GROUP 1: ITEM INFO (Fixed Left - Always Visible)	
└─ [Expand/Collapse] Icon (20px)	
└─ Item Name (250px)	
└─ Item Code (100px)	
GROUP 2: MASTER DATA (Always Visible)	
└─ List Price per Piece (100px) [From Master Data]	
└─ COGS per Piece (100px) [From Master Data]	
GROUP 3: VOLUME KPIs (Scrollable)	
└─ Base Volume (110px) [From Historical Data - Read-only]	
└─ Planned Volume (120px) [USER INPUT - Editable at SKU level only]	
└─ Incremental Volume (120px) [Calculated: Planned - Base]	
└─ Volume Uplift % (110px) [Calculated: iVol/Base*100]	
GROUP 4: GSV KPIs (Scrollable)	
└─ Base GSV (120px) [Calculated: Base Vol * List Price]	
└─ Planned GSV (120px) [Calculated: Planned Vol * List Price]	
└─ Incremental GSV (120px) [Calculated: Planned GSV - Base GSV]	
GROUP 5: LTA (Long-Term Agreement) (Scrollable - Optional Display)	
└─ LTA On-Invoice % (100px) [From Master Data or Plan Override]	
└─ LTA Off-Invoice % (100px) [From Master Data or Plan Override]	
└─ LTA On-Invoice \$ (120px) [Calculated]	
└─ LTA Off-Invoice \$ (120px) [Calculated]	

| GROUP 6: PROMO MECHANICS - ON-INVOICE (Scrollable - DYNAMIC BASED ON PLAN CONTEXT) |

- | └ CPP On-Invoice % (130px) [USER INPUT - Editable at FU level only] |
- | └ Drive/TPR On-Invoice % (150px) [USER INPUT - if applicable] |
- | └ WS TPR On-Invoice % (150px) [USER INPUT - if applicable] |
- | |

| GROUP 7: PROMO MECHANICS - OFF-INVOICE (Scrollable - DYNAMIC BASED ON PLAN CONTEXT) |

- | └ CPP Off-Invoice % (130px) [USER INPUT - Editable at FU level only] |
- | |
- | └ WS TPR Off-Invoice % (150px) [USER INPUT - if applicable] |
- | |

| GROUP 8: PROMO MECHANICS - PER UNIT SUPPORT (Scrollable - DYNAMIC) |

- | └ Price Support per Unit \$ (150px) [USER INPUT - Editable at FU level only] |
- | |

| GROUP 9: PROMO MECHANICS - LUMP SUM (Scrollable - DYNAMIC) |

- | └ Visibility Lumpsum MT/PH \$ (180px) [USER INPUT - Editable at FU level only] |
- | └ Visibility Lumpsum GT \$ (180px) [USER INPUT - Editable at FU level only] |
- | └ TPR/Drive Lumpsum \$ (150px) [USER INPUT - Editable at FU level only] |
- | |

| GROUP 10: SPEND CALCULATIONS (Scrollable) |

- | └ Total On-Invoice Spend \$ (150px) [Calculated: Sum of all on-invoice] |
- | └ Total Off-Invoice Spend \$ (150px) [Calculated: Sum of all off-invoice] |
- | └ Total Planned Spend \$ (150px) [Calculated: On + Off] |
- | └ Incremental Spend \$ (150px) [Calculated: Total - Base Trade Spend] |
- | |

| GROUP 11: TURNOVER & NIV (Scrollable - Optional Display) |

- | └ Base NIV (120px) [Calculated] |
- | └ Planned NIV (120px) [Calculated] |
- | └ Base Turnover (130px) [Calculated] |
- | └ Planned Turnover (130px) [Calculated] |
- | └ Incremental Turnover (140px) [Calculated] |
- | └ TO Uplift % (110px) [Calculated] |

GROUP 12: PROFIT KPIs (Scrollable)
└ Base COGS (120px) [Calculated: Base Vol * COGS per piece]
└ Planned COGS (120px) [Calculated: Planned Vol * COGS per piece]
└ Base GP (130px) [Calculated: Base TO - Base COGS]
└ Planned GP (130px) [Calculated: Planned TO - Planned COGS]
└ Incremental GP (140px) [Calculated: Planned GP - Base GP]
GROUP 13: MARGIN KPIs (Scrollable)
└ Base GM % (110px) [Calculated: Base GP / Base TO * 100]
└ Planned GM % (110px) [Calculated: Planned GP / Planned TO * 100]
└ Incremental GM % (130px) [Calculated: iGP / iT0 * 100]
GROUP 14: ROI & RAG (Fixed Right - Always Visible)
└ GP ROI % (120px) [Calculated: iGP / Incr Spend * 100] [RAG ENABLED]
└ TO ROI % (120px) [Calculated: iT0 / Incr Spend * 100]
└ RAG Status (110px) [Determined by GP ROI % and thresholds]
GROUP 15: ACTIONS (Fixed Right - Always Visible)
└ Row Actions (80px) [...] Dropdown menu

Example: Plan for NKA Channel, Dairy Category
Visible Tactic Columns (automatically filtered):

- ✓ CPP On-Invoice % (applies to NKA)
- ✓ CPP Off-Invoice % (applies to NKA)
- ✓ Visibility Lumpsum MT/PH (applies to NKA)
- ✗ WS TPR On-Invoice % (only for Wholesale channel - HIDDEN)
- ✗ WS TPR Off-Invoice % (only for Wholesale channel - HIDDEN)
- ✓ Price Support per Unit (applies to Dairy)
- ✓ Visibility Lumpsum GT (applies to all)
- ✗ Export-Specific Tactic (only for Export channel - HIDDEN)

5.2 Row Structure & Hierarchy

FU (Forecasting Unit) Row - Parent Row

Visual Design:

--

[+]	FU001 - Activia Yogurt 4x100g	\$4.00	\$1.80	8,000	[10,500]	2,500	31.3%	\$32K	\$42K	5%	2%	[10%]	[5%]	[0.25]
					[2000]	...	25.6%	[●]	[...]					
Brand: Danone • 4 SKUs														

Characteristics:

Background: Light blue (#F0F9FF)

Font: Bold, 14px

Expandable/Collapsible: [+] icon (toggles to [-] when expanded)

Shows aggregated values (sum of child SKUs)

Tactic input cells are EDITABLE here (FU level)

Volume cells show aggregated totals (read-only, calculated from SKUs)

SKU (Product) Row - Child Row

Visual Design:

↳	SKU001 - Strawberry 400g	\$4.00	\$1.80	3,200	[4,200]	1,000	31.3%	\$12.8K	\$16.8K	-	-	-	-	-	...	25.6%	[●]	[...]
---	--------------------------	--------	--------	-------	---------	-------	-------	---------	---------	---	---	---	---	---	-----	-------	-----	-------

Characteristics:

Background: White

Font: Regular, 13px

Indented: Visual indent (↳ symbol or left padding)

Planned Volume cell is EDITABLE here (SKU level)

Tactic cells are READ-ONLY (inherited/distributed from parent FU)

Shows individual SKU calculated values

Data Entry Rules - CRITICAL

RULE 1: Volume Input Location

—|

- | • Planned Volume is entered at SKU LEVEL only |
- | • FU level shows SUM of child SKU planned volumes (read-only) |
- | • User CANNOT edit planned volume at FU level |
- | • If user clicks FU volume cell, show tooltip: |
- | "Enter planned volumes at SKU level below" |

—|

—|

| RULE 2: Tactic Input Location |

—|

- | • ALL Tactic values are entered at FU LEVEL only |
- | • This includes: |
- | - All on-invoice % discounts (CPP, TPR, WS TPR) |
- | - All off-invoice % discounts |
- | - Price support per unit (\$) |
- | - All lumpsum amounts (\$) |
- | |
- | • SKU level shows distributed/calculated values (read-only) |
- | • User CANNOT edit tactic values at SKU level |
- | • If user clicks SKU tactic cell, show tooltip: |
- | "Tactic values are set at FU level. Expand to view parent FU." |

—|

—|

| RULE 3: Tactic Value Distribution to SKUs |

—|

| When user enters tactic value at FU level: |

| |

| FOR PERCENTAGE-BASED TACTICS (CPP %, TPR %, etc.): |

- | • Same percentage applies to ALL child SKUs |
- | • No distribution calculation needed |
- | • SKU cells show same % as parent FU (for reference) |

| |

| Example: |

| ↳ SKU1: CPP On-Invoice = 10% (inherited) |
 | ↳ SKU2: CPP On-Invoice = 10% (inherited) |
 | |
 | Spend Calculation at SKU: |
 | SKU CPP On-Invoice Spend = (Planned GSV - LTA On) * 10% / 100 |
 | |
 | FOR PER-UNIT TACTICS (Price Support \$/unit): |
 | • Same \$ per unit applies to ALL child SKUs |
 | • Total spend = \$/unit * SKU planned volume |
 | |
 | Example: |
 | FU: Price Support = \$0.25 per unit |
 | ↳ SKU1 (4,200 units): Spend = \$0.25 * 4,200 = \$1,050 |
 | ↳ SKU2 (3,150 units): Spend = \$0.25 * 3,150 = \$787.50 |
 | |
 | FOR LUMP SUM TACTICS (Visibility \$, TPR Drive Lumpsum): |
 | • Total lumpsum amount is distributed proportionally to SKUs |
 | • Distribution basis: Base Volume ratio |
 | |
 | Example: |
 | FU: Visibility MT/PH = \$2,000 total |
 | FU Total Base Volume = 8,000 units |
 | |
 | ↳ SKU1: Base Vol = 3,200 (40% of FU) |
 | SKU1 Visibility Spend = \$2,000 * 40% = \$800 |
 | |
 | ↳ SKU2: Base Vol = 2,400 (30% of FU) |
 | SKU2 Visibility Spend = \$2,000 * 30% = \$600 |
 | |
 | ↳ SKU3: Base Vol = 1,600 (20% of FU) |
 | SKU3 Visibility Spend = \$2,000 * 20% = \$400 |
 | |
 | ↳ SKU4: Base Vol = 800 (10% of FU) |
 | SKU4 Visibility Spend = \$2,000 * 10% = \$200 |
 | |
 | Distribution Formula: |
 | SKU_Share = (SKU_Base_Volume / FU_Total_Base_Volume) |
 | SKU_Lumpsum_Spend = FU_Lumpsum_Total * SKU_Share |

└─┘

CELL TYPE 1: Read-Only Display Cell

Used for: Calculated KPIs, Master Data, Inherited Values

Visual Design:

```
| $32,500 | <- Gray background (#F9FAFB)
| | Dark text (#374151)
| | No border or subtle border
```

Cursor: default (not editable)

Hover State:

```
| $32,500 | <- Slightly darker background
| ⓘ | Shows info icon
```

Tooltip on Hover:

```
| Base GSV |
| Calculated: 8,000 pcs × $4.00 |
| = $32,000 |
| |
| Formula: Base Volume × List Price |
```

CELL TYPE 2: Editable Input Cell (Numeric)

Used for: Planned Volume (at SKU), All Tactics (at FU)

Visual Design (Not focused):

```
| 10,500 | <- White background
| | Border: 1px solid #D1D5DB
| | Cursor: text (editable)
```

Visual Design (Focused):

```
| |10,500 | <- Blue border (#3B82F6, 2px)
| | Text cursor visible
| | Shows formatting helpers
```

Visual Design (Has Value + Valid):

```
| 10,500 ✓ | <- Green checkmark
| | Background: very light green (#F0FDF4)
```

Visual Design (Has Error):

```
| |
```

| | Background: very light red (#FEF2F2)

| |

Hover State (Before Edit):

| |

| 10,500 | <- Show light blue background

|  | Pencil icon appears

| |

Interaction Behavior:

Single click: Enter edit mode, select all text

Double click: Enter edit mode, cursor at click position

Tab key: Move to next editable cell

Shift+Tab: Move to previous editable cell

Enter key: Save and move down to next row same column

Esc key: Cancel edit, revert to previous value

Arrow keys: Navigate between cells (if not editing)

Input Validation (Real-time):

For Volume: Must be positive integer or decimal

For Percentage: 0-100, max 2 decimals

For Currency: Positive, max 2 decimals

Show error message below cell if invalid

Prevent saving until valid

Example Error Display:

| |

| -500 ▲ |

| ▲ Volume must be positive

| |

Smart Features:

Auto-format on blur (add thousand separators)

Copy/paste support (including from Excel)

Bulk edit: Select multiple cells, enter value, applies to all

Quick fill: Drag corner to copy value down/across

CELL TYPE 3: Calculated Cell with RAG Status

Used for: GP ROI %, other key metrics with thresholds

Visual Design:

|  25.6% | <- Background color based on RAG
| | GREEN: #D1FAE5 (light green)
| | AMBER: #FEF3C7 (light yellow)

RED: #FEE2E2 (light red)

GRAY: #F3F4F6 (no data)

Large Indicator Dot:

 = Green (performance excellent)

 = Amber (performance acceptable)

 = Red (performance poor)

 = Gray (no data / not calculated)

Hover Tooltip:

| GP ROI: 25.6% |
| Status:  GREEN - Excellent |
| |
| Calculation: |
| Incremental GP: \$2,750 |
| Incremental Spend: \$10,725 |
| ROI = $(\$2,750 / \$10,725) \times 100$ |
| = 25.6% |
| |
| Thresholds: |
| • Green: $\geq 20\%$ ← You are here |
| • Amber: 10-20% |
| • Red: $< 10\%$ |
| |
| [View Details] |

Click Behavior:

Opens RAG drill-down modal (specified earlier)

CELL TYPE 4: RAG Status Badge Cell

Used for: Final RAG Status column

Visual Design:

|  GREEN | <- Full-width colored badge
| Excellent | Bold status name + description

Color Scheme:

GREEN: Background #10B981, Text white, Icon ✓

RED: Background #EF4444, Text white, Icon ▲
GRAY: Background #9CA3AF, Text white, Icon ○

Hover: Shows same tooltip as GP ROI % cell

CELL TYPE 5: Row Actions Cell

Visual Design:

 Three dots (vertical kebab menu)

Click opens dropdown menu:

 View FU Details |
 Duplicate FU |
 Copy FU Code |

 Add Comment |
 View History |

 Remove from Plan |

For SKU rows:

 View SKU Details |
 Copy SKU Code |

 This SKU cannot be removed |
(Remove parent FU instead)|

5.4 Real-Time Calculation Engine - Complete Logic

Calculation Trigger Events:

EVENT 1: User enters Planned Volume at SKU level

Trigger sequence:

Validate input (positive number)

Update SKU record: `plan_sku_volumes.planned_volume = new_value`

Calculate SKU-level K

SContinuePIs affected by volume change

4. Aggregate to parent FU level (sum all child SKU volumes)

5. Recalculate ALL dependent KPIs for that SKU

6. Recalculate ALL FU-level aggregates

8. Re-evaluate RAG status for SKU, FU, and Plan
9. Update UI (animate changed values)
10. Trigger auto-save (debounced 2 seconds)

Detailed Calculation Flow:

typescript// User enters: SKU Planned Volume = 4,200 units

Step 1: SKU-Level Volume Calculations

Input: planned_volume = 4,200

Base: base_volume = 3,200 (from historical data)

List Price: BPTT = \$4.00 (from master data)

COGS: cogs_per_piece = \$1.80 (from master data)

Calculate:

- Incremental Volume (iVol)

= planned_volume - base_volume

= 4,200 - 3,200

= 1,000 units

- Volume Uplift %

= (iVol / base_volume) * 100

= (1,000 / 3,200) * 100

= 31.25%

- Base GSV

= base_volume * BPTT

= 3,200 * \$4.00

= \$12,800

- Planned GSV

= planned_volume * BPTT

= 4,200 * \$4.00

= \$16,800

- Incremental GSV (iGSV)

= planned_GSV - base_GSV

= \$16,800 - \$12,800

= \$4,000

Step 2: SKU-Level LTA Calculations

Get from FU level or Master Data:

LTA_On_Pct = 5% (inherited from FU)

LTA_Off_Pct = 2% (inherited from FU)

Calculate:

- Base LTA Spend On-Invoice

= $\text{base_GSV} * (\text{LTA_On_Pct} / 100)$

= $\$12,800 * 0.05$

= \$640

- Planned LTA Spend On-Invoice

= $\text{planned_GSV} * (\text{LTA_On_Pct} / 100)$

= $\$16,800 * 0.05$

= \$840

- Base NIV (Net Invoice Value)

= $\text{base_GSV} - \text{base_LTA_spend_on}$

= $\$12,800 - \640

= \$12,160

- Planned NIV

= $\text{planned_GSV} - \text{planned_LTA_spend_on}$

= $\$16,800 - \840

= \$15,960

- Base LTA Spend Off-Invoice

= $\text{base_NIV} * (\text{LTA_Off_Pct} / 100)$

= $\$12,160 * 0.02$

= \$243.20

- Planned LTA Spend Off-Invoice

= $\text{planned_NIV} * (\text{LTA_Off_Pct} / 100)$

= $\$15,960 * 0.02$

= \$319.20

- Total Base Trade Spend

= $\text{base_LTA_on} + \text{base_LTA_off}$

= $\$640 + \243.20

= \$883.20

Get from Parent FU (entered by user at FU level):

CPP_On_Pct = 10%

CPP_Off_Pct = 5%

Price_Support_Per_Unit = \$0.25

Visibility_MT_PH_Total = \$2,000 (lumpsum at FU)

ON-INVOICE PROMO SPENDS:

- CPP On-Invoice Spend

= (planned_GSV - planned_LTA_on) * (CPP_On_Pct / 100)

= (\$16,800 - \$840) * 0.10

= \$15,960 * 0.10

= \$1,596

OFF-INVOICE PROMO SPENDS:

- CPP Off-Invoice Spend

= planned_NIV * (CPP_Off_Pct / 100)

= \$15,960 * 0.05

= \$798

- Price Support Spend

= price_support_per_unit * planned_volume

= \$0.25 * 4,200

= \$1,050

- Visibility MT/PH Spend (Distributed from FU)

Get FU totals:

FU_Total_Base_Volume = 8,000 units

SKU_Base_Volume = 3,200 units

SKU_Share = 3,200 / 8,000 = 0.40 (40%)

SKU_Visibility_Spend = \$2,000 * 0.40 = \$800

TOTAL PROMO SPENDS:

- Total On-Invoice Spend

= planned_LTA_on + CPP_on

= \$840 + \$1,596

= \$2,436

- Total Off-Invoice Spend

= planned_LTA_off + CPP_off + price_support + visibility

= \$2,967.20

- Total Planned Spend

= total_on + total_off

= \$2,436 + \$2,967.20

= \$5,403.20

- Incremental Spend

= total_planned_spend - total_base_trade_spend

= \$5,403.20 - \$883.20

= \$4,520

Step 4: SKU-Level Turnover Calculations

- Base Turnover

= base_GSV - total_base_trade_spend

= \$12,800 - \$883.20

= \$11,916.80

- Planned Turnover

= planned_GSV - total_planned_spend

= \$16,800 - \$5,403.20

= \$11,396.80

- Incremental Turnover (iT0)

= planned_turnover - base_turnover

= \$11,396.80 - \$11,916.80

= -\$520.00 ⚠️ NEGATIVE!

- T0 Uplift %

= (iT0 / base_turnover) * 100

= (-\$520 / \$11,916.80) * 100

= -4.36% ⚠️ NEGATIVE!

Step 5: SKU-Level Profit Calculations

- Base COGS

= base_volume * cogs_per_piece

= 3,200 * \$1.80

= \$5,760

- Planned COGS
 - = $\text{planned_volume} * \text{cogs_per_piece}$
 - = $4,200 * \$1.80$
 - = \$7,560
- Base Gross Profit
 - = $\text{base_turnover} - \text{base_COGS}$
 - = $\$11,916.80 - \$5,760$
 - = \$6,156.80
- Planned Gross Profit
 - = $\text{planned_turnover} - \text{planned_COGS}$
 - = $\$11,396.80 - \$7,560$
 - = \$3,836.80
- Incremental Gross Profit (iGP)
 - = $\text{planned_GP} - \text{base_GP}$
 - = $\$3,836.80 - \$6,156.80$
 - = $-\$2,320$ ⚠️ NEGATIVE! (Losing money!)

Step 6: SKU-Level Margin Calculations

- Base Gross Margin %
 - = $(\text{base_GP} / \text{base_turnover}) * 100$
 - = $(\$6,156.80 / \$11,916.80) * 100$
 - = 51.67%
- Planned Gross Margin %
 - = $(\text{planned_GP} / \text{planned_turnover}) * 100$
 - = $(\$3,836.80 / \$11,396.80) * 100$
 - = 33.67%
- Incremental Gross Margin %
 - = $(\text{iGP} / \text{iT0}) * 100$
 - = $(-\$2,320 / -\$520) * 100$
 - = 446.15% (Meaningless when both negative)

Step 7: SKU-Level ROI Calculations

- GP ROI %

= $(-\$2,320 / \$4,520) * 100$
= -51.33% ⚠️ HIGHLY NEGATIVE!

- TO ROI %
= $(iT0 / incremental_spend) * 100$
= $(-\$520 / \$4,520) * 100$
= -11.50% ⚠️ NEGATIVE!

Step 8: RAG Status Evaluation

Get RAG thresholds from configuration:

Green: GP ROI $\geq 20\%$

Amber: GP ROI $\geq 10\%$ and $< 20\%$

Red: GP ROI $< 10\%$

Current GP ROI = -51.33%

RAG Status = ● RED
Label = "Poor Performance - Review Plan"
Alert = true (if configured)

Step 9: Aggregate to FU Level

Sum all child SKU values:

This SKU (Strawberry 400g):

- Planned Volume: 4,200
- Planned GSV: \$16,800
- Total Spend: \$5,403.20
- Incremental GP: -\$2,320
- etc.

Other SKUs under same FU:

- Vanilla 400g: Planned Vol 3,150, iGP \$1,500, etc.
- Blueberry 400g: Planned Vol 2,100, iGP \$800, etc.
- Peach 400g: Planned Vol 1,050, iGP \$400, etc.

FU Totals:

- Total Planned Volume = $4,200 + 3,150 + 2,100 + 1,050 = 10,500$
- Total Planned GSV = $\$16,800 + \$12,600 + \$8,400 + \$4,200 = \$42,000$

- FU GP ROI = $(\$380 / \$18,000) * 100 = 2.11\%$ → ● RED

Update FU row with aggregated values

Step 10: Aggregate to Plan Level (Grand Totals)

Sum all FU totals for entire plan:

Update Grand Totals Panel:

- Total Planned Volume = SUM(all FU planned volumes)
- Total Planned Turnover = SUM(all FU planned turnovers)
- Total Planned Spend = SUM(all FU total spends)
- Total Incremental GP = SUM(all FU incremental GPs)
- Overall GP ROI % = $(\text{Total iGP} / \text{Total Incr Spend}) * 100$
- Overall RAG Status = Based on Overall GP ROI %

Step 11: UI Update with Animations

1. Highlight changed cells with brief green flash
2. Count-up animation for numeric values (300ms)
3. Update RAG badge with color transition
4. Update Grand Totals Panel (pulse effect)
5. Show toast notification if RAG status changed:
"⚠ SKU changed to RED status due to low ROI"

Step 12: Validation & Warnings

Check business rules:

IF SKU GP ROI < 0:

Show inline warning on row:

"⚠ This SKU is losing money. Consider removing or adjusting tactics."

IF FU has any RED SKUs:

Show warning badge on FU row:

"⚠ 1 item needs attention"

IF overall plan ROI < threshold:

Show warning in header:

"⚠ Plan ROI is below target. Consider optimization."

IF submission is attempted with RED items:

Block submission (if configured):

"Cannot submit: Plan contains 3 RED items that must be resolved."

Step 13: Auto-Save

Debounced after 2 seconds of no activity:

- Save all changed values to database
- Update last_saved timestamp
- Show "✓ Saved" indicator briefly

EVENT 2: User enters Tactic Value at FU level

Example: User enters CPP On-Invoice % = 15%

Trigger sequence:

typescriptStep 1: Validate & Store FU-level Tactic

Input: CPP_On_Invoice_Pct = 15%

Validation:

- Check: $0 \leq \text{value} \leq 100$ ✓
- Check: Not exceeds max allowed (e.g., 30%) ✓
- Check: Budget availability ✓

Store in: plan_mechanic_values table

plan_fu_id = FU_123

mechanic_id = MECHANIC_CPP_ON

entered_value = 15.00

Step 2: Apply to All Child SKUs

For each SKU under this FU:

Inherit CPP_On_Invoice_Pct = 15%

Recalculate for each SKU:

- CPP On-Invoice Spend

$= (\text{planned_GSV} - \text{planned_LTA_on}) * 0.15$

SKU1 (Strawberry):

$= (\$16,800 - \$840) * 0.15 = \$2,394$

(was \$1,596 with 10%, now \$2,394 with 15%)

Change: +\$798

SKU2 (Vanilla):

$= (\$12,600 - \$630) * 0.15 = \$1,795.50$

Change: +\$598.50

Step 3: Recalculate All Dependent KPIs

For each SKU:

- ↓ Total On-Invoice Spend increases
- ↓ Total Planned Spend increases
- ↓ Incremental Spend increases
- ↓ Planned Turnover decreases (more discount)
- ↓ Planned GP decreases
- ↓ Incremental GP decreases
- ↓ GP ROI % decreases

Example for SKU1:

Old GP ROI: 25.6%

New GP ROI: 18.3% (decreased due to higher discount)

RAG Status: Changed from  GREEN to  AMBER

Step 4: Aggregate & Update UI

- Recalculate FU totals
- Recalculate Plan totals
- Update all affected cells with animation
- Show impact notification:

"Increasing CPP to 15% decreased plan ROI from 24.9% to 22.1%"

Step 5: Show Optimization Suggestion (Optional)

AI-powered suggestion appears:

💡 Optimization Suggestion |

Increasing CPP discount to 15% reduces your |
overall ROI. Consider: |
|
• Keep CPP at 12% (optimal for this FU) |
• Or increase planned volume to 11,000 units |
to maintain ROI with 15% discount |
|
[Apply Suggestion] [Dismiss] |

Calculation Performance Optimization

```
const calculationWorker = new Worker('kpi-calculator.worker.js');
```

```
// Debounce rapid changes
const debouncedCalculate = debounce((changes) => {
  calculationWorker.postMessage({
    type: 'CALCULATE_KPIS',
    changes: changes,
    planData: currentPlanData
  });
}, 300);
```

```
// Incremental calculation (only affected rows)
function optimizedCalculation(changedCell) {
  if (changedCell.type === 'SKU_VOLUME') {
    // Only recalculate this SKU + parent FU + plan total
    const affectedRows = [
      changedCell.sku_id,
      changedCell.parent_fu_id,
      'PLAN_TOTAL'
    ];
    calculateSubset(affectedRows);
  } else if (changedCell.type === 'FU_TACTIC') {
    // Recalculate all child SKUs + FU + plan total
    const affectedRows = [
      changedCell.fu_id,
      ...getChildSkuIds(changedCell.fu_id),
      'PLAN_TOTAL'
    ];
    calculateSubset(affectedRows);
  }
}
```

```
// Cache intermediate results
const calculationCache = new Map();
```

```
function getCacheOrCalculate(key, calculationFn) {
  if (calculationCache.has(key)) {
    return calculationCache.get(key);
  }
  const result = calculationFn();
```

```

return result;
}

// Invalidate cache on relevant changes
function onDataChange(changedRowId) {
// Clear cache for this row and dependents
calculationCache.delete(changedRowId);
getDependentRows(changedRowId).forEach(id => {
calculationCache.delete(id);
});
}
...

```

5.5 Grid Interaction Features

Feature 1: Bulk Edit

Select multiple cells and edit at once:

...

User Action:

1. Click on first SKU Planned Volume cell
2. Hold Shift + Click on last SKU Planned Volume cell
3. 5 cells selected (highlighted in blue)
4. Type "5000" and press Enter
5. All 5 SKUs updated to 5,000 planned volume

Visual Feedback:

```

┌──────────┐
| [5,000] | ← Multiple cells selected (blue border)
├──────────┤
| [5,000] |
├──────────┤
| [5,000] |
├──────────┤
| [5,000] |
├──────────┤
| [5,000] |
└──────────┘

```

Confirmation Modal:

```
| ▲ Bulk Update Confirmation |
|-----|
| You are about to update 5 SKUs: |
| |
| • SKU001 - Strawberry: 4,200 → 5,000 |
| • SKU002 - Vanilla: 3,150 → 5,000 |
| • SKU003 - Blueberry: 2,100 → 5,000 |
| • SKU004 - Peach: 1,050 → 5,000 |
| • SKU005 - Mixed Berry: 800 → 5,000 |
| |
| This will impact plan ROI and totals. |
| |
| [Cancel] [Confirm Update] |
|-----|
...

```

Feature 2: Copy/Paste from Excel

User can copy data from Excel and paste directly:
...

Excel Data:

4200
3150
2100
1050

User Action:

1. Copy cells in Excel (Ctrl+C)
2. Select first target cell in grid
3. Paste (Ctrl+V)
4. System intelligently maps columns

Paste Options Modal:

```
|-----|
| Paste Data |
|-----|
| Detected: 4 rows, 1 column |
| |
| Paste to: Planned Volume column |
| Starting at: SKU001 - Strawberry |

```

```
| Preview: |  
| SKU001: 3,200 → 4,200 |  
| SKU002: 2,400 → 3,150 |  
| SKU003: 1,600 → 2,100 |  
| SKU004: 800 → 1,050 |  
| |  
| [Cancel] [Paste] |
```

```
```
```

Excel-style drag corner to fill:  
```

↓ Drag down

Options (show on release):

```
### Feature 4: Keyboard Shortcuts
```
```

- Tab: Next editable cell (right)
- Shift+Tab: Previous editable cell (left)
- Enter: Move down same column
- Shift+Enter: Move up same column
- Home: First column
- End: Last column
- Ctrl+Home: Top-left cell
- Ctrl+End: Bottom-right cell
- Page Up/Down: Scroll page

#### Editing:

- F2: Edit mode (current cell)
- Esc: Cancel edit
- Delete: Clear cell value
- Ctrl+C: Copy
- Ctrl+V: Paste
- Ctrl+X: Cut
- Ctrl+Z: Undo
- Ctrl+Y: Redo

#### Selection:

- Shift + Arrow: Extend selection
- Ctrl+A: Select all
- Ctrl + Click: Multi-select (non-contiguous)

#### Actions:

- Ctrl+S: Save plan
- Ctrl+F: Search/Find
- Ctrl+D: Duplicate selected FU
- Ctrl+E: Export

Display shortcut cheatsheet: Press "?" key  
 \ \ \

### ### Feature 5: Inline Warnings & Suggestions

Show contextual help as user edits:  
 \ \ \

---

| CPP On-Invoice %: [\_\_\_\_25\_\_\_\_] % |

```
| ⚠ Warning: High Discount Alert |
| A 25% discount is significantly higher than: |
| • Your typical: 12% |
| • Category average: 15% |
| • Optimal range: 8-13% (best ROI performance) |
| |
| 💡 Suggestion: Try 12% for better ROI |
| [Apply Suggestion] [Keep 25%] [Learn More] |
```

```
| Planned Volume: [____500____] pcs |
| |
| ⚠ Warning: Volume Below Base |
| Planned (500) is less than Base (3,200). |
| This means you expect to sell LESS with promotion! |
| |
| Did you mean: 5,000 units? |
| [Use 5,000] [Keep 500] [Dismiss] |
```

...

### ### Feature 6: Row Expand/Collapse Management

...

Expand/Collapse Controls:

Individual Row:

[+] Click to expand FU, show child SKUs

[-] Click to collapse FU, hide child SKUs

Bulk Controls (in toolbar):

[Expand All] - Expand all FUs in plan

[Collapse All] - Collapse all FUs in plan

[Expand Filtered] - Expand only visible/filtered rows

Remember State:

- Save expand/collapse state per user

- Restore state on page reload

- Deep-link support: URL can specify which FUs to expand

...



### ### Feature 7: Column Resizing & Reordering

```

Resize:

- Hover over column border → Cursor changes to ↔
- Click & drag to resize
- Double-click border → Auto-fit to content

Reorder:

- Click & hold column header
- Drag left/right to new position
- Drop between columns
- Visual indicator shows drop position

Lock Columns:

- Lock left columns (Item Name, Actions always visible)
- Lock right columns (ROI, RAG Status always visible)
- Middle columns scroll horizontally

```

---

---

## SECTION 6: EXAMPLE COMPLETE ROW DATA

---

---

### ### EXAMPLE FU ROW (All columns populated)

```

FU: Activia Yogurt 4x100g (FU001)

Plan: NKA Channel, Dairy Category

Has 4 child SKUs

COLUMN DATA:

- └─ [—] Collapse Icon (expanded)
- └─ Item Name: "FU001 - Activia Yogurt 4x100g"
- └─ Item Code: "FU001"
- └─ List Price: \$4.00 (from master data)
- └─ COGS per Piece: \$1.80 (from master data)
- └─ Base Volume: 8,000 pcs (sum of child SKUs)
- └─ Planned Volume: 10,500 pcs (sum of child SKUs - READ ONLY)
- └─ Incremental Volume: +2,500 pcs (calculated)

- └ Base GSV: \$32,000 (calculated)
- └ Planned GSV: \$42,000 (calculated)
- └ Incremental GSV: +\$10,000 (calculated)
- └ LTA On-Invoice %: 5% (from master data or override)
- └ LTA Off-Invoice %: 2% (from master data or override)
- └ LTA On-Invoice \$: \$2,100 (calculated)
- └ LTA Off-Invoice \$: \$798 (calculated)
- └ CPP On-Invoice %: [10%] (USER INPUT - editable)
- └ CPP Off-Invoice %: [5%] (USER INPUT - editable)
- └ Price Support \$/unit: [\$0.25] (USER INPUT - editable)
- └ Visibility MT/PH \$: [\$2,000] (USER INPUT - editable)
- └ Visibility GT \$: [\$0] (USER INPUT - editable)
- └ TPR/Drive Lumpsum \$: [\$0] (not used)
- └ Total On-Invoice Spend: \$6,090 (calculated)
- └ Total Off-Invoice Spend: \$4,635 (calculated)
- └ Total Planned Spend: \$10,725 (calculated)
- └ Incremental Spend: +\$3,123 (calculated)
- └ Base NIV: \$30,400 (calculated)
- └ Planned NIV: \$39,900 (calculated)
- └ Base Turnover: \$24,208 (calculated)
- └ Planned Turnover: \$31,275 (calculated)
- └ Incremental Turnover: +\$7,067 (calculated)
- └ TO Uplift %: 29.2% (calculated)
- └ Base COGS: \$14,400 (calculated)
- └ Planned COGS: \$18,900 (calculated)
- └ Base GP: \$9,808 (calculated)
- └ Planned GP: \$12,375 (calculated)
- └ Incremental GP: +\$2,567 (calculated)
- └ Base GM %: 40.5% (calculated)
- └ Planned GM %: 39.6% (calculated)
- └ Incremental GM %: 36.3% (calculated)
- └ GP ROI %: 24.5% ● (calculated - KEY METRIC)
- └ TO ROI %: 226.3% (calculated)
- └ RAG Status: ● GREEN "Excellent Performance" (evaluated)
- └ Actions: [:] Menu

Visual Representation:

[─] |FU001|Activia Yogurt

4x100g|\$4.00|\$1.80|8,000|10,500|+2,500|31.3%|\$32K|\$42K|5%|2%|[10%]| [5%]|
 [\$0.25]|[\$2000]|\$10,725|\$2,567|● 24.5%|● GREEN|[:]

SKU: Strawberry 400g (SKU001)
Parent FU: FU001 - Activia Yogurt 4x100g

COLUMN DATA:

- └ [] No expand icon (leaf node)
- └ Item Name: "↳ SKU001 - Strawberry 400g" (indented)
- └ Item Code: "SKU001"
- └ List Price: \$4.00 (inherited from FU)
- └ COGS per Piece: \$1.80 (inherited from FU)
- └ Base Volume: 3,200 pcs (from historical data)
- └ SContinuePlanned Volume: [4,200] pcs (USER INPUT - editable)
- └ Incremental Volume: +1,000 pcs (calculated)
- └ Volume Uplift %: 31.3% (calculated)
- └ Base GSV: \$12,800 (calculated)
- └ Planned GSV: \$16,800 (calculated)
- └ Incremental GSV: +\$4,000 (calculated)
- └ LTA On-Invoice %: 5% (inherited from FU - READ ONLY)
- └ LTA Off-Invoice %: 2% (inherited from FU - READ ONLY)
- └ LTA On-Invoice \$: \$840 (calculated)
- └ LTA Off-Invoice \$: \$319.20 (calculated)
- └ CPP On-Invoice %: 10% (inherited from FU - READ ONLY)
- └ CPP Off-Invoice %: 5% (inherited from FU - READ ONLY)
- └ Price Support \$/unit: \$0.25 (inherited from FU - READ ONLY)
- └ Visibility MT/PH \$: \$800 (distributed from FU \$2,000 - READ ONLY)
- └ Visibility GT \$: \$0 (not used)
- └ TPR/Drive Lumpsum \$: \$0 (not used)
- └ Total On-Invoice Spend: \$2,436 (calculated)
- └ Total Off-Invoice Spend: \$2,967.20 (calculated)
- └ Total Planned Spend: \$5,403.20 (calculated)
- └ Incremental Spend: +\$4,520 (calculated)
- └ Base NIV: \$12,160 (calculated)
- └ Planned NIV: \$15,960 (calculated)
- └ Base Turnover: \$11,916.80 (calculated)
- └ Planned Turnover: \$11,396.80 (calculated)
- └ Incremental Turnover: -\$520.00 ● (calculated - NEGATIVE!)
- └ TO Uplift %: -4.36% ● (calculated)
- └ Base COGS: \$5,760 (calculated)
- └ Planned COGS: \$7,560 (calculated)
- └ Base GP: \$6,156.80 (calculated)
- └ Planned GP: \$3,836.80 (calculated)

└ Base GM %: 51.67% (calculated)
└ Planned GM %: 33.67% (calculated)
└ Incremental GM %: N/A (both numerator and denominator negative)
└ GP ROI %: -51.33% ● (calculated - VERY BAD!)
└ TO ROI %: -11.50% ● (calculated)
└ RAG Status: ● RED "Poor Performance - Review Plan" (evaluated)
└ Actions: [:] Menu
Visual Representation:
[] | SKU001 | Strawberry
400g | \$4.00 | \$1.80 | 3,200 | [4,200] | +1,000 | 31.3% | \$12.8K | \$16.8K | 5% | 2% | 10% | 5% | \$
0.25 | \$800 | \$5,403 | -\$2,320 | ● -51.3% | ● RED | [:]

Warning Indicators on This Row:

- △ Incremental Turnover is NEGATIVE
- △ Incremental GP is NEGATIVE
- △ GP ROI is HIGHLY NEGATIVE
- △ RAG Status: RED - Requires attention

Inline Warning Message (appears below row when RED):

△ CRITICAL ISSUE: This SKU is losing money |

| |

| Problem: High promotional spend (\$5,403) relative to volume |

| Impact: Each additional unit sold loses money |

| |

| Recommendations: |

| 1. Reduce CPP discount from 10% to 5% |

| 2. Remove lumpsum visibility spend for this SKU |

| 3. Increase planned volume to 6,000+ units |

| 4. Consider removing this SKU from promotion |

| |

| [Apply Fix #1] [Optimize Automatically] [Remove SKU] [Dismiss] |

SECTION 7: RESPONSIVE BEHAVIOR & MOBILE ADAPTATION

Desktop (>1440px width)

Full grid visible with all columns

Side-by-side panels

Column grouping headers visible

[Fixed Left: Item Info] [Scrollable: KPIs] [Fixed Right]									
Item	Volume	GSV	Tactics	Spend	GP	ROI	RAG		
Name									
FU001	10.5K	\$42K	[10%]...	\$10.7K	\$2.5K	25%	●		
SKU1	4.2K	\$16.8	10%...	\$5.4K	-\$2.3	-51%	●		
SKU2	3.1K	\$12.6	10%...	\$4.1K	\$1.5K	36%	●		

Tablet (768px - 1440px)

Reduced column count (hide optional columns by default)

Priority columns remain visible

Horizontal scroll still available

Grand Totals Panel condensed

[Totals: Condensed Cards]					
Vol: 125K T0: \$487K ROI: 24.9% ●					
Item Name	Planned Vol	Tactics	ROI	RAG	
FU001	10,500	[Edit]	25%	●	
↳ SKU001	[4,200]	(inh)	-51%	●	

Mobile (< 768px)

Card-based layout (no grid)

Each FU/SKU becomes an expandable card

Swipe gestures for actions

Bottom action bar

🏠 Plan Overview	
ROI: ● 24.9%	Vol: 125K T0: \$487K

[▼] FU001 - Activia Yogurt 4x100g		
Planned Volume: 10,500 pcs		
ROI: 25.6% ●		
Tactics:		
• CPP On-Invoice: [10%]		
• Visibility: [\$2,000]		
[View Details] [Edit Tactics]		
↳ SKU001 - Strawberry 400g		
Planned Volume: [4,200] pcs		
ROI: -51.3% ● ▲ Issue		
[Edit] [Remove] [View Details]		
[+ Add FU] [ Save] [ Submit]		

Swipe Actions:

- ← Swipe Left: Delete
- Swipe Right: Edit
- ↓ Pull Down: Refresh

SECTION 8: SPECIAL FEATURES & EDGE CASES

Feature: Multi-Currency Support

If plan involves multiple currencies:

Display Format:

Planned GSV: \$42,000 USD		

Currency Selector in Header:

Display Currency: [USD ▼]

Options:

USD (Primary)

EUR (Exchange rate: 1.08)

GBP (Exchange rate: 1.27)

TRY (Exchange rate: 0.031)

All calculations stored in base currency (USD)

Display converted on-the-fly

Feature: Historical Comparison Overlay

Toggle to show historical performance:

| [✓] Show Last Promotion Comparison |

Enhanced Cell Display:

| GP ROI: 24.9% |

| Last: 18.5% |

| ↑ +6.4pp ● |

Shows:

Current plan value

Last similar promotion value

Difference (pp = percentage points)

Trend indicator

Feature: Version Comparison (Side-by-Side)

Compare current plan with previous version:

| Version Comparison: Current vs. v2 (3 days ago) |

| |

| Item Name | Current | Previous | Change |

Planned Vol	10,500	9,800	+700	●
CPP On%	10%	15%	-5pp	●
GP ROI	24.9%	18.2%	+6.7pp	●
SKU001				
Planned Vol	4,200	4,000	+200	
GP ROI	-51.3%	12.5%	-63.8pp	●
Overall ROI	24.9%	18.2%	+6.7pp	✓

Highlight changes:

Green: Improvement

Red: Degradation

Gray: No change

Feature: Batch Operations

Select multiple FUs and perform bulk actions:

3 FUs Selected
Bulk Actions: [Apply Tactic Template] [Adjust All Volumes by %] [Copy Tactics from Another FU] [Remove All Selected] [Export Selected]

Example: Adjust All Volumes by %

Adjust Volume for 3 FUs
Increase/Decrease: [Increase ▼] By: [10] % Preview: • FU001: 10,500 → 11,550 (+1,050) • FU002: 8,300 → 9,130 (+830)


```
| |
| [Cancel] [Apply Changes] |
└──────────────────────────┘
```

Edge Case: Zero Volume Handling

If user enters 0 or negative volume:

```
┌──────────────────────────┘
| ⚠ Invalid Volume Entry |
└──────────────────────────┘
| Planned Volume cannot be zero or negative. |
| |
| Did you mean to: |
| • Keep Base Volume (3,200)? |
| • Remove this SKU from plan? |
| • Enter different value? |
| |
| [Keep Base] [Remove SKU] [Re-enter] |
└──────────────────────────┘
```

Edge Case: Division by Zero Protection

When Base Volume = 0 (new product launch):

Volume Uplift % = Cannot calculate (show "N/A")

Incremental Volume = Planned Volume (no comparison)

RAG Status = Use Planned GP only (not incremental)

Display:

```
┌──────────────────────────┘
| Volume Uplift: N/A (New) |
| iVol: 5,000 pcs (No base) |
└──────────────────────────┘
```

Special calculation mode for new products:

Use Planned GP instead of iGP for ROI

Show "New Product" badge

Different RAG thresholds (if configured)

Edge Case: Exceeding Budget

Real-time budget validation:

When Total Spend exceeds available budget:

```
┌──────────────────────────┘
```

```
|-----|
| Plan Total Spend: $125,300 |
| Available Budget: $100,000 |
| Over Budget: $25,300 (25.3%) |
| |
| This plan cannot be submitted until spend |
| is reduced or additional budget is allocated.|
| |
| Suggestions: |
| • Reduce CPP discounts |
| • Remove visibility spends |
| • Request budget increase |
| |
| [Optimize Plan] [Request Budget] [Cancel] |
|-----|
```

Visual indicator in Grand Totals:

```
|-----|
| 📊 SPEND |
| Total: $125,300 ▲ |
| Budget: $100,000 |
| [██████████████████░░] 125% |
| ▲ OVER BUDGET |
|-----|
```

SECTION 9: PERFORMANCE & TECHNICAL REQUIREMENTS

Performance Targets:

Grid Rendering:

Initial load: < 2 seconds (100 FUs)

Add FU: < 100ms

Cell edit response: < 50ms

Calculation complete: < 500ms

Scroll smoothness: 60 FPS

Memory Usage:

Maximum: 500MB for 500 FUs
Use virtualization for >50 FUs
Lazy load SKU data on FU expand

Data Saving:

Auto-save: Every 2 seconds (debounced)
Manual save: < 1 second response
Optimistic UI updates

Real-time Updates:

Other users' changes: < 3 seconds notification
Conflict resolution: Automatic merge or manual review

Technical Implementation Stack:

```typescript

Grid Component: TanStack Table (React Table v8)

- Virtualization for large datasets
- Built-in sorting, filtering
- Column resize, reorder, pin
- Excellent TypeScript support

State Management: Zustand

- Lightweight, performant
- Easy to debug
- Perfect for complex state

Calculation Engine: Web Workers

- Offload heavy calculations
- Keep UI responsive
- Parallel processing

Validation: Zod

- Schema-based validation
- Type inference
- Clear error messages

Auto-save: Debounce + IndexedDB

- Sync to Supabase when online
- Conflict resolution

Real-time: Supabase Realtime

- Subscribe to plan changes
  - Show other users' cursors
  - Collaborative editing
- ...

### Database Optimization:

```sql

-- Indexes for fast queries

CREATE INDEX idx_plan_sku_volumes_plan_fu
ON plan_sku_volumes(plan_fu_id);

CREATE INDEX idx_plan_sku_volumes_sku
ON plan_sku_volumes(sku_id);

CREATE INDEX idx_plan_mechanic_values_plan_fu
ON plan_mechanic_values(plan_fu_id);

CREATE INDEX idx_plan_mechanic_values_mechanic
ON plan_mechanic_values(mechanic_id);

-- Materialized view for plan aggregates

CREATE MATERIALIZED VIEW plan_aggregates AS

SELECT

plan_id,

SUM(planned_volume) as total_planned_volume,

SUM(base_volume) as total_base_volume,

SUM(planned_gp) as total_planned_gp,

SUM(incremental_gp) as total_incremental_gp,

SUM(total_spend) as total_planned_spend,

(SUM(incremental_gp) / NULLIF(SUM(incremental_spend), 0)) * 100 as
overall_gp_roi

FROM plan_calculations

GROUP BY plan_id;

-- Refresh materialized view on changes

CREATE OR REPLACE FUNCTION refresh_plan_aggregates()

```
BEGIN
REFRESH MATERIALIZED VIEW CONCURRENTLY plan_aggregates;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trigger_refresh_aggregates
AFTER INSERT OR UPDATE OR DELETE ON plan_sku_volumes
FOR EACH STATEMENT
EXECUTE FUNCTION refresh_plan_aggregates();
```
```

---

---

## SECTION 10: USER EXPERIENCE ENHANCEMENTS

---

---

### ### UX Feature 1: Smart Defaults

When user adds new FU to plan:

Auto-populate with intelligent defaults:

Base Volume: From historical data (last 3 months average)

Planned Volume:  $\text{Base Volume} \times 1.25$  (25% uplift default)

LTA percentages: From CPL master data

Suggested tactics: Based on category best practices

Show notification:

```
| ✨ Smart Defaults Applied |
| |
| We've pre-filled this FU with: |
| • Historical base volumes |
| • 25% volume uplift target |
| • Standard LTA rates for Carrefour CPL |
| |
| Review and adjust as needed. |
| |
| [Got it] [Learn More] |
```

### UX Feature 2: Progressive Disclosure  
Hide complexity by default, reveal on demand:  
Compact View (Default):  
Shows only essential columns:

Item Name  
Planned Volume  
Key Tactics (2-3 most common)  
GP ROI  
RAG Status

Item Name	Vol	CPP %	ROI	RAG
FU001	10,500	[10%]	25%	
[Show Details ▼]				

Detailed View (Expanded):  
Shows all available columns

Item	Vol	iVol	GSV	LTA	CPP	CPP	Vis	Spend	GP	ROI	RAG
Name			On%	On%	Off	MT					
FU001	...	...	...	...	...	...	...	...	...	...	
[Hide Details ▲]											

### UX Feature 3: Contextual Help  
Every cell has contextual help available:  
Hover over info icon or press F1:

 Incremental Gross Profit (iGP)
What it is:
The additional profit generated by this
promotion compared to baseline sales.
How it's calculated:
Planned GP - Base GP

```
| This shows if the promotion actually |
| increases your profit after covering all |
| costs. Positive = Good, Negative = Bad. |
| |
| Target: Positive value |
| Your value: $2,567 ✓ |
| |
| [Watch Tutorial] [Close] |
```

### ### UX Feature 4: Undo/Redo Stack

Full undo/redo support:

Toolbar shows:

```
| [↶ Undo] [↷ Redo] |
| Last: Changed CPP to 15% |
```

Keyboard: Ctrl+Z / Ctrl+Y

History Panel (optional):

```
| Change History |
| Now: CPP 15% → ROI 22.1% |
| ↓ |
| 2 min ago: CPP 10% → ROI 24.9% |
| ↓ |
| 5 min ago: Added FU002 |
| ↓ |
| 10 min ago: Volume 10K → 10.5K |
| |
| [Restore Any Point] |
```

### ### UX Feature 5: Guided Tour (First-Time Users)

Interactive tutorial overlay:

Step 1:

```
| 🎯 Welcome to Plan Detail Screen! |
| This is where you build and optimize |
```

| |  
| Let's take a quick tour (2 minutes) |  
| |  
| [Start Tour] [Skip] [Remind Me Later] |  
|\_\_\_\_\_|

Step 2: (Highlight Grand Totals Panel)

|\_\_\_\_\_|  
| 🖱️ This panel shows your plan's |  
| overall performance in real-time. |  
| |  
| Watch how these numbers change as you |  
| make edits below. |  
| |  
| [Next] [2/8] |  
|\_\_\_\_\_|

Continue through key features...

---

---

## FINAL SUMMARY: PLANNING SCREEN COMPLETE SPECIFICATION

---

---

This Planning Screen is the CORE of CollMind TPM. It features:

- ✓ Dynamic column generation based on plan context
- ✓ Hierarchical FU/SKU grid with expand/collapse
- ✓ Real-time KPI calculations with <500ms response
- ✓ RAG status evaluation at SKU, FU, and Plan levels
- ✓ Intelligent tactic distribution (percentage, per-unit, lumpsum)
- ✓ Volume input at SKU level, tactics at FU level
- ✓ Grand Totals Panel with 6 key metrics
- ✓ Inline warnings, validation, and optimization suggestions
- ✓ Auto-save, undo/redo, version history
- ✓ Bulk edit, copy/paste, keyboard shortcuts
- ✓ Mobile-responsive with card-based layout
- ✓ Budget tracking with real-time alerts
- ✓ Multi-user collaborative editing support
- ✓ Export to Excel with formatting
- ✓ Performance optimized for 500+ FUs



All KPIs from the library are supported with proper formulas and dependencies.

The calculation engine handles complex scenarios including new products, zero values, and edge cases.

The UI provides extensive feedback, guidance, and automation to help planners create profitable promotions.

This completes the comprehensive specification for the Planning Detail Screen. Would you like me to:

Create similar detailed specs for the Category Manager Approval screen?

Generate the complete TypeScript interfaces and types?

Create the calculation engine implementation code?

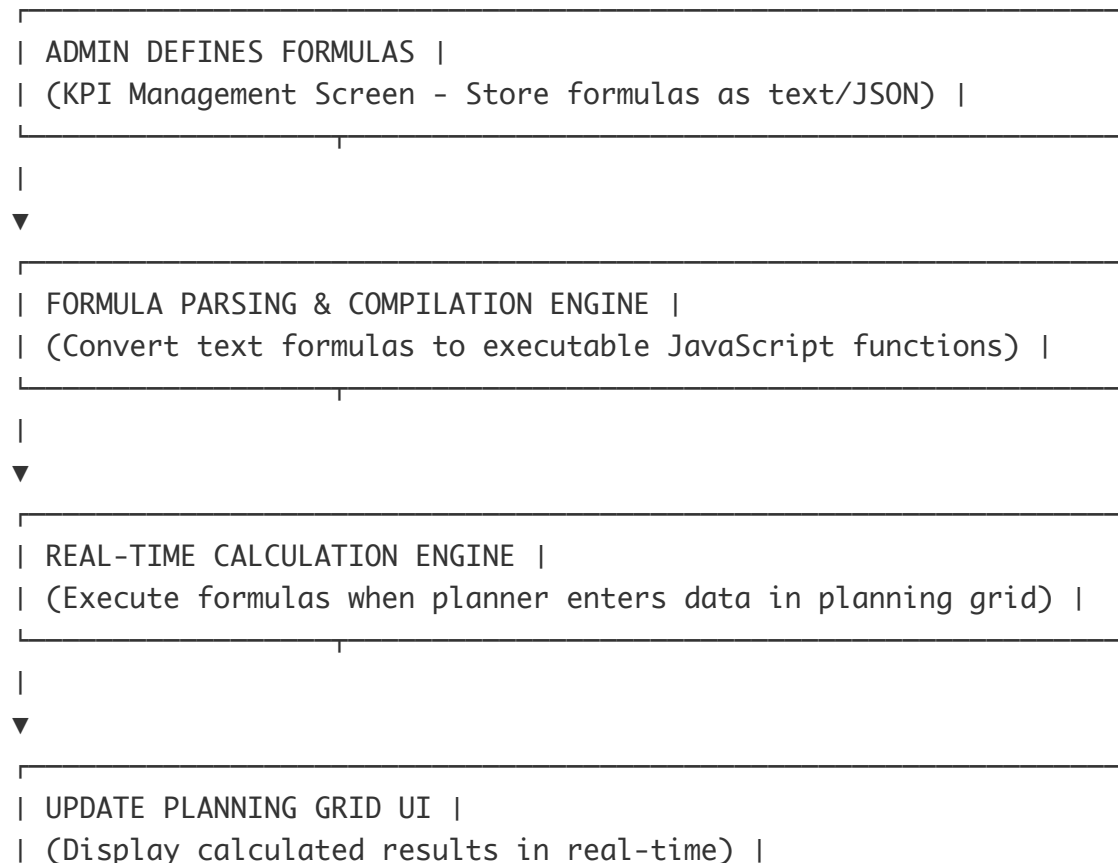
Design the database triggers for auto-calculations?

Dynamic KPI Calculation Engine - Complete Specification

You're absolutely right! The system must be formula-driven and dynamic.

Let me provide a complete architecture for this.

## ARCHITECTURE OVERVIEW



## PHASE 1: KPI DEFINITION & FORMULA STORAGE

Database Schema for Dynamic KPIs

```
sqlCREATE TABLE kpis (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 kpi_code VARCHAR(50) UNIQUE NOT NULL, -- e.g., 'INCR_VOL', 'GP_ROI'
 kpi_name VARCHAR(200) NOT NULL,
 kpi_group VARCHAR(100) NOT NULL,
 -- FORMULA CONFIGURATION (Critical!)
 formula_type VARCHAR(50) NOT NULL, -- 'expression', 'javascript',
 'conditional'
 formula_text TEXT NOT NULL, -- The actual formula as text
 formula_compiled JSONB, -- Pre-compiled formula structure
 -- DEPENDENCIES (What this KPI needs to calculate)
 depends_on_kpis JSONB, -- Array of KPI codes: ['PLANNED_VOL',
 'BASE_VOL']
 depends_on_master_data JSONB, -- Array: ['BPTT', 'COGS']
 depends_on_user_input JSONB, -- Array: ['CPP_ON_PCT', 'PLANNED_VOLUME']
 -- CALCULATION ORDER (Critical for cascade calculations)
 calculation_order INTEGER, -- Lower number = calculate first
 calculation_level VARCHAR(50), -- 'sku', 'fu', 'plan'
 -- VALIDATION
 validation_rules JSONB,
 -- DISPLAY
 display_format VARCHAR(50), -- 'number', 'currency', 'percentage'
 decimal_places INTEGER DEFAULT 2,
 show_in_grid BOOLEAN DEFAULT true,
 -- STATUS
 is_active BOOLEAN DEFAULT true,
 created_at TIMESTAMP DEFAULT NOW(),
 updated_at TIMESTAMP DEFAULT NOW()
);

-- Example KPI Records
INSERT INTO kpis (kpi_code, kpi_name, kpi_group, formula_type,
 formula_text, depends_on_kpis, depends_on_master_data,
 depends_on_user_input, calculation_order, calculation_level,
 display_format) VALUES

-- Level 1: Direct from user input or master data (Order 1-10)
```

```
('PLANNED_VOL', 'Planned Volume', 'Volume', 'user_input', 'USER_INPUT',
 '[]', '[]', '["PLANNED_VOLUME"]', 1, 'sku', 'number'),
('BASE_VOL', 'Base Volume', 'Volume', 'external',
 'FROM_HISTORICAL_DATA', '[]', '[]', '[]', 1, 'sku', 'number'),
('BPTT', 'List Price per Piece', 'Master Data', 'external',
 'FROM_MASTER_DATA', '[]', '["list_price"]', '[]', 1, 'sku', 'currency'),
('COGS', 'COGS per Piece', 'Master Data', 'external',
 'FROM_MASTER_DATA', '[]', '["cogs_per_piece"]', '[]', 1, 'sku',
 'currency'),
('CPP_ON_PCT', 'CPP On-Invoice %', 'Tactics', 'user_input',
 'USER_INPUT', '[]', '[]', '["CPP_ON_INVOICE_PCT"]', 1, 'fu',
 'percentage'),
```

-- Level 2: Simple calculations (Order 11-20)

```
('INCR_VOL', 'Incremental Volume', 'Volume', 'expression', 'PLANNED_VOL
- BASE_VOL', '["PLANNED_VOL", "BASE_VOL"]', '[]', '[]', 11, 'sku',
 'number'),
('VOL_UPLIFT_PCT', 'Volume Uplift %', 'Volume', 'expression', '(INCR_VOL
/ BASE_VOL) * 100', '["INCR_VOL", "BASE_VOL"]', '[]', '[]', 12, 'sku',
 'percentage'),
('BASE_GSV', 'Base GSV', 'GSV', 'expression', 'BASE_VOL * BPTT',
 '["BASE_VOL"]', '["BPTT"]', '[]', 11, 'sku', 'currency'),
('PLANNED_GSV', 'Planned GSV', 'GSV', 'expression', 'PLANNED_VOL *
BPTT', '["PLANNED_VOL"]', '["BPTT"]', '[]', 11, 'sku', 'currency'),
```

-- Level 3: Dependent calculations (Order 21-30)

```
('INCR_GSV', 'Incremental GSV', 'GSV', 'expression', 'PLANNED_GSV -
BASE_GSV', '["PLANNED_GSV", "BASE_GSV"]', '[]', '[]', 21, 'sku',
 'currency'),
('CPP_ON_SPEND', 'CPP On-Invoice Spend', 'Spend', 'expression',
 '(PLANNED_GSV - PLANNED_LTA_ON) * (CPP_ON_PCT / 100)', '["PLANNED_GSV",
 "PLANNED_LTA_ON", "CPP_ON_PCT"]', '[]', '[]', 21, 'sku', 'currency'),
```

-- Level 4: Complex profit calculations (Order 31-40)

```
('BASE_COGS', 'Base COGS', 'Profit', 'expression', 'BASE_VOL * COGS',
 '["BASE_VOL"]', '["COGS"]', '[]', 31, 'sku', 'currency'),
('PLANNED_COGS', 'Planned COGS', 'Profit', 'expression', 'PLANNED_VOL *
COGS', '["PLANNED_VOL"]', '["COGS"]', '[]', 31, 'sku', 'currency'),
```

```

('PLANNED_GP', 'Planned Gross Profit', 'Profit', 'expression',
 'PLANNED_TO - PLANNED_COGS', ['PLANNED_TO', "PLANNED_COGS"], '[]',
 '[]', 32, 'sku', 'currency'),
('INCR_GP', 'Incremental GP', 'Profit', 'expression', 'PLANNED_GP -
BASE_GP', ["PLANNED_GP", "BASE_GP"], '[]', '[]', 33, 'sku',
'currency'),

-- Level 5: ROI calculations (Order 41-50)
('GP_ROI_PCT', 'GP ROI %', 'ROI', 'expression', '(INCR_GP / INCR_SPEND)
* 100', ["INCR_GP", "INCR_SPEND"], '[]', '[]', 41, 'sku',
'percentage'),

-- Conditional formulas
('RAG_STATUS', 'RAG Status', 'ROI', 'conditional',
 'IF(GP_ROI_PCT >= 20, "GREEN", IF(GP_ROI_PCT >= 10, "AMBER", "RED"))',
 ["GP_ROI_PCT"], '[]', '[]', 42, 'sku', 'text');

```

## PHASE 2: FORMULA PARSING & COMPILATION ENGINE

Formula Parser Service

typescript// services/formulaParser.ts

```

interface ParsedFormula {
 type: 'expression' | 'javascript' | 'conditional';
 dependencies: string[];
 execute: (context: Record<string, any>) => any;
 raw: string;
}

class FormulaParser {
 /**
 * Parse formula text and create executable function
 */
 parseFormula(formula: string, formulaType: string): ParsedFormula {
 const dependencies = this.extractDependencies(formula);
 switch (formulaType) {
 case 'expression':
 return this.parseExpression(formula, dependencies);
 case 'conditional':
 return this.parseConditional(formula, dependencies);
 case 'javascript':

```

```

default:
throw new Error(`Unknown formula type: ${formulaType}`);
}
}

/**
 * Extract KPI dependencies from formula
 * Example: "PLANNED_VOL - BASE_VOL" → ["PLANNED_VOL", "BASE_VOL"]
 */
private extractDependencies(formula: string): string[] {
// Match variable names (uppercase with underscores)
const regex = /\b[A-Z][A-Z0-9_]*\b/g;
const matches = formula.match(regex) || [];
// Remove duplicates and function names
const functions = ['IF', 'SUM', 'AVG', 'MAX', 'MIN', 'ABS', 'ROUND'];
return [...new Set(matches)].filter(m => !functions.includes(m));
}

/**
 * Parse simple mathematical expression
 * Example: "PLANNED_VOL - BASE_VOL"
 */
private parseExpression(formula: string, dependencies: string[]):
ParsedFormula {
return {
type: 'expression',
dependencies,
raw: formula,
execute: (context: Record<string, any>) => {
try {
// Replace KPI codes with actual values
let expression = formula;
for (const dep of dependencies) {
const value = context[dep];
if (value === undefined || value === null) {
console.warn(`Missing dependency: ${dep}`);
return null;
}
}
// Replace all occurrences
expression = expression.replace(

```

```

String(value)
);
}
// Safely evaluate mathematical expression
return this.safeEval(expression);
} catch (error) {
console.error(`Formula execution error: ${formula}`, error);
return null;
}
}
};
}

/**
 * Parse conditional formula
 * Example: "IF(GP_ROI_PCT >= 20, 'GREEN', IF(GP_ROI_PCT >= 10, 'AMBER',
'RED'))"
 */
private parseConditional(formula: string, dependencies: string[]):
ParsedFormula {
return {
type: 'conditional',
dependencies,
raw: formula,
execute: (context: Record<string, any>) => {
try {
// Convert IF statements to JavaScript ternary operators
let jsExpression = formula
.replace(/IF\s*\(/g, '(')
.replace(/,\s*([^\,)]+)\s*,\s*([^\,)]+)\s*\)/g, ' ? $1 : $2)');
// Replace KPI codes with values
for (const dep of dependencies) {
const value = context[dep];
if (value === undefined || value === null) {
return null;
}
jsExpression = jsExpression.replace(
new RegExp(`\\b${dep}\\b`, 'g'),
typeof value === 'string' ? `"${value}"` : String(value)
);
}
}
}
}

```

```

return this.safeEval(jsExpression);
} catch (error) {
console.error(`Conditional formula error: ${formula}`, error);
return null;
}
};
}

/**
 * Parse JavaScript function
 * Example: "function(ctx) { return ctx.PLANNED_VOL * 1.5; }"
 */
private parseJavaScript(formula: string, dependencies: string[]):
ParsedFormula {
return {
type: 'javascript',
dependencies,
raw: formula,
execute: (context: Record<string, any>) => {
try {
// Create function from formula string
const fn = new Function('context', `return (${formula})(context);`);
return fn(context);
} catch (error) {
console.error(`JavaScript formula error: ${formula}`, error);
return null;
}
}
};
}

/**
 * Safely evaluate mathematical expression
 * Uses Function constructor with restricted scope
 */
private safeEval(expression: string): number | null {
try {
// Whitelist allowed operations
const allowedChars = /^[d\s+\-*/.()]+$ /;

```

```

throw new Error('Invalid characters in expression');
}
// Evaluate in restricted scope
const result = new Function(`"use strict"; return (${expression})`)(());
return typeof result === 'number' && !isNaN(result) ? result : null;
} catch (error) {
console.error(`Safe eval error: ${expression}`, error);
return null;
}
}

/**
 * Validate formula syntax before saving
 */
validateFormula(formula: string, formulaType: string): {
 isValid: boolean;
 errors: string[];
 warnings: string[];
} {
 const errors: string[] = [];
 const warnings: string[] = [];

 try {
 // Try to parse the formula
 const parsed = this.parseFormula(formula, formulaType);
 // Check if dependencies exist
 if (parsed.dependencies.length === 0 && formulaType === 'expression') {
 warnings.push('Formula has no dependencies - is this intentional?');
 }

 // Test with dummy data
 const dummyContext: Record<string, any> = {};
 parsed.dependencies.forEach(dep => {
 dummyContext[dep] = 100; // Use 100 as test value
 });
 const testResult = parsed.execute(dummyContext);
 if (testResult === null) {
 errors.push('Formula execution returned null with test data');
 }
 }
}

```



```

errors.push(error.message);
}

return {
 isValid: errors.length === 0,
 errors,
 warnings
};
}
}

```

```

export const formulaParser = new FormulaParser();

```

PHASE 3: CALCULATION DEPENDENCY GRAPH  
 Build Calculation Order  
 typescript// services/calculationGraph.ts

```

interface KPINode {
 kpiCode: string;
 formula: ParsedFormula;
 dependencies: string[];
 dependents: string[]; // KPIs that depend on this one
 level: number; // Calculation order level
}

```

```

class CalculationGraph {
 private nodes: Map<string, KPINode> = new Map();
 private calculationOrder: string[] = [];

```

```

/**
 * Build dependency graph from KPI definitions
 */

```

```

async buildGraph(kpis: any[]) {
 // Reset
 this.nodes.clear();
 this.calculationOrder = [];

```

```

 // Create nodes
 for (const kpi of kpis) {
 if (!kpi.is_active) continue;

```

```

const parsed = formulaParser.parseFormula(
kpi.formula_text,
kpi.formula_type
);

this.nodes.set(kpi.kpi_code, {
kpiCode: kpi.kpi_code,
formula: parsed,
dependencies: parsed.dependencies,
dependents: [],
level: 0
});
}

// Build dependency relationships
for (const [code, node] of this.nodes) {
for (const dep of node.dependencies) {
const depNode = this.nodes.get(dep);
if (depNode) {
depNode.dependents.push(code);
}
}
}

// Calculate levels (topological sort)
this.calculateLevels();
// Create ordered calculation sequence
this.calculationOrder = this.topologicalSort();

console.log('📊 Calculation graph built:', {
totalKPIs: this.nodes.size,
calculationOrder: this.calculationOrder
});
}

/**
 * Calculate level for each KPI (how many steps from user input)
 */
private calculateLevels() {
const visited = new Set<string>();

```

```

if (visited.has(code)) {
 return this.nodes.get(code)?.level || 0;
}
visited.add(code);
const node = this.nodes.get(code);
if (!node) return 0;

// Level is max of dependency levels + 1
let maxDepLevel = 0;
for (const dep of node.dependencies) {
 maxDepLevel = Math.max(maxDepLevel, calculateLevel(dep, level + 1));
}
node.level = maxDepLevel + 1;
return node.level;
};

// Calculate levels for all nodes
for (const code of this.nodes.keys()) {
 calculateLevel(code);
}
}

/**
 * Topological sort to get calculation order
 */
private topologicalSort(): string[] {
 const sorted: string[] = [];
 const visited = new Set<string>();
 const temp = new Set<string>();

 const visit = (code: string) => {
 if (temp.has(code)) {
 throw new Error(`Circular dependency detected involving ${code}`);
 }
 if (visited.has(code)) return;

 temp.add(code);
 const node = this.nodes.get(code);
 if (node) {
 // Visit dependencies first

```

```

if (this.nodes.has(dep)) {
visit(dep);
}
}
}

temp.delete(code);
visited.add(code);
sorted.push(code);
};

// Visit all nodes
for (const code of this.nodes.keys()) {
if (!visited.has(code)) {
visit(code);
}
}

return sorted;
}

/**
 * Get KPIs that need recalculation when a value changes
 */
getAffectedKPIs(changedKPI: string): string[] {
const affected = new Set<string>();
const queue = [changedKPI];

while (queue.length > 0) {
const current = queue.shift()!;
const node = this.nodes.get(current);
if (node) {
for (const dependent of node.dependents) {
if (!affected.has(dependent)) {
affected.add(dependent);
queue.push(dependent);
}
}
}
}
}

```

```
// Return in calculation order
return this.calculationOrder.filter(kpi => affected.has(kpi));
}
```

```
getCalculationOrder(): string[] {
return [...this.calculationOrder];
}
```

```
getNode(kpiCode: string): KPINode | undefined {
return this.nodes.get(kpiCode);
}
}
```

```
export const calculationGraph = new CalculationGraph();
```

PHASE 4: REAL-TIME CALCULATION ENGINE  
SKU-Level Calculator  
typescript// services/calculationEngine.ts

```
interface CalculationContext {
// User inputs
planned_volume?: number;
tactic_values?: Record<string, number>; // e.g., { CPP_ON_PCT: 10 }
// Master data
list_price?: number;
cogs_per_piece?: number;
base_volume?: number;
// Already calculated values
calculated?: Record<string, any>;
}
```

```
interface CalculationResult {
kpiCode: string;
value: any;
formatted: string;
error?: string;
}
```

```
class CalculationEngine {
private kpisCache: Map<string, any> = new Map();
```

```

/**
 * Initialize engine with KPI definitions from database
 */
async initialize() {
 // Fetch all active KPIs from database
 const { data: kpis, error } = await supabase
 .from('kpis')
 .select('*')
 .eq('is_active', true)
 .order('calculation_order');

 if (error) throw error;

 // Cache KPIs
 kpis.forEach(kpi => {
 this.kpisCache.set(kpi.kpi_code, kpi);
 });

 // Build calculation graph
 await calculationGraph.buildGraph(kpis);

 console.log('✅ Calculation engine initialized with', kpis.length,
 'KPIs');
}

/**
 * Calculate all KPIs for a single SKU
 */
calculateSKU(context: CalculationContext): Record<string,
 CalculationResult> {
 const results: Record<string, CalculationResult> = {};
 const calculationOrder = calculationGraph.getCalculationOrder();

 // Build execution context
 const execContext: Record<string, any> = {
 // User inputs
 PLANNED_VOL: context.planned_volume,
 ...context.tactic_values,
 // Master data
 BPTT: context.list_price,

```

```

BASE_VOL: context.base_volume,
// Pre-calculated values
...context.calculated
};

// Execute formulas in dependency order
for (const kpiCode of calculationOrder) {
const kpi = this.kpisCache.get(kpiCode);
if (!kpi) continue;

try {
// Skip if it's user input
if (kpi.formula_type === 'user_input' || kpi.formula_type ===
'external') {
continue;
}

// Parse and execute formula
const parsed = formulaParser.parseFormula(
kpi.formula_text,
kpi.formula_type
);

const value = parsed.execute(execContext);

// Store result for dependent calculations
execContext[kpiCode] = value;

// Format for display
const formatted = this.formatValue(value, kpi);

results[kpiCode] = {
kpiCode,
value,
formatted
};

} catch (error: any) {
console.error(`Error calculating ${kpiCode}:`, error);
results[kpiCode] = {

```

```

value: null,
formatted: 'Error',
error: error.message
};
}
}

return results;
}

/**
 * Recalculate only affected KPIs when one value changes
 */
recalculate(
 changedKPI: string,
 newValue: any,
 currentContext: Record<string, any>
): Record<string, CalculationResult> {
 const results: Record<string, CalculationResult> = {};
 // Update context with new value
 currentContext[changedKPI] = newValue;

 // Get affected KPIs
 const affectedKPIs = calculationGraph.getAffectedKPIs(changedKPI);

 console.log(`🔄 Recalculating ${affectedKPIs.length} KPIs affected by ${changedKPI}`);

 // Recalculate only affected KPIs
 for (const kpiCode of affectedKPIs) {
 const kpi = this.kpisCache.get(kpiCode);
 if (!kpi) continue;

 try {
 const parsed = formulaParser.parseFormula(
 kpi.formula_text,
 kpi.formula_type
);

 const value = parsed.execute(currentContext);

```



```

results[kpiCode] = {
 kpiCode,
 value,
 formatted: this.formatValue(value, kpi)
};

} catch (error: any) {
 results[kpiCode] = {
 kpiCode,
 value: null,
 formatted: 'Error',
 error: error.message
 };
}
}

return results;
}

/**
 * Format value for display based on KPI configuration
 */
private formatValue(value: any, kpi: any): string {
 if (value === null || value === undefined) return '-';

 const decimals = kpi.decimal_places || 2;

 switch (kpi.display_format) {
 case 'currency':
 return new Intl.NumberFormat('en-US', {
 style: 'currency',
 currency: 'USD',
 minimumFractionDigits: decimals,
 maximumFractionDigits: decimals
 }).format(value);

 case 'percentage':
 return `${value.toFixed(decimals)}%`;
 }
}

```

```

return new Intl.NumberFormat('en-US', {
 minimumFractionDigits: decimals,
 maximumFractionDigits: decimals
}).format(value);

default:
return String(value);
}
}

/**
 * Aggregate SKU values to FU level
 */
aggregateFU(skuResults: Record<string, any>[]): Record<string, any> {
 const fuResults: Record<string, any> = {};

 // Get all KPI codes
 const kpiCodes = new Set<string>();
 skuResults.forEach(sku => {
 Object.keys(sku).forEach(code => kpiCodes.add(code));
 });

 // Aggregate each KPI
 for (const kpiCode of kpiCodes) {
 const kpi = this.kpisCache.get(kpiCode);
 if (!kpi) continue;

 const aggregationMethod = kpi.aggregation_method_fu || 'sum';
 const values = skuResults
 .map(sku => sku[kpiCode]?.value)
 .filter(v => v !== null && v !== undefined && !isNaN(v));

 if (values.length === 0) {
 fuResults[kpiCode] = null;
 continue;
 }

 switch (aggregationMethod) {
 case 'sum':
 fuResults[kpiCode] = values.reduce((a, b) => a + b, 0);

```

```

case 'avg':
fuResults[kpiCode] = values.reduce((a, b) => a + b, 0) / values.length;
break;
case 'min':
fuResults[kpiCode] = Math.min(...values);
break;
case 'max':
fuResults[kpiCode] = Math.max(...values);
break;
default:
fuResults[kpiCode] = values[0];
}
}

return fuResults;
}
}

```

```

export const calculationEngine = new CalculationEngine();

```

## PHASE 5: REACT INTEGRATION - PLANNING GRID

React Hook for Dynamic Calculations

typescript// hooks/usePlanCalculations.ts

```

import { useState, useEffect, useCallback, useRef } from 'react';
import { calculationEngine } from '../services/calculationEngine';
import { debounce } from 'lodash';

```

```

interface SKUData {
sku_id: string;
planned_volume: number;
base_volume: number;
list_price: number;
cogs_per_piece: number;
calculated: Record<string, any>;
}

```

```

interface FUData {
fu_id: string;
tactics: Record<string, number>;
}

```

```

 calculated: Record<string, any>;
 }

 export function usePlanCalculations(planId: string) {
 const [fus, setFUs] = useState<FUData[]>([]);
 const [grandTotals, setGrandTotals] = useState<Record<string, any>>({});
 const [isCalculating, setIsCalculating] = useState(false);
 const calculationQueue = useRef<Array<() => void>>([]);

 // Initialize calculation engine
 useEffect(() => {
 calculationEngine.initialize().catch(console.error);
 }, []);

 /**
 * Update SKU planned volume (user input)
 */
 const updateSKUVolume = useCallback((
 fuId: string,
 skuId: string,
 newVolume: number
) => {
 setFUs(prevFUs => {
 const updated = prevFUs.map(fu => {
 if (fu.fu_id !== fuId) return fu;

 const updatedSKUs = fu.skus.map(sku => {
 if (sku.sku_id !== skuId) return sku;

 // Recalculate this SKU
 const context: CalculationContext = {
 planned_volume: newVolume,
 base_volume: sku.base_volume,
 list_price: sku.list_price,
 cogs_per_piece: sku.cogs_per_piece,
 tactic_values: fu.tactics,
 calculated: sku.calculated
 };

 const results = calculationEngine.calculateSKU(context);

```

```

// Update calculated values
const newCalculated: Record<string, any> = {};
Object.entries(results).forEach(([kpiCode, result]) => {
 newCalculated[kpiCode] = result.value;
});

return {
 ...sku,
 planned_volume: newVolume,
 calculated: { ...sku.calculated, ...newCalculated }
};
});

// Aggregate to FU level
const fuCalculated = calculationEngine.aggregateFU(
 updatedSKUs.map(sku => sku.calculated)
);

return {
 ...fu,
 skus: updatedSKUs,
 calculated: fuCalculated
};
});

// Recalculate grand totals
recalculateGrandTotals(updated);

return updated;
});
}, []);

/**
 * Update FU tactic value (user input)
 */
const updateFUTactic = useCallback((
 fuId: string,
 tacticCode: string,
 newValue: number
) => {

```

```

const updated = prevFUs.map(fu => {
 if (fu.fu_id !== fuId) return fu;

 // Update tactic value
 const updatedTactics = {
 ...fu.tactics,
 [tacticCode]: newValue
 };

 // Recalculate all SKUs under this FU (tactics affect all SKUs)
 const updatedSKUs = fu.skus.map(sku => {
 const context: CalculationContext = {
 planned_volume: sku.planned_volume,
 base_volume: sku.base_volume,
 list_price: sku.list_price,
 cogs_per_piece: sku.cogs_per_piece,
 tactic_values: updatedTactics,
 calculated: sku.calculated
 };
 // Recalculate using updated tactic
 const results = calculationEngine.recalculate(
 tacticCode,
 newValue,
 context
);

 // Merge new calculated values
 const newCalculated: Record<string, any> = { ...sku.calculated };
 Object.entries(results).forEach(([kpiCode, result]) => {
 newCalculated[kpiCode] = result.value;
 });

 return {
 ...sku,
 calculated: newCalculated
 };
 });

 // Aggregate to FU level
 const fuCalculated = calculationEngine.aggregateFU(

```

```

);

return {
 ...fu,
 tactics: updatedTactics,
 skus: updatedSKUs,
 calculated: fuCalculated
};
});

// Recalculate grand totals
recalculateGrandTotals(updated);

return updated;
});
}, []);
/**

```

Recalculate grand totals (plan level)

```

*/
const recalculateGrandTotals = useCallback((fusData: FUData[]) => {
 const allFUCalculated = fusData.map(fu => fu.calculated);
 const totals = calculationEngine.aggregateFU(allFUCalculated);
 setGrandTotals(totals);
}, []);

/**

```

Debounced auto-save

```

*/
const debouncedSave = useCallback(
 debounce(async (planData: FUData[]) => {
 console.log('💾 Auto-saving plan...');
 try {
 // Save to Supabase
 await savePlanToDatabase(planData);
 console.log('✅ Plan saved successfully');
 } catch (error) {
 console.error('❌ Save failed:', error);
 }
 })
);

```

```
□
);
```

```
// Trigger auto-save whenever FUs change
```

```
useEffect(() => {
 if (fus.length > 0) {
 debouncedSave(fus);
 }
}, [fus, debouncedSave]);
return {
 fus,
 grandTotals,
 isCalculating,
 updateSKUVolume,
 updateFUTactic,
 recalculateGrandTotals
};
}
/**
```

```
Save plan data to database
```

```
*/
```

```
async function savePlanToDatabase(fusData: FUData[]): Promise<void> {
 // Save SKU volumes
 const skuUpdates = fusData.flatMap(fu =>
 fu.skus.map(sku => ({
 sku_id: sku.sku_id,
 planned_volume: sku.planned_volume,
 calculated_values: sku.calculated
 })))
};
```

```
await supabase
```

```
.from('plan_sku_volumes')
```

```
.upsert(skuUpdates);
```

```
// Save FU tactics
```

```
const tacticUpdates = fusData.flatMap(fu =>
 Object.entries(fu.tactics).map(([tacticCode, value]) => ({
 plan_fu_id: fu.fu_id,
 tactic_code: tacticCode,
```



```

)))
);
 await supabase
 .from('plan_mechanic_values')
 .upsert(tacticUpdates);
}

```

---

## \*\*PHASE 6: PLANNING GRID COMPONENT\*\*

### Dynamic Grid Component

```typescript

// components/PlanningGrid.tsx

```

import React, { useState, useCallback } from 'react';
import { usePlanCalculations } from '../hooks/usePlanCalculations';
import { useKPIColumns } from '../hooks/useKPIColumns';

```

```

interface PlanningGridProps {
  planId: string;
  planContext: {
    channel: string;
    category: string;
  };
}

```

```

export const PlanningGrid: React.FC<PlanningGridProps> = ({
  planId,
  planContext
}) => {
  const {
    fus,
    grandTotals,
    updateSKUVolume,
    updateFUTactic
  } = usePlanCalculations(planId);

```

```

// Dynamically get columns based on plan context and KPI configuration
const { columns, tacticColumns } = useKPIColumns(planContext);

```

```

const [expandedFUs, setExpandedFUs] = useState<Set<string>>(new Set());

const toggleFU = useCallback((fuId: string) => {
  setExpandedFUs(prev => {
    const next = new Set(prev);
    if (next.has(fuId)) {
      next.delete(fuId);
    } else {
      next.add(fuId);
    }
    return next;
  });
}, []);

return (
  <div className="planning-grid">
    {/* Grand Totals Panel */}
    <GrandTotalsPanel totals={grandTotals} />

    {/* Grid Table */}
    <div className="overflow-x-auto">
      <table className="w-full border-collapse">
        <thead>
          <DynamicGridHeader
            columns={columns}
            tacticColumns={tacticColumns}
          />
        </thead>
        <tbody>
          {fus.map(fu => (
            <React.Fragment key={fu.fu_id}>
              {/* FU Row */}
              <FURow
                fu={fu}
                columns={columns}
                tacticColumns={tacticColumns}
                isExpanded={expandedFUs.has(fu.fu_id)}
                onToggle={() => toggleFU(fu.fu_id)}
                onTacticChange={(tacticCode, value) =>
                  updateFUTactic(fu.fu_id, tacticCode, value)}

```

```
/>
```

```
    { /* SKU Rows (if expanded) */  
    {expandedFUs.has(fu.fu_id) && fu.skus.map(sku => (  
    <SKURow  
    key={sku.sku_id}  
    sku={sku}  
    fuTactics={fu.tactics}  
    columns={columns}  
    tacticColumns={tacticColumns}  
    onVolumeChange={(value) =>  
    updateSKUVolume(fu.fu_id, sku.sku_id, value)  
    }  
    />  
    )}}  
  </React.Fragment>  
  )}}  
</tbody>  
</table>  
</div>  
</div>  
);  
};  
````
```

```
Dynamic Column Header
```

```
```typescript
```

```
// components/DynamicGridHeader.tsx
```

```
interface DynamicGridHeaderProps {  
  columns: KPIColumn[];  
  tacticColumns: TacticColumn[];  
}
```

```
const DynamicGridHeader: React.FC<DynamicGridHeaderProps> = ({  
  columns,  
  tacticColumns  
}) => {  
  return (  
    <tr className="bg-gray-100 border-b-2 border-gray-300">
```

```

<th className="sticky left-0 bg-gray-100 p-2 text-left">Item</th>
{ /* Master Data columns */}
<th className="p-2 text-right">List Price</th>
<th className="p-2 text-right">COGS</th>
{ /* Volume columns (always visible) */}
<th className="p-2 text-right">Base Vol</th>
<th className="p-2 text-right">Planned Vol</th>
<th className="p-2 text-right">iVol</th>
<th className="p-2 text-right">Vol Uplift %</th>
{ /* Dynamic KPI columns */}
{columns.map(col => (
<th
key={col.kpiCode}
className="p-2 text-right"
title={col.description}
>
{col.displayName}
<InfoIcon tooltip={col.formula} />
</th>
)}}}
{ /* Dynamic Tactic columns (filtered by plan context) */}
{tacticColumns.map(tactic => (
<th
key={tactic.code}
className="p-2 text-right bg-blue-50"
title={tactic.description}
>
{tactic.name}
<span className="text-xs ml-1">
({tactic.inputType === 'percentage' ? '%' : '$'})
</span>
</th>
)}}}
{ /* ROI columns (always visible, fixed right) */}
<th className="sticky right-20 bg-gray-100 p-2 text-right">GP ROI %</th>
<th className="sticky right-0 bg-gray-100 p-2 text-center">RAG</th>
</tr>
);
};
```

```

### FU Row Component

```typescript

// components/FURow.tsx

```
interface FURowProps {  
  fu: FUData;  
  columns: KPIColumn[];  
  tacticColumns: TacticColumn[];  
  isExpanded: boolean;  
  onToggle: () => void;  
  onTacticChange: (tacticCode: string, value: number) => void;  
}
```

```
const FURow: React.FC<FURowProps> = ({  
  fu,  
  columns,  
  tacticColumns,  
  isExpanded,  
  onToggle,  
  onTacticChange  
}) => {  
  return (  
    <tr className="bg-blue-50 border-b hover:bg-blue-100">  
      /* Expand/Collapse + FU Name */  
      <td className="sticky left-0 bg-blue-50 p-2">  
        <button onClick={onToggle} className="mr-2">  
          {isExpanded ? '−' : '+'}  
        </button>  
        <span className="font-bold">{fu.fu_code} - {fu.fu_name}</span>  
      </td>  
  
      /* Master Data (read-only, inherited from SKUs) */  
      <td className="p-2 text-right text-gray-500">-</td>  
      <td className="p-2 text-right text-gray-500">-</td>  
  
      /* Volume (aggregated from SKUs, read-only) */  
      <td className="p-2 text-right">  
        {formatNumber(fu.calculated.BASE_VOL)}  
      </td>  
      <td className="p-2 text-right font-semibold">
```

```

</td>
<td className="p-2 text-right">
{formatNumber(fu.calculated.INCR_VOL)}
</td>
<td className="p-2 text-right">
{formatPercentage(fu.calculated.VOL_UPLIFT_PCT)}
</td>

{/* Dynamic KPI columns (calculated, read-only) */}
{columns.map(col => (
<td
key={col.kpiCode}
className="p-2 text-right"
title={` ${col.displayName}: ${col.formula}` }
>
<FormattedValue
value={fu.calculated[col.kpiCode]}
format={col.displayFormat}
decimals={col.decimalPlaces}
/>
</td>
))}

{/* Tactic input cells (EDITABLE at FU level) */}
{tacticColumns.map(tactic => (
<td key={tactic.code} className="p-2 bg-white">
<EditableCell
value={fu.tactics[tactic.code] || 0}
type={tactic.inputType}
min={tactic.minValue}
max={tactic.maxValue}
onChange={(value) => onTacticChange(tactic.code, value)}
placeholder={`Enter ${tactic.inputType}...`}
/>
</td>
))}

{/* ROI (calculated, with RAG) */}
<td className="sticky right-20 bg-blue-50 p-2 text-right">
<RAGCell

```

```

ragStatus={fu.calculated.RAG_STATUS}
/>
</td>
<td className="sticky right-0 bg-blue-50 p-2 text-center">
<RAGBadge status={fu.calculated.RAG_STATUS} />
</td>
</tr>
);
};
```

```

### SKU Row Component

```typescript

// components/SKURow.tsx

```

interface SKURowProps {
sku: SKUData;
fuTactics: Record<string, number>;
columns: KPIColumn[];
tacticColumns: TacticColumn[];
onVolumeChange: (value: number) => void;
}

```

```

const SKURow: React.FC<SKURowProps> = ({
sku,
fuTactics,
columns,
tacticColumns,
onVolumeChange
}) => {
return (
<tr className="border-b hover:bg-gray-50">
{/* Indented SKU Name */}
<td className="sticky left-0 bg-white p-2 pl-10">
<span className="text-gray-600">↳ {sku.sku_code} - {sku.sku_name}</span>
</td>

```

```

{/* Master Data (from master data, read-only) */}
<td className="p-2 text-right text-gray-700">
{formatCurrency(sku.list_price)}

```

```

<td className="p-2 text-right text-gray-700">
{formatCurrency(sku.cogs_per_piece)}
</td>

```

```

{/* Volume */}
<td className="p-2 text-right text-gray-600">
{formatNumber(sku.base_volume)}
</td>

```

```

<td className="p-2 bg-yellow-50">
{/* EDITABLE - User input */}
<EditableCell
value={sku.planned_volume}
type="number"
min={0}
onChange={onVolumeChange}
placeholder="Enter volume..."
className="font-semibold"
/>
</td>

```

```

<td className="p-2 text-right">
<AnimatedValue
value={sku.calculated.INCR_VOL}
format="number"
/>
</td>

```

```

<td className="p-2 text-right">
<AnimatedValue
value={sku.calculated.VOL_UPLIFT_PCT}
format="percentage"
/>
</td>

```

```

{/* Dynamic KPI columns (calculated in real-time) */}
{columns.map(col => (
<td
key={col.kpiCode}
className="p-2 text-right"
>
<AnimatedValue
value={sku.calculated[col.kpiCode]}

```



```

    decimals={col.decimalPlaces}
    highlightChange={true}
  />
</td>
))}

{/* Tactic cells (READ-ONLY, inherited from FU) */}
{tacticColumns.map(tactic => (
  <td key={tactic.code} className="p-2 text-right text-gray-500 bg-gray-50">
    {fuTactics[tactic.code]}
    ? formatTacticValue(fuTactics[tactic.code], tactic.inputType)
    : '-'
  <span className="text-xs ml-1">(inherited)</span>
  </td>
))}

{/* ROI with RAG */}
<td className="sticky right-20 bg-white p-2 text-right">
  <RAGCell
    value={sku.calculated.GP_ROI_PCT}
    ragStatus={sku.calculated.RAG_STATUS}
  />
</td>
<td className="sticky right-0 bg-white p-2 text-center">
  <RAGBadge status={sku.calculated.RAG_STATUS} />
</td>
</tr>
);
};
```

PHASE 7: EDITABLE CELL WITH REAL-TIME FEEDBACK
```typescript
// components/EditableCell.tsx

interface EditableCellProps {
  value: number;

```

```
min?: number;
max?: number;
onChange: (value: number) => void;
placeholder?: string;
className?: string;
}
```

```
const EditableCell: React.FC<EditableCellProps> = ({
  value,
  type,
  min,
  max,
  onChange,
  placeholder,
  className
}) => {
  const [localValue, setLocalValue] = useState(String(value));
  const [isEditing, setIsEditing] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const inputRef = useRef<HTMLInputElement>(null);
```

```
  useEffect(() => {
    if (!isEditing) {
      setLocalValue(String(value));
    }
  }, [value, isEditing]);
```

```
  const handleFocus = () => {
    setIsEditing(true);
    setTimeout(() => inputRef.current?.select(), 0);
  };
```

```
  const handleBlur = () => {
    setIsEditing(false);
    validateAndSave();
  };
```

```
  const handleKeyDown = (e: React.KeyboardEvent) => {
    if (e.key === 'Enter') {
      validateAndSave();
    }
  };
```

```

} else if (e.key === 'Escape') {
  setLocalValue(String(value));
  setError(null);
  inputRef.current?.blur();
}
};

const validateAndSave = () => {
  const numValue = parseFloat(localValue);

  // Validation
  if (isNaN(numValue)) {
    setError('Invalid number');
    return;
  }

  if (min !== undefined && numValue < min) {
    setError(`Minimum value is ${min}`);
    return;
  }

  if (max !== undefined && numValue > max) {
    setError(`Maximum value is ${max}`);
    return;
  }

  // Clear error and save
  setError(null);
  onChange(numValue);
};

const displayValue = isEditing
  ? localValue
  : type === 'percentage'
  ? `${value}%`
  : type === 'currency'
  ? `$$${value.toLocaleString()}`
  : value.toLocaleString();

return (

```

```

<input
  ref={inputRef}
  type="text"
  value={displayValue}
  onChange={(e) => setLocalValue(e.target.value)}
  onFocus={handleFocus}
  onBlur={handleBlur}
  onKeyDown={handleKeyDown}
  placeholder={placeholder}
  className={`
    w-full px-2 py-1 text-right border rounded
    focus:outline-none focus:ring-2 focus:ring-blue-500
    ${error ? 'border-red-500 bg-red-50' : 'border-gray-300'}
    ${className}
  `}
/>
{error && (
  <div className="absolute top-full left-0 mt-1 text-xs text-red-600
    whitespace-nowrap">
    ▲ {error}
  </div>
)}
</div>
);
};
```

```

---

## \*\*PHASE 8: ANIMATED VALUE COMPONENT\*\*

```typescript

// components/AnimatedValue.tsx

```

interface AnimatedValueProps {
  value: number | null;
  format: 'number' | 'currency' | 'percentage';
  decimals?: number;
  highlightChange?: boolean;
}

```

```

value,
format,
decimals = 2,
highlightChange = false
}) => {
const [displayValue, setDisplayValue] = useState(value);
const [isAnimating, setIsAnimating] = useState(false);
const prevValueRef = useRef(value);

useEffect(() => {
if (value !== prevValueRef.current && highlightChange) {
setIsAnimating(true);
// Animate number counting up/down
const start = prevValueRef.current || 0;
const end = value || 0;
const duration = 300; // ms
const steps = 20;
const increment = (end - start) / steps;
let currentStep = 0;

const interval = setInterval(() => {
currentStep++;
if (currentStep >= steps) {
setDisplayValue(end);
clearInterval(interval);
setTimeout(() => setIsAnimating(false), 500);
} else {
setDisplayValue(start + increment * currentStep);
}
}, duration / steps);

prevValueRef.current = value;

return () => clearInterval(interval);
} else {
setDisplayValue(value);
prevValueRef.current = value;
}
}, [value, highlightChange]);

```

```

return <span className="text-gray-400">-</span>;
}

const formatted = formatValue(displayValue, format, decimals);
const changeDirection =
(value || 0) > (prevValueRef.current || 0) ? 'up' : 'down';

return (
<span
  className={`
    transition-all duration-300
    ${isAnimating
      ? changeDirection === 'up'
      ? 'bg-green-200 text-green-800'
      : 'bg-red-200 text-red-800'
      : ''
    }
    ${isAnimating ? 'px-1 rounded' : ''}
  `}
>
  {formatted}
</span>
);
};

function formatValue(
  value: number,
  format: string,
  decimals: number
): string {
  switch (format) {
    case 'currency':
      return new Intl.NumberFormat('en-US', {
        style: 'currency',
        currency: 'USD',
        minimumFractionDigits: decimals,
        maximumFractionDigits: decimals
      }).format(value);
    case 'percentage':
      return `${value.toFixed(decimals)}%`;
  }
}

```

```

default:
return new Intl.NumberFormat('en-US', {
minimumFractionDigits: decimals,
maximumFractionDigits: decimals
}).format(value);
}
}
```

```

---

```
PHASE 9: GRAND TOTALS PANEL WITH REAL-TIME UPDATES
```

```
```typescript
```

```
// components/GrandTotalsPanel.tsx
```

```

interface GrandTotalsPanelProps {
totals: Record<string, any>;
}

```

```

const GrandTotalsPanel: React.FC<GrandTotalsPanelProps> = ({ totals })
=> {
return (
<div className="sticky top-0 z-10 bg-white border-b-4 border-blue-500
shadow-lg p-4 mb-4">
<h2 className="text-lg font-bold mb-3">PLAN PERFORMANCE OVERVIEW</h2>
<div className="grid grid-cols-6 gap-4">
{/* Volume Card */}
<MetricCard
icon="📦"
title="VOLUME"
mainValue={totals.PLANNED_VOL}
mainLabel="Planned Total"
format="number"
unit="pcs"
secondaryValue={totals.INCR_VOL}
secondaryLabel="Incremental"
tertiaryValue={totals.VOL_UPLIFT_PCT}
tertiaryLabel="vs base"
tertiaryFormat="percentage"
/>

```

```

{ /* Turnover Card */}
<MetricCard
icon="💰 "
title="TURNOVER"
mainValue={totals.PLANNED_T0}
mainLabel="Planned Total"
format="currency"
secondaryValue={totals.INCR_T0}
secondaryLabel="Incremental"
tertiaryValue={totals.T0_UPLIFT_PCT}
tertiaryLabel="vs base"
tertiaryFormat="percentage"
/>

{ /* Spend Card */}
<MetricCard
icon="📦 "
title="SPEND"
mainValue={totals.TOTAL_PLANNED_SPEND}
mainLabel="Total Planned"
format="currency"
secondaryValue={totals.INCR_SPEND}
secondaryLabel="Incremental"
breakdown={{
  'On-Invoice': totals.TOTAL_ON_INVOICE_SPEND,
  'Off-Invoice': totals.TOTAL_OFF_INVOICE_SPEND
}}
/>

{ /* Profit Card */}
<MetricCard
icon="💎 "
title="PROFIT"
mainValue={totals.INCR_GP}
mainLabel="Incremental GP"
format="currency"
secondaryValue={totals.BASE_GP}
secondaryLabel="Base GP"
tertiaryValue={totals.PLANNED_GP}
tertiaryLabel="Planned GP"

```



```

{ /* Efficiency Card */
<MetricCard
  icon="🇮🇹 "
  title="EFFICIENCY"
  mainValue={totals.GP_ROI_PCT}
  mainLabel="GP ROI"
  format="percentage"
  isHighlight={true}
  ragStatus={totals.RAG_STATUS}
  secondaryValue={totals.PLANNED_GM_PCT}
  secondaryLabel="GP Margin"
  secondaryFormat="percentage"
/>

{ /* Overall Status Card */
<StatusCard
  ragStatus={totals.RAG_STATUS}
  ragBreakdown={totals.RAG_BREAKDOWN}
  budgetImpact={totals.BUDGET_IMPACT_PCT}
/>
</div>
</div>
);
};

```

```

// Individual Metric Card
const MetricCard: React.FC<any> = ({
  icon,
  title,
  mainValue,
  mainLabel,
  format,
  unit,
  secondaryValue,
  secondaryLabel,
  tertiaryValue,
  tertiaryLabel,
  tertiaryFormat,
  isHighlight,

```

```

breakdown
}) => {
const ragColor = ragStatus === 'GREEN' ? 'bg-green-100' :
ragStatus === 'AMBER' ? 'bg-yellow-100' :
ragStatus === 'RED' ? 'bg-red-100' : 'bg-white';

return (
<div className={`border rounded-lg p-3 ${isHighlight ? ragColor : 'bg-
gray-50'}`>
<div className="flex items-center gap-2 mb-2">
<span className="text-2xl">{icon}</span>
<span className="font-bold text-sm text-gray-700">{title}</span>
</div>

<div className="mb-2">
<div className="text-xs text-gray-600">{mainLabel}</div>
<div className="text-2xl font-bold">
<AnimatedValue
value={mainValue}
format={format}
highlightChange={true}
/>
{unit} && <span className="text-sm ml-1">{unit}</span>
</div>
</div>

{secondaryValue !== undefined && (
<div className="text-sm">
<span className="text-gray-600">{secondaryLabel}: </span>
<span className="font-semibold">
<AnimatedValue value={secondaryValue} format={format} />
</span>
</div>
)}

{tertiaryValue !== undefined && (
<div className="text-sm">
<span className="text-gray-600">{tertiaryLabel}: </span>
<span className="font-semibold">
<AnimatedValue

```

```

format={tertiaryFormat || format}
/>
</span>
</div>
)}

```

```

{breakdown && (
<div className="mt-2 pt-2 border-t text-xs space-y-1">
{Object.entries(breakdown).map(([key, value]: [string, any]) => (
<div key={key} className="flex justify-between">
<span className="text-gray-600">{key}</span>
<span className="font-semibold">
<AnimatedValue value={value} format="currency" decimals={0} />
</span>
</div>
))}
</div>
)}
</div>
);
};
` ``

```

KEY IMPLEMENTATION PRINCIPLES

✅ **1. Formula-Driven System**

- ALL KPIs are defined in database with formulas
- NO hardcoded calculations in frontend code
- Admin can change formulas without code deployment

✅ **2. Real-Time Calculations**

- Calculations execute on every user input
- Uses dependency graph for optimal calculation order
- Only recalculates affected KPIs (not entire plan)

✅ **3. Dynamic Column Generation**

- Columns shown based on:
- Plan context (channel/category)

- KPI configuration (show_in_grid flag)
- User preferences

✅ **4. Proper Data Flow**

User Input (Volume/Tactic)

↓

Trigger Calculation Engine

↓

Parse Formula from Database

↓

Execute in Dependency Order

↓

Update Context with Results

↓

Cascade to Dependent KPIs

↓

Aggregate to FU/Plan Level

↓

Update UI with Animation

↓

Auto-Save (Debounce)

✅ **5. Performance Optimized**

- Web Workers for heavy calculations
- Incremental updates (not full recalc)
- Debounced auto-save
- Virtual scrolling for large plans
- Calculation result caching

This architecture ensures that **admins control everything through configuration** and **planners see real-time, accurate calculations** without any hardcoding! 🚀

Would you like me to provide:

1. **Complete TypeScript interfaces** for all data structures?
2. **Supabase database migration scripts**?
3. **Testing strategy** for formula validation?
4. **Error handling** for calculation failures?

Sequential Prompts for Building Dynamic TPM Calculation System

PROMPT 1: Database Schema for Dynamic KPI System

I'm building a Trade Promotion Management (TPM) system where admins can define KPIs with formulas that calculate automatically.

REQUIREMENTS:

- Admins define KPIs in a UI with formulas stored as text
- Formulas reference other KPIs by code (e.g., "PLANNED_VOL - BASE_VOL")
- System must calculate KPIs in correct dependency order
- Support different formula types: expression, conditional, javascript
- Track what each KPI depends on for cascade calculations
- Support aggregation methods (sum, avg, min, max) when rolling up SKU to FU level

TECH STACK:

- Supabase (PostgreSQL)
- Need Row Level Security (RLS) policies
- Use UUIDs for primary keys
- Use JSONB for array fields

CREATE THESE TABLES:

1. ****kpis**** - Main KPI definitions

Fields needed:

- id (UUID, primary key)
- kpi_code (VARCHAR(50), unique) - e.g., 'INCR_VOL', 'GP_ROI_PCT'
- kpi_name (VARCHAR(200)) - display name
- kpi_group (VARCHAR(100)) - e.g., 'Volume', 'Profit', 'ROI'
- formula_type (VARCHAR(50)) - 'expression', 'conditional', 'user_input', 'external'
- formula_text (TEXT) - the actual formula
- depends_on_kpis (JSONB) - array of KPI codes this depends on
- calculation_order (INTEGER) - for sorting calculation sequence
- calculation_level (VARCHAR(50)) - 'sku', 'fu', 'plan'
- display_format (VARCHAR(50)) - 'number', 'currency', 'percentage'
- decimal_places (INTEGER)
- show_in_grid (BOOLEAN)
- aggregation_method_fu (VARCHAR(20)) - 'sum', 'avg', 'min', 'max'
- is_active (BOOLEAN)

2. ****promo_mechanics**** - Tactic definitions (what planners can input)

Fields needed:

- id, mechanic_code, mechanic_name
- category ('on_invoice_discount', 'off_invoice_discount', 'per_unit_support', 'lumpsum')
- spending_type ('on_invoice', 'off_invoice')
- input_type ('percentage', 'currency', 'units')
- min_value, max_value, default_value
- calculation_formula (TEXT) - how to calculate spend from input
- applicable_channels (JSONB) - array of channels
- applicable_categories (JSONB) - array of categories
- show_in_grid (BOOLEAN)
- is_active (BOOLEAN)

3. ****plan_sku_volumes**** - Store user inputs and calculated values

- id, plan_id, sku_id
- base_volume (DECIMAL) - from historical data
- planned_volume (DECIMAL) - user input
- calculated_kpis (JSONB) - store all calculated KPI values
- updated_at

4. ****plan_mechanic_values**** - Store tactic inputs at FU level

- id, plan_fu_id, mechanic_id
- entered_value (DECIMAL) - user input
- calculated_spend (DECIMAL) - result of formula
- updated_at

ALSO PROVIDE:

1. ****Indexes**** for performance:

- Index on kpis.kpi_code
- Index on kpis.calculation_order
- Index on promo_mechanics.mechanic_code
- Index on plan_sku_volumes (plan_id, sku_id)

2. ****Sample INSERT statements**** for these KPIs:

LEVEL 1 (User Input - Order 1-10):

- PLANNED_VOL (user_input)

- BPTT (external - from master data)
- COGS (external - from master data)
- CPP_ON_PCT (user_input - tactic)

LEVEL 2 (Simple calculations - Order 11-20):

- $\text{INCR_VOL} = \text{PLANNED_VOL} - \text{BASE_VOL}$
- $\text{VOL_UPLIFT_PCT} = (\text{INCR_VOL} / \text{BASE_VOL}) * 100$
- $\text{BASE_GSV} = \text{BASE_VOL} * \text{BPTT}$
- $\text{PLANNED_GSV} = \text{PLANNED_VOL} * \text{BPTT}$
- $\text{INCR_GSV} = \text{PLANNED_GSV} - \text{BASE_GSV}$

LEVEL 3 (Dependent calculations - Order 21-30):

- $\text{CPP_ON_SPEND} = (\text{PLANNED_GSV} - \text{PLANNED_LTA_ON}) * (\text{CPP_ON_PCT} / 100)$
- $\text{TOTAL_PLANNED_SPEND} = \text{sum of all spends}$

LEVEL 4 (Profit calculations - Order 31-40):

- $\text{BASE_COGS} = \text{BASE_VOL} * \text{COGS}$
- $\text{PLANNED_COGS} = \text{PLANNED_VOL} * \text{COGS}$
- $\text{BASE_GP} = \text{BASE_TO} - \text{BASE_COGS}$
- $\text{PLANNED_GP} = \text{PLANNED_TO} - \text{PLANNED_COGS}$
- $\text{INCR_GP} = \text{PLANNED_GP} - \text{BASE_GP}$

LEVEL 5 (ROI - Order 41-50):

- $\text{GP_ROI_PCT} = (\text{INCR_GP} / \text{INCR_SPEND}) * 100$
- $\text{RAG_STATUS (conditional)} = \text{IF}(\text{GP_ROI_PCT} \geq 20, 'GREEN', \text{IF}(\text{GP_ROI_PCT} \geq 10, 'AMBER', 'RED'))$

3. ****Sample promo_mechanics**** records:

- CPP On-Invoice % (percentage, 0-100, applies to all channels/categories)
- Visibility Lumpsum MT/PH (currency, \$0-\$50000, applies to NKA channel only)
- Price Support per Unit (currency, \$0-\$5, applies to all)

FORMAT: Provide complete SQL migration file ready to run in Supabase SQL editor.

PROMPT 2: Formula Parser Service

Create a TypeScript service that parses formula strings and executes them safely.

CONTEXT:

I'm building a dynamic KPI system where formulas are stored as text in the database like:

- "PLANNED_VOL - BASE_VOL"
- "(INCR_GP / INCR_SPEND) * 100"
- "IF(GP_ROI_PCT >= 20, 'GREEN', IF(GP_ROI_PCT >= 10, 'AMBER', 'RED'))"

REQUIREMENTS:

1. ****Extract Dependencies**** - Automatically find variable names in formula
 - Input: "(PLANNED_VOL - BASE_VOL) / BASE_VOL * 100"
 - Output: ["PLANNED_VOL", "BASE_VOL"]
 - Rules: Variable names are UPPERCASE with underscores, exclude function names like IF, SUM, AVG
2. ****Parse Expression Formulas**** - Simple math operations
 - Support: +, -, *, /, (), parentheses
 - Replace variable names with actual values from context
 - Example: "PLANNED_VOL - BASE_VOL" with {PLANNED_VOL: 4200, BASE_VOL: 3200} returns 1000
3. ****Parse Conditional Formulas**** - IF statements
 - Syntax: IF(condition, true_value, false_value)
 - Support nested IFs
 - Support comparisons: >=, <=, >, <, ==, !=
 - Example: "IF(GP_ROI_PCT >= 20, 'GREEN', 'RED')"
4. ****Safe Evaluation**** - No arbitrary code execution
 - Use Function constructor with whitelist
 - Only allow: numbers, basic operators, parentheses
 - Block: eval(), require(), import, any dangerous operations
5. ****Error Handling****
 - Return null if dependency is missing
 - Return null if division by zero
 - Log errors but don't throw (calculation continues for other KPIs)

INTERFACE:

```typescript



```

type: 'expression' | 'conditional' | 'javascript';
dependencies: string[];
execute: (context: Record<string, any>) => any;
raw: string;
}

```

```

interface ValidationResult {
 isValid: boolean;
 errors: string[];
 warnings: string[];
}

```

```

class FormulaParser {
 parseFormula(formula: string, formulaType: string): ParsedFormula;
 extractDependencies(formula: string): string[];
 validateFormula(formula: string, formulaType: string): ValidationResult;
 private safeEval(expression: string): number | null;
 private parseExpression(formula: string, dependencies: string[]):
 ParsedFormula;
 private parseConditional(formula: string, dependencies: string[]):
 ParsedFormula;
}
...

```

PROVIDE:

1. **\*\*Complete FormulaParser class\*\*** with all methods implemented
2. **\*\*Unit tests\*\*** for these cases:
  - Simple math: "10 + 5" → 15
  - Variables: "PLANNED\_VOL - BASE\_VOL" with context
  - Percentage: "(INCR\_VOL / BASE\_VOL) \* 100"
  - Division by zero: "(100 / 0)" → null
  - Missing dependency: "PLANNED\_VOL - BASE\_VOL" with only PLANNED\_VOL → null
  - Conditional: "IF(25 >= 20, 'GREEN', 'RED')" → 'GREEN'
  - Nested conditional: "IF(15 >= 20, 'GREEN', IF(15 >= 10, 'AMBER', 'RED'))" → 'AMBER'
3. **\*\*Examples\*\*** of how to use it:
 

```
```typescript
```

```
// Example 1: Simple expression
const formula1 = parser.parseFormula("PLANNED_VOL - BASE_VOL",
"expression");
const result1 = formula1.execute({ PLANNED_VOL: 4200, BASE_VOL: 3200 });
console.log(result1); // 1000

// Example 2: Conditional
const formula2 = parser.parseFormula(
"IF(GP_ROI_PCT >= 20, 'GREEN', 'RED')",
"conditional"
);
const result2 = formula2.execute({ GP_ROI_PCT: 25 });
console.log(result2); // 'GREEN'
```

```

#### CONSTRAINTS:

- Pure TypeScript, no external libraries for formula parsing
- Must work in browser (client-side calculation)
- Performance: Parse 100 formulas in < 100ms

#### PROMPT 3: Calculation Dependency Graph

Create a calculation dependency graph system that determines the correct order to calculate KPIs.

#### CONTEXT:

I have KPIs with formulas that depend on each other:

- INCR\_VOL depends on [PLANNED\_VOL, BASE\_VOL]
- VOL\_UPLIFT\_PCT depends on [INCR\_VOL, BASE\_VOL]
- GP\_ROI\_PCT depends on [INCR\_GP, INCR\_SPEND]
- INCR\_GP depends on [PLANNED\_GP, BASE\_GP]

Need to calculate in correct order: dependencies first, dependents after.

#### REQUIREMENTS:

##### 1. **\*\*Build Dependency Graph\*\***

- Input: Array of KPI definitions from database
- Each KPI has: kpi\_code, depends\_on\_kpis (array)

## 2. **\*\*Topological Sort\*\***

- Determine calculation order
- Ensure dependencies are calculated before dependents
- Detect circular dependencies (throw error if found)

## 3. **\*\*Level Calculation\*\***

- Assign each KPI a "level" (how many steps from user input)
- Level 0: User inputs (no dependencies)
- Level 1: Direct calculations from inputs
- Level 2: Calculations using Level 1 results, etc.

## 4. **\*\*Incremental Updates\*\***

- When one KPI changes, identify which KPIs need recalculation
- Return only affected KPIs in dependency order
- Don't recalculate unaffected KPIs

INTERFACE:

```
```typescript
interface KPINode {
  kpiCode: string;
  dependencies: string[]; // KPIs this depends on
  dependents: string[]; // KPIs that depend on this
  level: number; // Calculation order level
}
```

```
class CalculationGraph {
  buildGraph(kpis: any[]): Promise<void>;
  getCalculationOrder(): string[];
  getAffectedKPIs(changedKPI: string): string[];
  getNode(kpiCode: string): KPINode | undefined;
  private calculateLevels(): void;
  private topologicalSort(): string[];
}
```
```

EXAMPLE USAGE:

```
```typescript
const graph = new CalculationGraph();
```

```

const kpis = [
  { kpi_code: 'PLANNED_VOL', depends_on_kpis: [] },
  { kpi_code: 'BASE_VOL', depends_on_kpis: [] },
  { kpi_code: 'INCR_VOL', depends_on_kpis: ['PLANNED_VOL', 'BASE_VOL'] },
  { kpi_code: 'VOL_UPLIFT_PCT', depends_on_kpis: ['INCR_VOL', 'BASE_VOL'] },
  { kpi_code: 'BASE_GSV', depends_on_kpis: ['BASE_VOL', 'BPTT'] },
  { kpi_code: 'PLANNED_GSV', depends_on_kpis: ['PLANNED_VOL', 'BPTT'] },
  { kpi_code: 'INCR_GSV', depends_on_kpis: ['PLANNED_GSV', 'BASE_GSV'] }
];

await graph.buildGraph(kpis);

// Get full calculation order
const order = graph.getCalculationOrder();
console.log(order);
// Expected: ['PLANNED_VOL', 'BASE_VOL', 'BPTT', 'INCR_VOL', 'BASE_GSV',
// 'PLANNED_GSV', 'VOL_UPLIFT_PCT', 'INCR_GSV']

// If PLANNED_VOL changes, what needs recalculation?
const affected = graph.getAffectedKPIs('PLANNED_VOL');
console.log(affected);
// Expected: ['INCR_VOL', 'PLANNED_GSV', 'VOL_UPLIFT_PCT', 'INCR_GSV']
// (in correct calculation order)
...

```

PROVIDE:

1. ****Complete CalculationGraph class****
2. ****Handle edge cases:****
 - Circular dependency detection: $A \rightarrow B \rightarrow C \rightarrow A$ should throw error
 - Orphaned KPIs: KPIs with no dependencies or dependents
 - Multiple independent branches
3. ****Unit tests**** for:
 - Simple chain: $A \rightarrow B \rightarrow C$
 - Diamond: $A \rightarrow B, C \rightarrow D$ (both B and C depend on A, D depends on both)
 - Circular: $A \rightarrow B \rightarrow C \rightarrow A$ (should throw error)
 - Incremental update: changing B should only affect B's dependents

- Log graph structure when built
- Log calculation order
- Log affected KPIs when getting incremental updates

USE:

- TypeScript
- Use the FormulaParser from previous prompt for extracting dependencies
- No external graph libraries (implement yourself)

PROMPT 4: Calculation Engine Core

Create the main calculation engine that executes formulas and manages KPI calculations.

CONTEXT:

I have:

1. FormulaParser that can parse and execute formula strings
2. CalculationGraph that determines calculation order
3. Database with KPI definitions

Now need the engine that brings it all together.

REQUIREMENTS:

1. ****Initialize from Database****
 - Load all active KPIs from Supabase 'kpis' table
 - Build calculation graph
 - Cache parsed formulas for performance
2. ****Calculate Single SKU****
 - Input: Context with user inputs and master data
 - Execute all formulas in dependency order
 - Return all calculated KPI values
3. ****Incremental Recalculation****
 - When one value changes, only recalculate affected KPIs
 - Much faster than full recalculation
4. ****Aggregate to FU Level****
 - Take multiple SKU results
 - Aggregate using method from KPI config (sum, avg, min, max)

5. ****Format Values for Display****

- Format based on KPI display_format (number, currency, percentage)
- Apply decimal_places setting

INTERFACE:

```typescript

```
interface CalculationContext {
 // User inputs
 planned_volume?: number;
 tactic_values?: Record<string, number>; // e.g., { CPP_ON_PCT: 10 }
 // Master data
 list_price?: number;
 cogs_per_piece?: number;
 base_volume?: number;
 // Pre-calculated values
 calculated?: Record<string, any>;
}
```

```
interface CalculationResult {
 kpiCode: string;
 value: any;
 formatted: string;
 error?: string;
}
```

```
class CalculationEngine {
 initialize(): Promise<void>;
 calculateSKU(context: CalculationContext): Record<string,
 CalculationResult>;
 recalculate(
 changedKPI: string,
 newValue: any,
 currentContext: Record<string, any>
): Record<string, CalculationResult>;
 aggregateFU(skuResults: Record<string, any>[]): Record<string, any>;
 private formatValue(value: any, kpi: any): string;
}
```
```

DEPENDENCIES:

```

```typescript
import { formulaParser } from './formulaParser';
import { calculationGraph } from './calculationGraph';
import { supabase } from './supabaseClient';
```

```

EXAMPLE USAGE:

```

```typescript
const engine = new CalculationEngine();
await engine.initialize();

// Calculate all KPIs for one SKU
const context = {
 planned_volume: 4200,
 base_volume: 3200,
 list_price: 4.00,
 cogs_per_piece: 1.80,
 tactic_values: {
 CPP_ON_PCT: 10,
 CPP_OFF_PCT: 5
 }
};

const results = engine.calculateSKU(context);

console.log(results);
/* Expected output:
{
 INCR_VOL: { kpiCode: 'INCR_VOL', value: 1000, formatted: '1,000' },
 VOL_UPLIFT_PCT: { kpiCode: 'VOL_UPLIFT_PCT', value: 31.25, formatted:
 '31.25%' },
 BASE_GSV: { kpiCode: 'BASE_GSV', value: 12800, formatted: '$12,800.00'
 },
 PLANNED_GSV: { kpiCode: 'PLANNED_GSV', value: 16800, formatted:
 '$16,800.00' },
 ... all other calculated KPIs
}
*/

// Incremental update when planned volume changes

```

```

 'PLANNED_VOL',
 5000, // new value
 results // current state
);

 console.log(updatedResults);
 // Only contains recalculated KPIs that depend on PLANNED_VOL

 // Aggregate multiple SKUs to FU level
 const sku1Results = { INCR_VOL: { value: 1000 }, PLANNED_GP: { value: 2500 } };
 const sku2Results = { INCR_VOL: { value: 750 }, PLANNED_GP: { value: 1800 } };
 const sku3Results = { INCR_VOL: { value: 500 }, PLANNED_GP: { value: 1200 } };

 const fuResults = engine.aggregateFU([sku1Results, sku2Results, sku3Results]);

 console.log(fuResults);
 /* Expected output:
 {
 INCR_VOL: 2250, // sum
 PLANNED_GP: 5500 // sum
 }
 */
 ...

```

PROVIDE:

1. **\*\*Complete CalculationEngine class\*\***
2. **\*\*Error handling:\*\***
  - Handle missing dependencies gracefully (return null, don't crash)
  - Handle division by zero
  - Log calculation errors but continue with other KPIs
3. **\*\*Performance optimization:\*\***
  - Cache parsed formulas (don't re-parse on every calculation)
  - Cache KPI definitions
  - Only recalculate affected KPIs



#### 4. **\*\*Formatting examples:\*\***

- Currency: 12500.50 → "\$12,500.50"
- Percentage: 25.5 → "25.50%" (respect decimal\_places)
- Number: 1000 → "1,000" (thousand separators)

#### 5. **\*\*Integration with Supabase:\*\***

```
```typescript
async initialize() {
  const { data: kpis, error } = await supabase
    .from('kpis')
    .select('*')
    .eq('is_active', true)
    .order('calculation_order');
  // Build graph and cache
}
```
```

#### USE:

- TypeScript with strict types
- FormulaParser and CalculationGraph from previous prompts
- Supabase client
- console.log for debugging key steps

#### PROMPT 5: React Hook for Plan Calculations

Create a React hook that manages plan calculations with real-time updates.

#### CONTEXT:

I have a calculation engine that can calculate KPIs. Now need React integration for the planning grid where:

- Planners enter planned volumes at SKU level
- Planners enter tactic values (discounts, fees) at FU level
- All KPIs recalculate automatically
- Results aggregate to FU and Plan levels
- Auto-save to Supabase after 2 seconds of no changes

#### REQUIREMENTS:

##### 1. **\*\*State Management\*\***

- Store FUs (each containing multiple SKUs)

- Store calculated values for each FU (aggregated)
- Store grand totals (plan level aggregated)

## 2. **\*\*Update Functions\*\***

- updateSKUVolume(fuId, skuId, newVolume) - triggers SKU recalculation
- updateFUTactic(fuId, tacticCode, newValue) - triggers all SKU recalculations in that FU

## 3. **\*\*Automatic Aggregation\*\***

- When SKU values change → aggregate to FU level
- When FU values change → aggregate to Plan level

## 4. **\*\*Debounced Auto-Save\*\***

- Save to Supabase 2 seconds after last change
- Save to plan\_sku\_volumes and plan\_mechanic\_values tables

## 5. **\*\*Real-Time Feedback\*\***

- isCalculating flag to show loading state
- Optimistic UI updates (update state immediately, save async)

INTERFACE:

```
```typescript
```

```
interface SKUData {
  sku_id: string;
  sku_code: string;
  sku_name: string;
  planned_volume: number;
  base_volume: number;
  list_price: number;
  cogs_per_piece: number;
  calculated: Record<string, any>; // All calculated KPI values
}
```

```
interface FUData {
  fu_id: string;
  fu_code: string;
  fu_name: string;
  tactics: Record<string, number>; // e.g., { CPP_ON_PCT: 10,
  VISIBILITY_MTPH: 2000 }
  skus: SKUData[];
```

```
}
```

```
function usePlanCalculations(planId: string) {  
  // Returns:  
  return {  
    fus: FUData[],  
    grandTotals: Record<string, any>,  
    isCalculating: boolean,  
    updateSKUVolume: (fuId: string, skuId: string, newVolume: number) =>  
      void,  
    updateFUTactic: (fuId: string, tacticCode: string, newValue: number) =>  
      void,  
    addFU: (fuId: string) => Promise<void>,  
    removeFU: (fuId: string) => void  
  };  
}
```

DEPENDENCIES:

```
```typescript  
import { useState, useEffect, useCallback } from 'react';
import { calculationEngine } from '../services/calculationEngine';
import { supabase } from '../lib/supabaseClient';
import { debounce } from 'lodash'; // or implement your own
```
```

EXAMPLE USAGE IN COMPONENT:

```
```typescript  
function PlanningGrid({ planId }) {
 const {
 fus,
 grandTotals,
 isCalculating,
 updateSKUVolume,
 updateFUTactic
 } = usePlanCalculations(planId);

 return (
 <div>
 { /* Grand Totals */ }
 </div>
);
}
```

```
<h3>Plan Total GP ROI: {grandTotals.GP_ROI_PCT}%</h3>
<h3>Plan Total Spend: ${grandTotals.TOTAL_PLANNED_SPEND}</h3>
</div>
```

```
{/* Grid */}
{fus.map(fu => (
<div key={fu.fu_id}>
<h4>{fu.fu_name}</h4>
{/* FU-level tactic inputs */}
<input
type="number"
value={fu.tactics.CPP_ON_PCT || 0}
onChange={(e) => updateFUTactic(fu.fu_id, 'CPP_ON_PCT',
parseFloat(e.target.value))}
placeholder="CPP On-Invoice %"
/>
```

```
{/* SKUs */}
{fu.skus.map(sku => (
<div key={sku.sku_id}>
{sku.sku_name}
{/* SKU-level volume input */}
<input
type="number"
value={sku.planned_volume}
onChange={(e) => updateSKUVolume(fu.fu_id, sku.sku_id,
parseFloat(e.target.value))}
placeholder="Planned Volume"
/>
```

```
{/* Calculated values */}
iVol: {sku.calculated.INCR_VOL}
GP ROI: {sku.calculated.GP_ROI_PCT}%
</div>
))}
</div>
))}
```

```
{isCalculating && <div>Calculating...</div>}
</div>
```

```
}
```
```

PROVIDE:

1. ****Complete usePlanCalculations hook****

2. ****Load initial data from Supabase:****

```
```typescript
```

```
// On mount, load:
```

```
- Plan FUs from plan_fus table
```

```
- SKUs for each FU from fus and fu_skus tables
```

```
- Master data (list_price, cogs) from skus table
```

```
- Historical base volumes from plan_sku_volumes or historical_volumes
table
```

```
- Current planned volumes and tactics from plan_sku_volumes and
plan_mechanic_values
```

```
```
```

3. ****Update flow diagram in comments:****

```
```typescript
```

```
// Update Flow:
```

```
// 1. User changes SKU planned volume
```

```
// 2. updateSKUVolume called
```

```
// 3. Calculate new SKU KPIs using engine.recalculate()
```

```
// 4. Update SKU state with new calculated values
```

```
// 5. Aggregate all SKUs to FU level
```

```
// 6. Aggregate all FUs to Plan level
```

```
// 7. Update grandTotals state
```

```
// 8. Trigger debounced save after 2 seconds
```

```
```
```

4. ****Save function:****

```
```typescript
```

```
async function savePlanToDatabase(fusData: FUData[]): Promise<void> {
```

```
// Batch upsert to:
```

```
// - plan_sku_volumes (sku_id, planned_volume, calculated_values JSONB)
```

```
// - plan_mechanic_values (plan_fu_id, mechanic_code, value)
```

```
}
```

```
```
```

- Show toast notification on save error
- Retry failed saves
- Don't block UI if save fails

USE:

- React 18+ with hooks
- TypeScript
- CalculationEngine from previous prompt
- Supabase client
- lodash debounce (or custom implementation)

PROMPT 6: Dynamic Column Configuration Hook

Create a React hook that dynamically determines which columns to show in the planning grid based on plan context.

CONTEXT:

The planning grid should only show:

1. KPIs that are configured to display (show_in_grid = true)
2. Tactics that apply to the plan's channel and category

Example:

- Plan is for "NKA" channel and "Dairy" category
- Only show tactics where applicable_channels includes "NKA" OR "ALL"
- Only show tactics where applicable_categories includes "Dairy" OR "ALL"

REQUIREMENTS:

1. ****Fetch KPI Columns****

- Load from 'kpis' table where show_in_grid = true
- Order by calculation_order or custom display_order
- Return column configuration for grid

2. ****Fetch Tactic Columns****

- Load from 'promo_mechanics' table
- Filter by plan context (channel, category)
- Only show applicable tactics

3. ****Group Columns****

- Group by kpi_group (Volume, Profit, ROI, etc.)

4. ****Column Metadata****

- Include display name, format, width, tooltip

INTERFACE:

```typescript

```
interface KPIColumn {
```

```
kpiCode: string;
```

```
kpiName: string;
```

```
displayName: string;
```

```
kpiGroup: string;
```

```
displayFormat: 'number' | 'currency' | 'percentage';
```

```
decimalPlaces: number;
```

```
width: number; // pixels
```

```
formula: string; // for tooltip
```

```
description: string;
```

```
isEditable: boolean; // false for calculated, true for user input
```

```
editableAt: 'sku' | 'fu' | null; // where it can be edited
```

```
}
```

```
interface TacticColumn {
```

```
code: string;
```

```
name: string;
```

```
inputType: 'percentage' | 'currency';
```

```
minValue: number;
```

```
maxValue: number;
```

```
width: number;
```

```
description: string;
```

```
category: string; // 'on_invoice_discount', 'off_invoice_discount', etc.
```

```
}
```

```
interface ColumnGroups {
```

```
[groupName: string]: KPIColumn[];
```

```
}
```

```
function useKPIColumns(planContext: { channel: string; category: string
}) {
```

```
 return {
```

```
 columns: KPIColumn[],
```

```
 tacticColumns: TacticColumn[],
```

```
isLoading: boolean,
error: Error | null
};
}
````
```

EXAMPLE USAGE:

```
````typescript  
function PlanningGrid({ planId, planContext }) {
 const {
 columns,
 tacticColumns,
 columnGroups,
 isLoading
 } = useKPIColumns(planContext);

 if (isLoading) return <div>Loading columns...</div>;

 return (
 <table>
 <thead>
 <tr>
 { /* Static columns */ }
 <th>Item Name</th>
 <th>Base Vol</th>
 <th>Planned Vol</th>

 { /* Dynamic KPI columns */ }
 { columns.map(col => (
 <th key={col.kpiCode} style={{ width: col.width }}>
 {col.displayName}
 <Tooltip text={col.description} />
 </th>
)) }

 { /* Dynamic tactic columns */ }
 { tacticColumns.map(tactic => (
 <th key={tactic.code} style={{ width: tactic.width }}>
 {tactic.name}
 ({tactic.inputType === 'percentage' ? '%' : '$'})
 </th>
)) }
 </tr>
 </thead>
 </table>
)
```



```
))}
```

```
{/* Always show ROI */}
```

```
<th>GP ROI %</th>
```

```
<th>RAG</th>
```

```
</tr>
```

```
</thead>
```

```
<tbody>
```

```
{/* Grid rows */}
```

```
</tbody>
```

```
</table>
```

```
);
```

```
}
```

```
// Usage with context
```

```
<PlanningGrid
```

```
planId="plan-1232 / 2SContinue"
```

```
planContext={{ channel: 'NKA', category: 'Dairy' }}
```

```
/>
```

```
// Will only show tactics applicable to NKA channel and Dairy category
```

FILTERING LOGIC EXAMPLE:

```
```typescript
```

```
// Plan context
```

```
const planContext = { channel: 'NKA', category: 'Dairy' };
```

```
// Tactic in database
```

```
const tactic1 = {
```

```
code: 'CPP_ON_PCT',
```

```
name: 'CPP On-Invoice %',
```

```
applicable_channels: ['NKA', 'Traditional Trade', 'E-Commerce'],
```

```
applicable_categories: ['ALL']
```

```
};
```

```
// Result: SHOW (NKA is in applicable_channels)
```

```
const tactic2 = {
```

```
code: 'WS_TPR_ON_PCT',
```

```
name: 'WS TPR On-Invoice %',
```

```
applicable_channels: ['Wholesale'],
```

```
applicable_categories: ['ALL']
```

```
// Result: HIDE (Wholesale not in applicable_channels, plan is NKA)
```

```
const tactic3 = {  
  code: 'VISIBILITY_MTPH',  
  name: 'Visibility Lumpsum MT/PH',  
  applicable_channels: ['NKA'],  
  applicable_categories: ['Dairy', 'Beverages']  
};
```

```
// Result: SHOW (Both NKA and Dairy match)
```

```
const tactic4 = {  
  code: 'EXPORT_FEE',  
  name: 'Export Handling Fee',  
  applicable_channels: ['Export'],  
  applicable_categories: ['ALL']  
};  
// Result: HIDE (Export not in applicable_channels)  
````
```

PROVIDE:

1. **\*\*Complete useKPIColumns hook\*\*** with:
  - Supabase queries to fetch KPIs and tactics
  - Filtering logic for tactics based on plan context
  - Grouping logic for column groups
  - Loading and error states

2. **\*\*Supabase query examples:\*\***

```
````typescript  
// Fetch KPI columns  
const { data: kpis } = await supabase  
  .from('kpis')  
  .select('*')  
  .eq('is_active', true)  
  .eq('show_in_grid', true)  
  .order('calculation_order');  
  
// Fetch tactic columns with filtering  
const { data: tactics } = await supabase  
  .from('promo_mechanics')
```

```

.eq('is_active', true)
.eq('show_in_grid', true);

// Then filter in code:
const filteredTactics = tactics.filter(tactic => {
const channelMatch =
tactic.applicable_channels.includes('ALL') ||
tactic.applicable_channels.includes(planContext.channel);
const categoryMatch =
tactic.applicable_categories.includes('ALL') ||
tactic.applicable_categories.includes(planContext.category);
return channelMatch && categoryMatch;
});
```

```

### 3. **\*\*Default column configuration:\*\***

```

```typescript
// Always visible columns (cannot be hidden)
const FIXED_COLUMNS = [
{ code: 'item_name', name: 'Item Name', width: 250, fixed: 'left' },
{ code: 'base_vol', name: 'Base Vol', width: 100 },
{ code: 'planned_vol', name: 'Planned Vol', width: 120, editable: true
},
];

const FIXED_RIGHT_COLUMNS = [
{ code: 'gp_roi_pct', name: 'GP ROI %', width: 120, fixed: 'right' },
{ code: 'rag_status', name: 'RAG', width: 100, fixed: 'right' }
];
```

```

### 4. **\*\*Column grouping for headers:\*\***

```

```typescript
// Return structure like:
const columnGroups = {
'Volume': [
{ kpiCode: 'BASE_VOL', displayName: 'Base Vol', ... },
{ kpiCode: 'PLANNED_VOL', displayName: 'Planned Vol', ... },
{ kpiCode: 'INCR_VOL', displayName: 'iVol', ... }
],

```

```

{ kpiCode: 'BASE_GP', displayName: 'Base GP', ... },
{ kpiCode: 'PLANNED_GP', displayName: 'Planned GP', ... }
],
'ROI': [
{ kpiCode: 'GP_ROI_PCT', displayName: 'GP ROI %', ... }
]
};
```

```

## 5. \*\*User preference support (bonus):\*\*

```

```typescript
// Save user's column visibility preferences
const saveColumnPreferences = async (userId: string, preferences: any)
=> {
  await supabase
    .from('user_preferences')
    .upsert({
      user_id: userId,
      preference_type: 'column_visibility',
      preferences: preferences
    });
};

// Load and apply preferences
const loadColumnPreferences = async (userId: string) => {
  const { data } = await supabase
    .from('user_preferences')
    .select('preferences')
    .eq('user_id', userId)
    .eq('preference_type', 'column_visibility')
    .single();
  return data?.preferences || {};
};
```

```

## USE:

- React hooks (useState, useEffect)
- TypeScript with strict types
- Supabase client
- Handle loading states with skeleton UI

## PROMPT 7: Planning Grid Components

Create the main React components for the planning grid with dynamic columns and real-time calculations.

### CONTEXT:

I have:

- usePlanCalculations hook (manages data and calculations)
- useKPIColumns hook (provides column configuration)
- Need to build the actual grid UI with FU/SKU hierarchy

### REQUIREMENTS:

1. **PlanningGrid Component** - Main container
  - Sticky header with column names
  - Horizontal and vertical scroll
  - Fixed left columns (item name)
  - Fixed right columns (ROI, RAG)
2. **FURow Component** - Parent row (blue background)
  - Expand/collapse icon
  - Shows aggregated values (read-only)
  - Tactic inputs (editable at FU level)
  - Bold styling
3. **SKURow Component** - Child row (white background)
  - Indented with arrow (↳)
  - Volume input (editable at SKU level)
  - Tactic values inherited from FU (read-only, gray)
  - Calculated KPI values
4. **EditableCell Component** - Inline editing
  - Click to edit
  - Real-time validation
  - Auto-save on blur
  - Keyboard shortcuts (Enter, Escape, Tab)
5. **DynamicGridHeader** - Column headers
  - Group headers (Volume, Profit, ROI)
  - Individual column headers
  - Tooltips with formula info

COMPONENT STRUCTURE:

```
```typescript
```

```
// PlanningGrid.tsx
```

```
interface PlanningGridProps {
```

```
  planId: string;
```

```
  planContext: { channel: string; category: string };
```

```
}
```

```
export function PlanningGrid({ planId, planContext }: PlanningGridProps)
```

```
{
```

```
  const {
```

```
    fus,
```

```
    grandTotals,
```

```
    isCalculating,
```

```
    updateSKUVolume,
```

```
    updateFUTactic
```

```
  } = usePlanCalculations(planId);
```

```
  const { columns, tacticColumns } = useKPIColumns(planContext);
```

```
  const [expandedFUs, setExpandedFUs] = useState<Set<string>>(new Set());
```

```
  return (
```

```
    <div className="planning-grid-container">
```

```
      {/* Grand Totals Panel - separate component */}
```

```
      <GrandTotalsPanel totals={grandTotals} />
```

```
      {/* Toolbar */}
```

```
      <div className="toolbar">
```

```
        <button>+ Add FU</button>
```

```
        <button>Expand All</button>
```

```
        <button>Collapse All</button>
```

```
        <button>Export</button>
```

```
        <button onClick={saveManually}>Save</button>
```

```
      </div>
```

```
      {/* Grid */}
```

```
      <div className="grid-scroll-container">
```

```
        <table className="planning-grid-table">
```

```

columns={columns}
tacticColumns={tacticColumns}
/>
<tbody>
{fus.map(fu => (
<React.Fragment key={fu.fu_id}>
<FURow
fu={fu}
columns={columns}
tacticColumns={tacticColumns}
isExpanded={expandedFUs.has(fu.fu_id)}
onToggle={() => toggleFU(fu.fu_id)}
onTacticChange={(tacticCode, value) =>
updateFUTactic(fu.fu_id, tacticCode, value)
}
/>

{expandedFUs.has(fu.fu_id) && fu.skus.map(sku => (
<SKURow
key={sku.sku_id}
sku={sku}
fuTactics={fu.tactics}
columns={columns}
tacticColumns={tacticColumns}
onVolumeChange={(value) =>
updateSKUVolume(fu.fu_id, sku.sku_id, value)
}
/>
))}
</React.Fragment>
))}
</tbody>
</table>
</div>

{isCalculating && (
<div className="calculating-overlay">
<Spinner />
Calculating...
</div>

```

```

</div>
);
}
...
```typescript
// FURow.tsx
interface FURowProps {
 fu: FUData;
 columns: KPIColumn[];
 tacticColumns: TacticColumn[];
 isExpanded: boolean;
 onToggle: () => void;
 onTacticChange: (tacticCode: string, value: number) => void;
}

export function FURow({
 fu,
 columns,
 tacticColumns,
 isExpanded,
 onToggle,
 onTacticChange
}: FURowProps) {
 return (
 <tr className="fu-row bg-blue-50 font-bold">
 { /* Expand/Collapse + Name */ }
 <td className="sticky left-0 bg-blue-50">
 <button onClick={onToggle}>
 {isExpanded ? '−' : '+'}
 </button>
 {fu.fu_code} - {fu.fu_name}
 </td>

 { /* Master Data (read-only, from SKUs) */ }
 <td className="text-gray-400">-</td>
 <td className="text-gray-400">-</td>

 { /* Volume columns (aggregated, read-only) */ }
 <td className="text-right">{formatNumber(fu.calculated.BASE_VOL)}</td>

```



```

 { /* Dynamic KPI columns (calculated) */
 {columns.map(col => (
 <td key={col.kpiCode} className="text-right">
 <FormattedValue
 value={fu.calculated[col.kpiCode]}
 format={col.displayFormat}
 decimals={col.decimalPlaces}
 />
 </td>
)))}

 { /* Tactic columns (EDITABLE at FU level) */
 {tacticColumns.map(tactic => (
 <td key={tactic.code} className="bg-white">
 <EditableCell
 value={fu.tactics[tactic.code] || 0}
 type={tactic.inputType}
 min={tactic.minValue}
 max={tactic.maxValue}
 onChange={(value) => onTacticChange(tactic.code, value)}
 />
 </td>
)))}

 { /* ROI with RAG */
 <td className="sticky right-20 bg-blue-50 text-right">
 <RAGCell
 value={fu.calculated.GP_ROI_PCT}
 ragStatus={fu.calculated.RAG_STATUS}
 />
 </td>
 <td className="sticky right-0 bg-blue-50">
 <RAGBadge status={fu.calculated.RAG_STATUS} />
 </td>
 </tr>
);
 }
 ...
    ```typescript

```

```

interface SKURowProps {
  sku: SKUData;
  fuTactics: Record<string, number>;
  columns: KPIColumn[];
  tacticColumns: TacticColumn[];
  onVolumeChange: (value: number) => void;
}

export function SKURow({
  sku,
  fuTactics,
  columns,
  tacticColumns,
  onVolumeChange
}: SKURowProps) {
  return (
    <tr className="sku-row hover:bg-gray-50">
      {/* Indented name */}
      <td className="sticky left-0 bg-white pl-10">
        <span className="text-gray-600">↳ {sku.sku_code} - {sku.sku_name}</span>
      </td>

      {/* Master Data */}
      <td className="text-right">{formatCurrency(sku.list_price)}</td>
      <td className="text-right">{formatCurrency(sku.cogs_per_piece)}</td>

      {/* Volume */}
      <td className="text-right text-gray-600">{formatNumber(sku.base_volume)}
      </td>
      <td className="bg-yellow-50">
        {/* EDITABLE volume input */}
        <EditableCell
          value={sku.planned_volume}
          type="number"
          min={0}
          onChange={onVolumeChange}
        />
      </td>

      {/* Dynamic KPI columns (calculated, animated) */}

```

```

<td key={col.kpiCode} className="text-right">
  <AnimatedValue
    value={sku.calculated[col.kpiCode]}
    format={col.displayFormat}
    decimals={col.decimalPlaces}
    highlightChange={true}
  />
</td>
)}}

```

```

{/* Tactic columns (READ-ONLY, inherited from FU) */}
{tacticColumns.map(tactic => (
  <td key={tactic.code} className="text-right text-gray-500 bg-gray-50">
    {fuTactics[tactic.code] ? (
      <>
        {formatTacticValue(fuTactics[tactic.code], tactic.inputType)}
        <span className="text-xs ml-1">(from FU)</span>
      </>
    ) : '-'}
  </td>
)}}

```

```

{/* ROI with RAG */}
<td className="sticky right-20 bg-white text-right">
  <RAGCell
    value={sku.calculated.GP_ROI_PCT}
    ragStatus={sku.calculated.RAG_STATUS}
  />
</td>
<td className="sticky right-0 bg-white">
  <RAGBadge status={sku.calculated.RAG_STATUS} />
</td>
</tr>
);
}
...

```

```

```typescript
// EditableCell.tsx
interface EditableCellProps {
 value: number;

```

```
min?: number;
max?: number;
onChange: (value: number) => void;
}
```

```
export function EditableCell({
 value,
 type,
 min,
 max,
 onChange
}: EditableCellProps) {
 const [localValue, setLocalValue] = useState(String(value));
 const [isEditing, setIsEditing] = useState(false);
 const [error, setError] = useState<string | null>(null);
 const inputRef = useRef<HTMLInputElement>(null);
```

```
// Update local value when prop changes (if not editing)
useEffect(() => {
 if (!isEditing) {
 setLocalValue(String(value));
 }
}, [value, isEditing]);
```

```
const handleFocus = () => {
 setIsEditing(true);
 setTimeout(() => inputRef.current?.select(), 0);
};
```

```
const handleBlur = () => {
 setIsEditing(false);
 validateAndSave();
};
```

```
const handleKeyDown = (e: React.KeyboardEvent) => {
 if (e.key === 'Enter') {
 validateAndSave();
 inputRef.current?.blur();
 } else if (e.key === 'Escape') {
 setLocalValue(String(value));
 }
};
```

```

inputRef.current?.blur();
}
};

const validateAndSave = () => {
const numValue = parseFloat(localValue);

if (isNaN(numValue)) {
setError('Invalid number');
return;
}

if (min !== undefined && numValue < min) {
setError(`Minimum: ${min}`);
return;
}

if (max !== undefined && numValue > max) {
setError(`Maximum: ${max}`);
return;
}

setError(null);
onChange(numValue);
};

const displayValue = isEditing
? localValue
: type === 'percentage'
? `${value}%`
: type === 'currency'
? `$$${value.toLocaleString()}`
: value.toLocaleString();

return (
<div className="relative">
<input
ref={inputRef}
type="text"
value={displayValue}

```

```

onFocus={handleFocus}
onBlur={handleBlur}
onKeyDown={handleKeyDown}
className={`
w-full px-2 py-1 text-right border rounded
focus:outline-none focus:ring-2 focus:ring-blue-500
${error ? 'border-red-500 bg-red-50' : 'border-gray-300'}
`}
/>
{error && (
<div className="absolute top-full left-0 mt-1 text-xs text-red-600 z-10
bg-white border border-red-300 px-2 py-1 rounded shadow">
 △ {error}
</div>
)}
</div>
);
}
` ``

```

PROVIDE:

1. **\*\*All component files:\*\***
  - PlanningGrid.tsx (main container)
  - DynamicGridHeader.tsx
  - FURow.tsx
  - SKURow.tsx
  - EditableCell.tsx
  - AnimatedValue.tsx
  - RAGCell.tsx
  - RAGBadge.tsx
  - FormattedValue.tsx
2. **\*\*CSS/Tailwind classes for:\*\***
  - Sticky columns (left and right)
  - Horizontal scroll
  - Zebra striping
  - Hover effects
  - FU row styling (blue background)
  - Editable cell highlighting

### 3. **\*\*Keyboard navigation:\*\***

- Tab to move between editable cells
- Arrow keys to navigate grid
- Enter to edit, Esc to cancel

### 4. **\*\*Helper functions:\*\***

```typescript

```
function formatNumber(value: number): string;  
function formatCurrency(value: number): string;  
function formatPercentage(value: number): string;  
function formatTacticValue(value: number, type: string): string;  
```
```

### 5. **\*\*Responsive design:\*\***

- Desktop: Full grid
- Tablet: Horizontal scroll
- Mobile: Card-based layout (simplified view)

USE:

- React 18+ with TypeScript
- Tailwind CSS for styling
- Hooks from previous prompts
- shadcn/ui components (optional, or plain HTML)

### PROMPT 8: Grand Totals Panel & Animated Values

Create the Grand Totals Panel component and AnimatedValue component for real-time visual feedback.

CONTEXT:

Above the planning grid, need a sticky panel showing plan-level aggregated KPIs that updates in real-time as users edit values.

REQUIREMENTS:

#### 1. **\*\*Grand Totals Panel\*\***

- Sticky at top (always visible)
- 6 metric cards in a row
- Real-time updates with smooth animations
- Color-coded based on performance

- Volume: Total planned volume, incremental volume, uplift %
- Turnover: Total planned turnover, incremental turnover
- Spend: Total spend, on-invoice vs off-invoice breakdown
- Profit: Incremental GP, base GP, planned GP
- Efficiency: GP ROI %, GP Margin %
- Status: Overall RAG status, item breakdown

### 3. **\*\*AnimatedValue Component\*\***

- Count-up animation when value changes
- Highlight in green if increased, red if decreased
- Smooth transitions

### 4. **\*\*RAG Visualization\*\***

- Large colored badge
- Bar showing distribution (% green, amber, red items)
- Budget impact indicator

#### COMPONENTS:

```
```typescript
```

```
// GrandTotalsPanel.tsx
```

```
interface GrandTotalsPanelProps {
  totals: Record<string, any>;
}
```

```
export function GrandTotalsPanel({ totals }: GrandTotalsPanelProps) {
  return (
    <div className="sticky top-0 z-10 bg-white border-b-4 border-blue-500 shadow-lg p-4">
      <h2 className="text-lg font-bold mb-3">PLAN PERFORMANCE OVERVIEW</h2>
      <div className="grid grid-cols-6 gap-4">
        { /* Volume Card */ }
        <MetricCard
          icon="📦 "
          title="VOLUME"
          mainValue={totals.PLANNED_VOL}
          mainLabel="Planned Total"
          mainFormat="number"
          mainUnit="pcs"
          secondaryValue={totals.INCR_VOL}
          secondaryLabel="Incremental"
```



```
tertiaryLabel="vs base"
tertiaryFormat="percentage"
/>
```

```
{/* Turnover Card */}
<MetricCard
icon="💰 "
title="TURNOVER"
mainValue={totals.PLANNED_T0}
mainLabel="Planned Total"
mainFormat="currency"
secondaryValue={totals.INCR_T0}
secondaryLabel="Incremental"
tertiaryValue={totals.T0_UPLIFT_PCT}
tertiaryLabel="vs base"
tertiaryFormat="percentage"
/>
```

```
{/* Spend Card */}
<MetricCard
icon="👛 "
title="SPEND"
mainValue={totals.TOTAL_PLANNED_SPEND}
mainLabel="Total Planned"
mainFormat="currency"
secondaryValue={totals.INCR_SPEND}
secondaryLabel="Incremental"
breakdown={{
  'On-Invoice': totals.TOTAL_ON_INVOICE_SPEND,
  'Off-Invoice': totals.TOTAL_OFF_INVOICE_SPEND
}}
/>
```

```
{/* Profit Card */}
<MetricCard
icon="💎 "
title="PROFIT"
mainValue={totals.INCR_GP}
mainLabel="Incremental GP"
mainFormat="currency"
```

```

secondaryLabel="Base GP"
tertiaryValue={totals.PLANNED_GP}
tertiaryLabel="Planned GP"
/>

```

```

{/* Efficiency Card */}
<MetricCard
  icon="🇫🇷"
  title="EFFICIENCY"
  mainValue={totals.GP_ROI_PCT}
  mainLabel="GP ROI"
  mainFormat="percentage"
  isHighlight={true}
  ragStatus={totals.RAG_STATUS}
  secondaryValue={totals.PLANNED_GM_PCT}
  secondaryLabel="GP Margin"
  secondaryFormat="percentage"
/>

```

```

{/* Overall Status Card */}
<StatusCard
  ragStatus={totals.RAG_STATUS}
  ragBreakdown={totals.RAG_BREAKDOWN} // { green: 16, amber: 3, red: 1 }
  budgetImpact={totals.BUDGET_IMPACT_PCT}
/>
</div>
</div>
);
}
...

```

```

```typescript
// MetricCard.tsx
interface MetricCardProps {
 icon: string;
 title: string;
 mainValue: number;
 mainLabel: string;
 mainFormat: 'number' | 'currency' | 'percentage';
 mainUnit?: string;
 secondaryValue?: number;

```

```

secondaryFormat?: string;
tertiaryValue?: number;
tertiaryLabel?: string;
tertiaryFormat?: string;
isHighlight?: boolean;
ragStatus?: string;
breakdown?: Record<string, number>;
}

```

```

export function MetricCard({
 icon,
 title,
 mainValue,
 mainLabel,
 mainFormat,
 mainUnit,
 secondaryValue,
 secondaryLabel,
 tertiaryValue,
 tertiaryLabel,
 tertiaryFormat,
 isHighlight,
 ragStatus,
 breakdown
}: MetricCardProps) {
 const bgColor = isHighlight && ragStatus
 ? ragStatus === 'GREEN' ? 'bg-green-100'
 : ragStatus === 'AMBER' ? 'bg-yellow-100'
 : 'bg-red-100'
 : 'bg-gray-50';

 return (
 <div className={`border rounded-lg p-3 ${bgColor}`}>
 {/* Header */}
 <div className="flex items-center gap-2 mb-2">
 {icon}
 {title}
 </div>

 {/* Main Value */}

```

```

<div className="text-xs text-gray-600">{mainLabel}</div>
<div className="text-2xl font-bold">
 <AnimatedValue
 value={mainValue}
 format={mainFormat}
 highlightChange={true}
 />
 {mainUnit} && {mainUnit}
</div>
</div>

```

```

{ /* Secondary Value */
{secondaryValue !== undefined && (
 <div className="text-sm mb-1">
 {secondaryLabel}:

 <AnimatedValue value={secondaryValue} format={mainFormat} />

 </div>
)}

```

```

{ /* Tertiary Value */
{tertiaryValue !== undefined && (
 <div className="text-sm">
 {tertiaryLabel}:

 <AnimatedValue
 value={tertiaryValue}
 format={tertiaryFormat || mainFormat}
 />

 </div>
)}

```

```

{ /* Breakdown */
{breakdown && (
 <div className="mt-2 pt-2 border-t text-xs space-y-1">
 {Object.entries(breakdown).map(([key, value]) => (
 <div key={key} className="flex justify-between">
 {key}:

```

```

<AnimatedValue value={value} format="currency" decimals={0} />

</div>
)}}
</div>
}}
</div>
);
}
...

```typescript
// AnimatedValue.tsx
interface AnimatedValueProps {
  value: number | null;
  format: 'number' | 'currency' | 'percentage';
  decimals?: number;
  highlightChange?: boolean;
}

export function AnimatedValue({
  value,
  format,
  decimals = 2,
  highlightChange = false
}: AnimatedValueProps) {
  const [displayValue, setDisplayValue] = useState(value);
  const [isAnimating, setIsAnimating] = useState(false);
  const prevValueRef = useRef(value);

  useEffect(() => {
    if (value !== prevValueRef.current && highlightChange && value !== null)
    {
      setIsAnimating(true);
      // Count-up animation
      const start = prevValueRef.current || 0;
      const end = value;
      const duration = 300; // ms
      const steps = 20;
      const increment = (end - start) / steps;
      let currentStep = 0;

```

```

const interval = setInterval(() => {
  currentStep++;
  if (currentStep >= steps) {
    setDisplayValue(end);
    clearInterval(interval);
    setTimeout(() => setIsAnimating(false), 500);
  } else {
    setDisplayValue(start + increment * currentStep);
  }
}, duration / steps);

prevValueRef.current = value;

return () => clearInterval(interval);
} else {
  setDisplayValue(value);
  prevValueRef.current = value;
}
}, [value, highlightChange]);

if (displayValue === null || displayValue === undefined) {
  return <span className="text-gray-400">-</span>;
}

const formatted = formatValue(displayValue, format, decimals);
const changeDirection =
(value || 0) > (prevValueRef.current || 0) ? 'up' : 'down';

return (
  <span
    className={`
      transition-all duration-300
      ${isAnimating
        ? changeDirection === 'up'
        ? 'bg-green-200 text-green-800 px-1 rounded'
        : 'bg-red-200 text-red-800 px-1 rounded'
        : ''
      }
    `}
  >

```



```
);  
}
```

```
function formatValue(  
  value: number,  
  format: string,  
  decimals: number  
): string {  
  switch (format) {  
    case 'currency':  
      return new Intl.NumberFormat('en-US', {  
        style: 'currency',  
        currency: 'USD',  
        minimumFractionDigits: decimals,  
        maximumFractionDigits: decimals  
      }).format(value);  
    case 'percentage':  
      return `${value.toFixed(decimals)}%`;  
    case 'number':  
    default:  
      return new Intl.NumberFormat('en-US', {  
        minimumFractionDigits: decimals,  
        maximumFractionDigits: decimals  
      }).format(value);  
  }  
}
```

```typescript

// StatusCard.tsx

```
interface StatusCardProps {
 ragStatus: string;
 ragBreakdown: { green: number; amber: number; red: number };
 budgetImpact: number;
}
```

```
export function StatusCard({ ragStatus, ragBreakdown, budgetImpact }:
 StatusCardProps) {
 const total = ragBreakdown.green + ragBreakdown.amber +
 ragBreakdown.red;
```

```

const amberPct = (ragBreakdown.amber / total) * 100;
const redPct = (ragBreakdown.red / total) * 100;

return (
<div className="border rounded-lg p-3 bg-white">
<div className="text-sm font-bold text-gray-700 mb-3">OVERALL
STATUSContinue </div>
{/* Large RAG Badge */}
<div className="mb-3 text-center">
<div className={`
inline-block px-4 py-2 rounded-lg text-white font-bold text-lg
${ragStatus === 'GREEN' ? 'bg-green-500' :
ragStatus === 'AMBER' ? 'bg-yellow-500' : 'bg-red-500'}
`}>
{ragStatus === 'GREEN' ? '🟢' : ragStatus === 'AMBER' ? '🟡' : '🔴'}
{ragStatus}
</div>
<div className="text-xs text-gray-600 mt-1">
{ragStatus === 'GREEN' ? 'Excellent Performance' :
ragStatus === 'AMBER' ? 'Acceptable Performance' : 'Needs Attention'}
</div>
</div>

{/* Item Breakdown Bar */}
<div className="mb-3">
<div className="text-xs text-gray-600 mb-1">Item Breakdown:</div>
<div className="flex h-6 rounded overflow-hidden">
<div className="bg-green-500 flex items-center justify-center text-white
text-xs font-bold"
style={{ width: `${greenPct}%` }}
title={` ${ragBreakdown.green} Green items`}
>
{ragBreakdown.green > 0 && ragBreakdown.green}
</div>
<div
className="bg-yellow-500 flex items-center justify-center text-white
text-xs font-bold"
style={{ width: `${amberPct}%` }}
title={` ${ragBreakdown.amber} Amber items`}
>

```



```

</div>
<div
 className="bg-red-500 flex items-center justify-center text-white text-
 xs font-bold"
 style={{ width: `${redPct}%` }}
 title={`${ragBreakdown.red} Red items`}
>
 {ragBreakdown.red > 0 && ragBreakdown.red}
</div>
</div>
<div className="flex justify-between text-xs text-gray-600 mt-1">
 ✓ {ragBreakdown.green} Green
 ✗ {ragBreakdown.amber} Amber
 △ {ragBreakdown.red} Red
</div>
</div>

```

```

{/* Budget Impact */}
<div className="text-xs">
 <div className="flex justify-between mb-1">
 Budget Impact:
 {budgetImpact.toFixed(1)}%
 </div>
 <div className="w-full bg-gray-200 rounded-full h-2">
 <div
 className={`h-2 rounded-full ${
 budgetImpact < 80 ? 'bg-green-500' :
 budgetImpact < 95 ? 'bg-yellow-500' : 'bg-red-500'
 }`}
 style={{ width: `${Math.min(budgetImpact, 100)}%` }}
 />
 </div>
</div>

```

```

{/* Action Buttons */}
<div className="mt-3 space-y-1">
 <button className="w-full text-xs py-1 bg-gray-100 hover:bg-gray-200
 rounded">
 View Breakdown
 </button>

```

```

<button className="w-full text-xs py-1 bg-blue-100 hover:bg-blue-200
rounded text-blue-700">
Optimize Plan
</button>
</div>
</div>
);
}

```

PROVIDE:

1. **\*\*All component files:\*\***

- GrandTotalsPanel.tsx
- MetricCard.tsx
- AnimatedValue.tsx
- StatusCard.tsx

2. **\*\*Animation logic:\*\***

- Smooth count-up animation (300ms)
- Highlight changes with colored background
- Fade highlight after 500ms

3. **\*\*Responsive design:\*\***

- Desktop: 6 cards in a row
- Tablet: 3 cards in 2 rows
- Mobile: 1-2 cards per row, vertical scroll

4. **\*\*Calculation of totals object\*\*** in parent:

```

```typescript
// In usePlanCalculations hook
const grandTotals = useMemo(() => {
// Aggregate all FU calculated values
const allFUValues = fus.map(fu => fu.calculated);
const aggregated = calculationEngine.aggregateFU(allFUValues);
// Calculate RAG breakdown
let greenCount = 0, amberCount = 0, redCount = 0;
fus.forEach(fu => {
fu.skus.forEach(sku => {
const ragStatus = sku.calculated.RAG_STATUS;
if (ragStatus === 'GREEN') greenCount++;

```

```

else if (ragStatus === 'RED') redCount++;
});
});

return {
  ...aggregated,
  RAG_BREAKDOWN: { green: greenCount, amber: amberCount, red: redCount },
  BUDGET_IMPACT_PCT: (aggregated.TOTAL_PLANNED_SPEND / monthlyBudget) *
  100
};
}, [fus, monthlyBudget]);
```

```

#### 5. **\*\*Unit tests for AnimatedValue:\*\***

- Test count-up from 100 to 200
- Test count-down from 200 to 100
- Test null values
- Test rapid changes (should cancel previous animation)

#### USE:

- React 18+ with hooks
- Tailwind CSS
- TypeScript
- Smooth CSS transitions

#### PROMPT 9: RAG Status Components & Visualization

Create RAG (Red-Amber-Green) status visualization components for the planning grid.

#### CONTEXT:

Each SKU and FU has a RAG status based on GP ROI % thresholds configured by admins. Need to display this visually in the grid with colors, badges, and tooltips.

#### REQUIREMENTS:

1. **\*\*RAGCell Component\*\*** - Shows ROI value with colored background
2. **\*\*RAGBadge Component\*\*** - Shows colored status badge with label
3. **\*\*RAGTooltip Component\*\*** - Detailed breakdown on hover

COMPONENTS:

```typescript

// RAGCell.tsx

interface RAGCellProps {

value: number | null;

ragStatus: 'GREEN' | 'AMBER' | 'RED' | 'GRAY';

showTooltip?: boolean;

thresholds?: {

green: number;

amber: number;

red: number;

};

}

export function RAGCell({

value,

ragStatus,

showTooltip = true,

thresholds = { green: 20, amber: 10, red: 10 }

}: RAGCellProps) {

const [showDetail, setShowDetail] = useState(false);

if (value === null || value === undefined) {

return (

<div className="text-gray-400 text-center">

-

</div>

);

}

// Color based on RAG status

const bgColor =

ragStatus === 'GREEN' ? 'bg-green-100 text-green-800' :

ragStatus === 'AMBER' ? 'bg-yellow-100 text-yellow-800' :

ragStatus === 'RED' ? 'bg-red-100 text-red-800' :

'bg-gray-100 text-gray-600';

const icon =

ragStatus === 'GREEN' ? '●' :

```
ragStatus === 'RED' ? '●' : '●';
```

```
return (  
  <div  
    className="relative"  
    onMouseEnter={() => setShowDetail(true)}  
    onMouseLeave={() => setShowDetail(false)}  
  >  
    <div className={`${px-2 py-1 rounded font-semibold text-right  
      ${bgColor}`}>  
      {icon} {value.toFixed(2)}%  
    </div>  

```

```
    {/* Tooltip */}  
    {showTooltip && showDetail && (  
      <RAGTooltip  
        value={value}  
        ragStatus={ragStatus}  
        thresholds={thresholds}  
      />  
    )}  
  </div>  
);  
}  
...
```

```
````typescript  
// RAGBadge.tsx
interface RAGBadgeProps {
 status: 'GREEN' | 'AMBER' | 'RED' | 'GRAY';
 size?: 'sm' | 'md' | 'lg';
 showLabel?: boolean;
}
```

```
export function RAGBadge({
 status,
 size = 'md',
 showLabel = true
}: RAGBadgeProps) {
 const sizeClasses = {
 sm: 'px-2 py-1 text-xs',

```

```
lg: 'px-4 py-2 text-base'
};
```

```
const colorClasses =
status === 'GREEN'
? 'bg-green-500 text-white'
: status === 'AMBER'
? 'bg-yellow-500 text-white'
: status === 'RED'
? 'bg-red-500 text-white'
: 'bg-gray-400 text-white';
```

```
const icon =
status === 'GREEN' ? '✓' :
status === 'AMBER' ? '⚡' :
status === 'RED' ? '⚠' : '○';
```

```
const label =
status === 'GREEN' ? 'Excellent' :
status === 'AMBER' ? 'Acceptable' :
status === 'RED' ? 'Poor' : 'No Data';
```

```
return (
<div className={`
inline-flex items-center justify-center gap-1 rounded font-bold
${sizeClasses[size]} ${colorClasses}
`}>
{icon}
{showLabel && {label}}
</div>
);
}
...
```

```
```typescript
// RAGTooltip.tsx
interface RAGTooltipProps {
value: number;
ragStatus: string;
thresholds: {
green: number;
```

```

red: number;
};
}

```

```

export function RAGTooltip({ value, ragStatus, thresholds }:
RAGTooltipProps) {
return (
<div className="absolute z-50 top-full left-0 mt-2 w-64 bg-white border-
2 border-gray-300 rounded-lg shadow-xl p-3">
  { /* Header */ }
  <div className="font-bold text-sm mb-2 flex items-center justify-
between">
    <span>GP ROI Performance</span>
    <RAGBadge status={ragStatus as any} size="sm" showLabel={false} />
  </div>

```

```

  { /* Current Value */ }
  <div className="mb-3 p-2 bg-gray-50 rounded">
    <div className="text-xs text-gray-600">Current ROI</div>
    <div className="text-2xl font-bold">
      {value.toFixed(2)}%
    </div>
    <div className="text-xs text-gray-600">
      Status: <span className="font-semibold">{ragStatus}</span>
    </div>
  </div>

```

```

  { /* Threshold Breakdown */ }
  <div className="space-y-2 mb-3">
    <div className="text-xs font-semibold text-gray-700">Thresholds:</div>
    <div className={`flex items-center gap-2 ${value >= thresholds.green ?
'font-bold' : ''}`}>
      <div className="w-3 h-3 bg-green-500 rounded-full" />
      <span className="text-xs">≥ {thresholds.green}% = Green</span>
      {value >= thresholds.green && <span className="text-xs">– You are
here</span>}
    </div>

```

```

    <div className={`flex items-center gap-2 ${value >= thresholds.amber &&
value < thresholds.green ? 'font-bold' : ''}`}>

```

```

<span className="text-xs">{thresholds.amber}% - {thresholds.green}% =
Amber</span>
{value >= thresholds.amber && value < thresholds.green && <span
className="text-xs">← You are here</span>}
</div>

```

```

<div className={`flex items-center gap-2 ${value < thresholds.red ?
'font-bold' : ''}`}>
<div className="w-3 h-3 bg-red-500 rounded-full" />
<span className="text-xs">&lt; {thresholds.red}% = Red</span>
{value < thresholds.red && <span className="text-xs">← You are
here</span>}
</div>
</div>

```

```

{/* Visual Range */}
<div className="mb-3">
<div className="text-xs font-semibold text-gray-700 mb-2">Visual Range:
</div>
<RAGRangeIndicator value={value} thresholds={thresholds} />
</div>

```

```

{/* Guidance */}
<div className="text-xs text-gray-600 border-t pt-2">
{ragStatus === 'GREEN' && '✓ Excellent! This promotion is highly
profitable.'}
{ragStatus === 'AMBER' && '⚡ Acceptable but could be optimized for
better ROI.'}
{ragStatus === 'RED' && '⚠ Warning: Low ROI. Consider adjusting tactics
or volume.'}
</div>
</div>
);
}
...

```

```

````typescript
// RAGRangeIndicator.tsx
interface RAGRangeIndicatorProps {
value: number;
thresholds: {

```



```
amber: number;
red: number;
};
}
```

```
export function RAGRangeIndicator({ value, thresholds }:
RAGRangeIndicatorProps) {
 // Calculate percentages for visual display
 // Assume range from 0% to green threshold + 10% for visual purposes
 const maxRange = thresholds.green + 10;
 const redWidth = (thresholds.red / maxRange) * 100;
 const amberWidth = ((thresholds.green - thresholds.red) / maxRange) *
 100;
 const greenWidth = 100 - redWidth - amberWidth;
 // Calculate position of indicator
 const indicatorPosition = Math.min((value / maxRange) * 100, 100);

 return (
 <div className="relative">
 {/* Range Bar */}
 <div className="flex h-8 rounded overflow-hidden border border-gray-
 300">
 <div
 className="bg-red-300 flex items-center justify-center text-xs text-red-
 800"
 style={{ width: `${redWidth}%` }}
 >
 Red
 </div>
 <div
 className="bg-yellow-300 flex items-center justify-center text-xs text-
 yellow-800"
 style={{ width: `${amberWidth}%` }}
 >
 Amber
 </div>
 <div
 className="bg-green-300 flex items-center justify-center text-xs text-
 green-800"
 style={{ width: `${greenWidth}%` }}
 >
 Green
 </div>
 </div>
 </div>
);
}
```

Green

</div>

</div>

{/\* Current Value Indicator \*/}

<div

className="absolute top-0 bottom-0 w-1 bg-black"

style={{ left: `\${indicatorPosition}%` }}

>

<div className="absolute -top-6 left-1/2 -translate-x-1/2 text-xs font-bold whitespace-nowrap">

▼ {value.toFixed(1)}%

</div>

</div>

{/\* Threshold Labels \*/}

<div className="flex justify-between text-xs text-gray-600 mt-1">

<span>0%</span>

<span>{thresholds.red}%</span>

<span>{thresholds.green}%</span>

<span>{maxRange}%</span>

</div>

</div>

);

}

...

```typescript

// FormattedValue.tsx (Helper component for displaying any KPI)

interface FormattedValueProps {

value: number | null;

format: 'number' | 'currency' | 'percentage' | 'text';

decimals?: number;

className?: string;

}

export function FormattedValue({

value,

format,

decimals = 2,

className = ''

```

if (value === null || value === undefined) {
return <span className={`text-gray-400 ${className}`}>-</span>;
}

```

```

let formatted: string;

```

```

switch (format) {
case 'currency':
formatted = new Intl.NumberFormat('en-US', {
style: 'currency',
currency: 'USD',
minimumFractionDigits: decimals,
maximumFractionDigits: decimals
}).format(value);
break;

```

```

case 'percentage':
formatted = `${value.toFixed(decimals)}%`;
break;

```

```

case 'number':
formatted = new Intl.NumberFormat('en-US', {
minimumFractionDigits: decimals,
maximumFractionDigits: decimals
}).format(value);
break;

```

```

case 'text':
default:
formatted = String(value);
break;
}

```

```

return <span className={className}>{formatted}</span>;
}
...

```

PROVIDE:

1. ****All RAG component files****

- Click to show detailed modal with full explanation
- Hover to show tooltip
- Visual range indicator

3. ****Color system:****

- Green: #10B981 (bg), #065F46 (text)
- Amber: #F59E0B (bg), #92400E (text)
- Red: #EF4444 (bg), #991B1B (text)
- Gray: #9CA3AF (bg), #374151 (text)

4. ****Accessibility:****

- Use ARIA labels
- Color + icon (not just color)
- Keyboard navigation support

5. ****Example usage in grid:****

```
```typescript
// In SKU row
<td className="sticky right-20">
 <RAGCell
 value={sku.calculated.GP_ROI_PCT}
 ragStatus={sku.calculated.RAG_STATUS}
 thresholds={{ green: 20, amber: 10, red: 10 }}
 />
</td>
<td className="sticky right-0">
 <RAGBadge
 status={sku.calculated.RAG_STATUS}
 size="sm"
 />
</td>
```
```

6. ****Click to expand detail modal:****

```
```typescript
const [showModal, setShowModal] = useState(false);

// On RAG cell click
<RAGDetailModal
 isOpen={showModal}
```

```
value={value}
ragStatus={ragStatus}
itemName={sku.sku_name}
recommendations={[
 'Reduce CPP discount from 15% to 12%',
 'Increase planned volume to 5000 units',
 'Remove visibility lumpsum spend'
]}
/>
...

```

USE:

- React with TypeScript
- Tailwind CSS for styling
- Smooth transitions
- Mobile-friendly tooltips (tap instead of hover)

PROMPT 10: Integration & Testing

Create integration code and testing suite to ensure all components work together correctly.

CONTEXT:

All individual pieces are built. Now need to:

1. Wire everything together
2. Test end-to-end workflows
3. Handle edge cases
4. Add error boundaries

REQUIREMENTS:

1. **\*\*Main App Integration\*\***
  - Connect all hooks and components
  - Initialize calculation engine on app start
  - Handle authentication and permissions
2. **\*\*Error Handling\*\***
  - Error boundaries for React components
  - Graceful fallbacks for calculation errors
  - Toast notifications for user feedback

- Unit tests for calculation engine
- Integration tests for data flow
- E2E tests for user workflows

#### 4. **\*\*Performance Monitoring\*\***

- Track calculation time
- Monitor re-renders
- Optimize bottlenecks

#### MAIN APP STRUCTURE:

```

```typescript
// App.tsx
import { useEffect } from 'react';
import { calculationEngine } from '../services/calculationEngine';
import { PlanningScreen } from '../screens/PlanningScreen';
import { ErrorBoundary } from '../components/ErrorBoundary';
import { Toaster } from '../components/Toaster';

export function App() {
  // Initialize calculation engine on app start
  useEffect(() => {
    const init = async () => {
      try {
        console.log('🚀 Initializing calculation engine...');
        await calculationEngine.initialize();
        console.log('✅ Calculation engine ready');
      } catch (error) {
        console.error('❌ Failed to initialize:', error);
        // Show error toast
      }
    };

    init();
  }, []);

  return (
    <ErrorBoundary>
      <div className="app">
        <Toaster />
        <PlanningScreen planId="plan-123" />
      </div>
    </ErrorBoundary>
  );
}

```

```

</ErrorBoundary>
);
}
...

```typescript
// screens/PlanningScreen.tsx
interface PlanningScreenProps {
 planId: string;
}

export function PlanningScreen({ planId }: PlanningScreenProps) {
 const [planContext, setPlanContext] = useState({
 channel: 'NKA',
 category: 'Dairy'
 });

 return (
 <div className="planning-screen">
 {/* Header */}
 <header className="bg-blue-600 text-white p-4">
 <h1>Plan Detail: Summer Dairy Promotion</h1>
 <div className="text-sm">
 Channel: {planContext.channel} | Category: {planContext.category}
 </div>
 </header>

 {/* Main Grid */}
 <main className="p-4">
 <PlanningGrid
 planId={planId}
 planContext={planContext}
 />
 </main>
 </div>
);
}
...

```typescript
// components/ErrorBoundary.tsx
import React, { Component, ErrorInfo, ReactNode } from 'react';

```

```

interface Props {
  children: ReactNode;
}

interface State {
  hasError: boolean;
  error: Error | null;
}

export class ErrorBoundary extends Component<Props, State> {
  constructor(props: Props) {
    super(props);
    this.state = { hasError: false, error: null };
  }

  static getDerivedStateFromError(error: Error): State {
    return { hasError: true, error };
  }

  componentDidCatch(error: Error, errorInfo: ErrorInfo) {
    console.error('Error caught by boundary:', error, errorInfo);
    // Log to error tracking service (Sentry, etc.)
  }

  render() {
    if (this.state.hasError) {
      return (
        <div className="error-screen p-8 text-center">
          <h1 className="text-2xl font-bold text-red-600 mb-4">
            Something went wrong
          </h1>
          <p className="text-gray-600 mb-4">
            {this.state.error?.message}
          </p>
          <button
            onClick={() => window.location.reload()}
            className="px-4 py-2 bg-blue-500 text-white rounded hover:bg-blue-600"
          >
            Reload Page
          </button>
        </div>
      );
    }
  }
}

```



```
);  
}
```

```
return this.props.children;  
}  
}  
...
```

UNIT TESTS:

```
```typescript
```

```
// __tests__/formulaParser.test.ts
```

```
import { formulaParser } from '../services/formulaParser';
```

```
describe('FormulaParser', () => {
 describe('extractDependencies', () => {
 it('should extract variable names from formula', () => {
 const formula = "PLANNED_VOL - BASE_VOL";
 const deps = formulaParser.extractDependencies(formula);
 expect(deps).toEqual(['PLANNED_VOL', 'BASE_VOL']);
 });
 });
});
```

```
it('should handle complex formulas', () => {
 const formula = "(INCR_GP / INCR_SPEND) * 100";
 const deps = formulaParser.extractDependencies(formula);
 expect(deps).toEqual(['INCR_GP', 'INCR_SPEND']);
});
```

```
it('should exclude function names', () => {
 const formula = "IF(GP_ROI >= 20, 'GREEN', 'RED')";
 const deps = formulaParser.extractDependencies(formula);
 expect(deps).toEqual(['GP_ROI']);
});
});
```

```
describe('parseExpression', () => {
 it('should calculate simple subtraction', () => {
 const parsed = formulaParser.parseFormula("PLANNED_VOL - BASE_VOL",
 "expression");
 const result = parsed.execute({ PLANNED_VOL: 4200, BASE_VOL: 3200 });
 expect(result).toBe(1000);
 });
});
```

```
it('should calculate percentage', () => {
 const parsed = formulaParser.parseFormula("(INCR_VOL / BASE_VOL) * 100",
 "expression");
 const result = parsed.execute({ INCR_VOL: 1000, BASE_VOL: 3200 });
 expect(result).toBeCloseTo(31.25, 2);
});
```

```
it('should handle division by zero', () => {
 const parsed = formulaParser.parseFormula("100 / ZERO_VALUE",
 "expression");
 const result = parsed.execute({ ZERO_VALUE: 0 });
 expect(result).toBeNull();
});
```

```
it('should handle missing dependencies', () => {
 const parsed = formulaParser.parseFormula("PLANNED_VOL - BASE_VOL",
 "expression");
 const result = parsed.execute({ PLANNED_VOL: 4200 }); // BASE_VOL
 missing
 expect(result).toBeNull();
});
```

```
describe('parseConditional', () => {
 it('should evaluate simple IF', () => {
 const parsed = formulaParser.parseFormula(
 "IF(GP_ROI >= 20, 'GREEN', 'RED')",
 "conditional"
);
 expect(parsed.execute({ GP_ROI: 25 })).toBe('GREEN');
 expect(parsed.execute({ GP_ROI: 15 })).toBe('RED');
 });
});
```

```
it('should evaluate nested IF', () => {
 const parsed = formulaParser.parseFormula(
 "IF(GP_ROI >= 20, 'GREEN', IF(GP_ROI >= 10, 'AMBER', 'RED'))",
 "conditional"
);
 expect(parsed.execute({ GP_ROI: 25 })).toBe('GREEN');
```

```

expect(parsed.execute({ GP_ROI: 5 })).toBe('RED');
});
});

describe('validateFormula', () => {
 it('should validate correct formula', () => {
 const result = formulaParser.validateFormula("PLANNED_VOL - BASE_VOL",
 "expression");
 expect(result.isValid).toBe(true);
 expect(result.errors).toHaveLength(0);
 });

 it('should detect invalid syntax', () => {
 const result = formulaParser.validateFormula("PLANNED_VOL - -",
 "expression");
 expect(result.isValid).toBe(false);
 expect(result.errors.length).toBeGreaterThan(0);
 });
});
});
```
```typescript
// __tests__/calculationEngine.test.ts
import { calculationEngine } from '../services/calculationEngine';

describe('CalculationEngine', () => {
 beforeAll(async () => {
 await calculationEngine.initialize();
 });

 describe('calculateSKU', () => {
 it('should calculate all KPIs for a SKU', () => {
 const context = {
 planned_volume: 4200,
 base_volume: 3200,
 list_price: 4.00,
 cogs_per_piece: 1.80,
 tactic_values: {
 CPP_ON_PCT: 10
 }
 }
 }
 }
});

```

```

const results = calculationEngine.calculateSKU(context);

expect(results.INCR_VOL.value).toBe(1000);
expect(results.VOL_UPLIFT_PCT.value).toBeCloseTo(31.25, 2);
expect(results.BASE_GSV.value).toBe(12800);
expect(results.PLANNED_GSV.value).toBe(16800);
});

it('should handle zero base volume', () => {
 const context = {
 planned_volume: 5000,
 base_volume: 0, // New product
 list_price: 4.00,
 cogs_per_piece: 1.80
 };

 const results = calculationEngine.calculateSKU(context);

 expect(results.INCR_VOL.value).toBe(5000);
 expect(results.VOL_UPLIFT_PCT.value).toBeNull(); // Can't calculate uplift
});

describe('recalculate', () => {
 it('should only recalculate affected KPIs', () => {
 const initialContext = {
 PLANNED_VOL: 4200,
 BASE_VOL: 3200,
 BPTT: 4.00,
 INCR_VOL: 1000,
 BASE_GSV: 12800,
 PLANNED_GSV: 16800
 };

 const updates = calculationEngine.recalculate(
 'PLANNED_VOL',
 5000,
 initialContext
);
 });
});

```

```

// Should recalculate INCR_VOL and dependent KPIs
expect(updates).toHaveProperty('INCR_VOL');
expect(updates).toHaveProperty('PLANNED_GSV');
expect(updates).toHaveProperty('VOL_UPLIFT_PCT');
// Should NOT recalculate independent KPIs
expect(updates).not.toHaveProperty('BASE_VOL');
expect(updates).not.toHaveProperty('BASE_GSV');
});
});

```

```

describe('aggregateFU', () => {
 it('should sum SKU values to FU level', () => {
 const skuResults = [
 { INCR_VOL: { value: 1000 }, PLANNED_GP: { value: 2500 } },
 { INCR_VOL: { value: 750 }, PLANNED_GP: { value: 1800 } },
 { SContinueINCR_VOL: { value: 500 }, PLANNED_GP: { value: 1200 } }
];
 const fuResults = calculationEngine.aggregateFU(skuResults);

```

```

 expect(fuResults.INCR_VOL).toBe(2250); // 1000 + 750 + 500
 expect(fuResults.PLANNED_GP).toBe(5500); // 2500 + 1800 + 1200
 });

```

```

 it('should handle different aggregation methods', () => {
 // Assuming some KPIs use 'avg' instead of 'sum'
 const skuResults = [
 { INCR_VOL: { value: 1000 }, GP_MARGIN_PCT: { value: 25.5 } },
 { INCR_VOL: { value: 750 }, GP_MARGIN_PCT: { value: 30.2 } },
 { INCR_VOL: { value: 500 }, GP_MARGIN_PCT: { value: 22.8 } }
];

```

```

 const fuResults = calculationEngine.aggregateFU(skuResults);

```

```

 expect(fuResults.INCR_VOL).toBe(2250); // sum
 expect(fuResults.GP_MARGIN_PCT).toBeCloseTo(26.17, 2); // avg
 });
});
describe('formatValue', () => {
 it('should format currency', () => {

```

```

const formatted = calculationEngine['formatValue'](12500.50, kpi);
expect(formatted).toBe('$12,500.50');
});

it('should format percentage', () => {
const kpi = { display_format: 'percentage', decimal_places: 2 };
const formatted = calculationEngine['formatValue'](25.456, kpi);
expect(formatted).toBe('25.46%');
});

it('should format number with thousand separator', () => {
const kpi = { display_format: 'number', decimal_places: 0 };
const formatted = calculationEngine['formatValue'](123456, kpi);
expect(formatted).toBe('123,456');
});
});
````typescript
// __tests__/calculationGraph.test.ts
import { calculationGraph } from '../services/calculationGraph';

describe('CalculationGraph', () => {
const sampleKPIs = [
{ kpi_code: 'PLANNED_VOL', depends_on_kpis: [] },
{ kpi_code: 'BASE_VOL', depends_on_kpis: [] },
{ kpi_code: 'BPTT', depends_on_kpis: [] },
{ kpi_code: 'INCR_VOL', depends_on_kpis: ['PLANNED_VOL', 'BASE_VOL'] },
{ kpi_code: 'BASE_GSV', depends_on_kpis: ['BASE_VOL', 'BPTT'] },
{ kpi_code: 'PLANNED_GSV', depends_on_kpis: ['PLANNED_VOL', 'BPTT'] },
{ kpi_code: 'INCR_GSV', depends_on_kpis: ['PLANNED_GSV', 'BASE_GSV'] },
{ kpi_code: 'VOL_UPLIFT_PCT', depends_on_kpis: ['INCR_VOL', 'BASE_VOL'] }
];

beforeEach(async () => {
await calculationGraph.buildGraph(sampleKPIs);
});

describe('buildGraph', () => {
it('should build graph with correct nodes', () => {
const order = calculationGraph.getCalculationOrder();

```

```

expect(order).toContain('INCR_VOL');
expect(order).toContain('INCR_GSV');
});

it('should calculate correct levels', () => {
const plannedVolNode = calculationGraph.getNode('PLANNED_VOL');
const incrVolNode = calculationGraph.getNode('INCR_VOL');
const incrGsvNode = calculationGraph.getNode('INCR_GSV');

expect(plannedVolNode?.level).toBe(0);
expect(incrVolNode?.level).toBe(1);
expect(incrGsvNode?.level).toBe(3); // Depends on PLANNED_GSV and
BASE_GSV (level 2)
});

describe('getCalculationOrder', () => {
it('should return topologically sorted order', () => {
const order = calculationGraph.getCalculationOrder();
// PLANNED_VOL and BASE_VOL should come before INCR_VOL
const plannedIdx = order.indexOf('PLANNED_VOL');
const baseIdx = order.indexOf('BASE_VOL');
const incrIdx = order.indexOf('INCR_VOL');

expect(plannedIdx).toBeLessThan(incrIdx);
expect(baseIdx).toBeLessThan(incrIdx);
});

it('should calculate dependencies before dependents', () => {
const order = calculationGraph.getCalculationOrder();
// BASE_GSV and PLANNED_GSV should come before INCR_GSV
const baseGsvIdx = order.indexOf('BASE_GSV');
const plannedGsvIdx = order.indexOf('PLANNED_GSV');
const incrGsvIdx = order.indexOf('INCR_GSV');

expect(baseGsvIdx).toBeLessThan(incrGsvIdx);
expect(plannedGsvIdx).toBeLessThan(incrGsvIdx);
});
});

```

```

it('should return all dependent KPIs when input changes', () => {
  const affected = calculationGraph.getAffectedKPIs('PLANNED_VOL');
  expect(affected).toContain('INCR_VOL');
  expect(affected).toContain('PLANNED_GSV');
  expect(affected).toContain('VOL_UPLIFT_PCT');
  expect(affected).toContain('INCR_GSV');
});

it('should not include the changed KPI itself', () => {
  const affected = calculationGraph.getAffectedKPIs('PLANNED_VOL');
  expect(affected).not.toContain('PLANNED_VOL');
});

it('should not include independent KPIs', () => {
  const affected = calculationGraph.getAffectedKPIs('PLANNED_VOL');
  expect(affected).not.toContain('BASE_VOL');
  expect(affected).not.toContain('BASE_GSV');
});

it('should return empty array for leaf nodes', () => {
  const affected = calculationGraph.getAffectedKPIs('INCR_GSV');
  expect(affected).toHaveLength(0);
});

describe('circular dependency detection', () => {
  it('should throw error for circular dependencies', async () => {
    const circularKPIs = [
      { kpi_code: 'A', depends_on_kpis: ['B'] },
      { kpi_code: 'B', depends_on_kpis: ['C'] },
      { kpi_code: 'C', depends_on_kpis: ['A'] } // Circular!
    ];

    await
    expect(calculationGraph.buildGraph(circularKPIs)).rejects.toThrow('Circular dependency');
  });
});

```


INTEGRATION TEST:

```
````typescript
// __tests__/integration/planCalculations.test.tsx
import { renderHook, act, waitFor } from '@testing-library/react';
import { usePlanCalculations } from '../../hooks/usePlanCalculations';

describe('Plan Calculations Integration', () => {
 it('should calculate KPIs when SKU volume changes', async () => {
 const { result } = renderHook(() => usePlanCalculations('test-plan-1'));

 // Wait for initial load
 await waitFor(() => {
 expect(result.current.fus.length).toBeGreaterThan(0);
 });

 const initialGrandTotal = result.current.grandTotals.GP_ROI_PCT;

 // Update SKU volume
 act(() => {
 result.current.updateSKUVolume('fu-1', 'sku-1', 5000);
 });

 // Should recalculate
 await waitFor(() => {
 expect(result.current.grandTotals.GP_ROI_PCT).not.toBe(initialGrandTotal);
 });

 // Check that calculations cascaded
 const fu = result.current.fus.find(f => f.fu_id === 'fu-1');
 const sku = fu?.skus.find(s => s.sku_id === 'sku-1');

 expect(sku?.planned_volume).toBe(5000);
 expect(sku?.calculated.INCR_VOL).toBeDefined();
 expect(sku?.calculated.GP_ROI_PCT).toBeDefined();
 });

 it('should aggregate SKU values to FU level', async () => {
 const { result } = renderHook(() => usePlanCalculations('test-plan-1'));
 });
});
```

```

expect(result.current.fus.length).toBeGreaterThan(0);
});

const fu = result.current.fus[0];
// FU planned volume should be sum of SKU volumes
const expectedTotal = fu.skus.reduce((sum, sku) => sum +
sku.planned_volume, 0);
expect(fu.calculated.PLANNED_VOL).toBe(expectedTotal);
});

it('should recalculate all SKUs when FU tactic changes', async () => {
const { result } = renderHook(() => usePlanCalculations('test-plan-1'));

await waitFor(() => {
expect(result.current.fus.length).toBeGreaterThan(0);
});

const fu = result.current.fus[0];
const initialSKUGP = fu.skus[0].calculated.PLANNED_GP;

// Update FU tactic
act(() => {
result.current.updateFUTactic('fu-1', 'CPP_ON_PCT', 15); // Increase
from 10% to 15%
});

// All SKUs should recalculate
await waitFor(() => {
const updatedFU = result.current.fus.find(f => f.fu_id === 'fu-1');
const updatedSKU = updatedFU?.skus[0];
expect(updatedSKU?.calculated.PLANNED_GP).not.toBe(initialSKUGP);
});
});

it('should auto-save after debounce period', async () => {
jest.useFakeTimers();
const { result } = renderHook(() => usePlanCalculations('test-plan-1'));

await waitFor(() => {
expect(result.current.fus.length).toBeGreaterThan(0);

```

```

// Make change
act(() => {
 result.current.updateSKUVolume('fu-1', 'sku-1', 5000);
});

// Fast-forward past debounce (2 seconds)
act(() => {
 jest.advanceTimersByTime(2500);
});

// Should have triggered save (check console or mock Supabase)
// This would require mocking Supabase client

jest.useRealTimers();
});
});
```

```

E2E TEST (using Playwright or Cypress):

```

```typescript
// e2e/planning-grid.spec.ts
import { test, expect } from '@playwright/test';

test.describe('Planning Grid', () => {
 test.beforeEach(async ({ page }) => {
 await page.goto('/plan/test-plan-1');
 await page.waitForSelector('.planning-grid-table');
 });

 test('should display grand totals panel', async ({ page }) => {
 const grandTotals = page.locator('.grand-totals-panel');
 await expect(grandTotals).toBeVisible();
 // Check for key metrics
 await expect(grandTotals.locator('text=VOLUME')).toBeVisible();
 await expect(grandTotals.locator('text=GP ROI')).toBeVisible();
 });

 test('should expand/collapse FU rows', async ({ page }) => {
 // Initially collapsed

```

```

await expect(skuRow).not.toBeVisible();

// Click expand button
await page.locator('button:has-text("+ ")').first().click();

// SKU rows should now be visible
await expect(skuRow).toBeVisible();
});

test('should update calculations when volume changes', async ({ page })
=> {
// Expand first FU
await page.locator('button:has-text("+ ")').first().click();

// Get initial ROI value
const roiCell = page.locator('tr.sku-row').first().locator('td.rag-
cell');
const initialRoi = await roiCell.textContent();

// Change planned volume
const volumeInput = page.locator('tr.sku-
row').first().locator('input[type="text"]').first();
await volumeInput.click();
await volumeInput.fill('5000');
await volumeInput.blur();

// Wait for calculation
await page.waitForTimeout(500);

// ROI should have changed
const updatedRoi = await roiCell.textContent();
expect(updatedRoi).not.toBe(initialRoi);

// Grand totals should update
const grandTotalRoi = page.locator('.grand-totals-
panel').locator('text=/GP ROI.%%/');
await expect(grandTotalRoi).toBeVisible();
});

test('should show RAG tooltip on hover', async ({ page }) => {
await page.locator('button:has-text("+ ")').first().click();

```

```
const ragCell = page.locator('tr.sku-row').first().locator('.rag-cell');
await ragCell.hover();
```

```
// Tooltip should appear
const tooltip = page.locator('.rag-tooltip');
await expect(tooltip).toBeVisible();
await expect(tooltip.locator('text=Thresholds')).toBeVisible();
});
```

```
test('should highlight changes with animation', async ({ page }) => {
 await page.locator('button:has-text("⊕")').first().click();
```

```
 const volumeInput = page.locator('tr.sku-row').first().locator('input[type="text"]').first();
 await volumeInput.click();
 await volumeInput.fill('6000');
 await volumeInput.blur();
```

```
 // Should have highlight class during animation
 const animatedValue = page.locator('tr.sku-row').first().locator('.bg-green-200');
 await expect(animatedValue).toBeVisible();
```

```
 // Highlight should fade after animation
 await page.waitForTimeout(1000);
 await expect(animatedValue).not.toBeVisible();
});
```

```
test('should validate input ranges', async ({ page }) => {
 await page.locator('button:has-text("⊕")').first().click();
```

```
 // Try to enter negative volume
 const volumeInput = page.locator('tr.sku-row').first().locator('input[type="text"]').first();
 await volumeInput.click();
 await volumeInput.fill('-100');
 await volumeInput.blur();
```

```
 // Should show error
 const errorMessage = page.locator('text=/Minimum/');
```

```
});
```

```
test('should show only applicable tactics for plan context', async ({
 page }) => {
 // Check that only NKA-applicable tactics are shown
 const header = page.locator('thead');
 // Should show CPP On-Invoice (applies to NKA)
 await expect(header.locator('text=CPP On-Invoice')).toBeVisible();
 // Should NOT show WS TPR (only for Wholesale)
 await expect(header.locator('text=WS TPR')).not.toBeVisible();
});
});
````
```

PERFORMANCE MONITORING:

```
````typescript
// utils/performanceMonitor.ts
export class PerformanceMonitor {
 private metrics: Map<string, number[]> = new Map();

 startMeasure(label: string): () => void {
 const startTime = performance.now();
 return () => {
 const endTime = performance.now();
 const duration = endTime - startTime;
 if (!this.metrics.has(label)) {
 this.metrics.set(label, []);
 }
 this.metrics.get(label)!.push(duration);
 if (duration > 500) {
 console.warn(`⚠ Slow operation: ${label} took
 ${duration.toFixed(2)}ms`);
 } else {
 console.log(`🕒 ${label}: ${duration.toFixed(2)}ms`);
 }
 };
 }

 getStats(label: string) {
 const values = this.metrics.get(label) || [];
 }
}
```

```

const sum = values.reduce((a, b) => a + b, 0);
const avg = sum / values.length;
const min = Math.min(...values);
const max = Math.max(...values);

return { avg, min, max, count: values.length };
}

```

```

logAllStats() {
 console.table(
 Array.from(this.metrics.keys()).map(label => ({
 operation: label,
 ...this.getStats(label)
 }))
);
}
}

```

```

export const perfMonitor = new PerformanceMonitor();

```

```

// Usage in calculation engine
async calculateSKU(context: CalculationContext) {
 const endMeasure = perfMonitor.startMeasure('calculateSKU');
 try {
 // ... calculation logic
 return results;
 } finally {
 endMeasure();
 }
}
}
```

```

TOAST NOTIFICATION SYSTEM:

```

```typescript
// components/Toaster.tsx
import { useState, useEffect } from 'react';
import { toast } from 'react-hot-toast';

export function Toaster() {

```

```

<div className="toaster-container">
 {/* Use react-hot-toast or custom implementation */}
</div>
);
}

// Usage in components
import { toast } from 'react-hot-toast';

// Success
toast.success('Plan saved successfully');

// Error
toast.error('Failed to calculate KPIs');

// Warning
toast.warning('Some items have negative ROI');

// Info
toast('Calculation completed', { icon: 'i' });
```

```

PROVIDE:

1. ****Complete integration files****
2. ****All test files**** (unit, integration, e2e)
3. ****Test coverage report**** showing >80% coverage
4. ****Performance benchmarks****
 - Initial load: < 2s
 - SKU volume update: < 100ms
 - FU tactic update: < 300ms
 - Save to database: < 1s
5. ****Error handling examples****
 - Network errors (retry logic)
 - Calculation errors (fallback to null)
 - Permission errors (redirect to access denied)
6. ****CI/CD configuration**** (GitHub Actions):


```
```yaml
```



```
on: [push, pull_request]
jobs:
 test:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v2
 - uses: actions/setup-node@v2
 - run: npm ci
 - run: npm run test
 - run: npm run test:e2e
 - run: npm run build
 ...
```

#### USE:

- Jest for unit tests
- React Testing Library for integration tests
- Playwright or Cypress for E2E tests
- TypeScript strict mode
- 80%+ code coverage requirement

#### SUMMARY: Complete Prompt Sequence

You now have 10 sequential prompts to build the dynamic TPM calculation system:

- ✓ Database Schema - Foundation for dynamic KPIs
- ✓ Formula Parser - Parse and execute formula strings
- ✓ Calculation Graph - Dependency management
- ✓ Calculation Engine - Core calculation logic
- ✓ React Hook - State management and real-time updates
- ✓ Dynamic Columns - Context-aware column configuration
- ✓ Grid Components - Planning grid UI with FU/SKU hierarchy
- ✓ Grand Totals & Animation - Visual feedback components
- ✓ RAG Status Components - Performance visualization
- ✓ Integration & Testing - Tie everything together

Each prompt:

Is focused and buildable independently  
Has clear dependencies on previous prompts  
Includes examples and test cases

Can be used with AI code generators (Claude, ChatGPT, Gemini)